A Survey of Reinforcement Learning For Economics

Pranjal Rawat, Georgetown University*

March 2026

Abstract

This chapter introduces reinforcement learning methods to researchers in the social sciences. The curse of dimensionality limits exact dynamic programming, forcing reliance on tractable reductions that a growing class of economic models resists. Reinforcement learning algorithms extend dynamic programming to problems with high-dimensional states, continuous actions, and strategic interactions through sample-based approximation. I review the theory connecting classical planning to modern learning algorithms, covering value-based methods, policy gradients, and their convergence properties. Applications include structural estimation of dynamic models, equilibrium computation in games of imperfect information, and preference learning from human feedback. I also examine the practical vulnerabilities of these methods, including brittleness, sample inefficiency, sensitivity to hyperparameters, and the absence of convergence guarantees outside of tabular settings. A companion survey (Rust and Rawat, 2026) covers the inverse problem of inferring preferences from observed behavior. Simulation code is publicly available.[1]

Keywords: Reinforcement Learning, Economics, Structural Estimation, Multi-Agent, RLHF

*I am grateful to John Rust for encouraging me to undertake this survey. I thank Simon LeBastard for sharing his RL notes with me.

[1] https://github.com/rawatpranjal/survey-of-reinforcement-learning-in-economics

Contents

1 Introduction

This chapter introduces reinforcement learning to researchers studying sequential decision problems. I review the theoretical connections between dynamic programming and reinforcement learning, demonstrating how value iteration, Q-learning, and policy gradient methods are common solution methods to the same class of optimization problems. I then examine applications across several domains, including structural economic models with high-dimensional state spaces; strategic games in which multi-agent algorithms compute equilibria under imperfect information; and preference learning from human feedback. The exposition combines formal theory, practical applications and computational illustration.

Both dynamic programming and reinforcement learning solve the Bellman equation; they differ in the information requirements and the way in which the solution is refined. First, dynamic programming requires knowledge of the transition in the environment and the reward function which allows the reduction of the average Bellman error, reinforcement learning estimates value functions only from sampled transitions (observed sets of state, action, reward, next-state) which only allows reduction of the sampled Bellman error *at that state-action pair*. This allows improving policies in domains where it is easier to build a simulator than specify the model of the environment and rewards, such as board games and physics simulators for robots. Second, dynamic programming makes a "breadth-first" (across all states and actions) update of the solution at each sweep, while reinforcement learning makes an "incremental" (only for the current state and action) update; this greatly reduces the computational burden and enhances scalability.

Reduction of average Bellman errors gives dynamic programming a geometric rate of convergence to the optimal solution, while the incremental updates and reduction of sampled Bellman errors, when combined with "sufficient exploration" of the state-action space, gives reinforcement learning only sublinear convergence guarantees. This is, however, sufficient in practice; the scalability attained by sampling transitions and making incremental updates more than compensates for the slower convergence rates. These approximation methods sacrifice theoretical guarantees. RL algorithms lack the convergence assurances of exact dynamic programming. They exhibit sensitivity to hyperparameters and initialization. They can converge to suboptimal policies without diagnostic indication. This chapter presents reinforcement learning as a computationally flexible framework while acknowledging its methodological limitations.

Theory in reinforcement learning trails empirical success, often by years. Details matter disproportionately; small implementation choices can determine whether an algorithm converges or diverges.

Throughout, $s \in \mathcal{S}$ denotes the state, $a \in \mathcal{A}$ the action, $\pi(a|s)$ the policy (decision rule), $P(s'|s,a)$ the transition kernel, $r(s,a)$ the per-period reward (utility), and $\gamma \in [0,1)$ the discount factor. The value function $V^\pi(s) = \mathbb{E}_\pi[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s]$ is the expected discounted return under policy π; the action-value function $Q^\pi(s,a)$ conditions additionally on the first action. Table 2 provides a complete mapping to econometric equivalents.

This chapter focuses on intersections between reinforcement learning and applied decision-making, including theoretical foundations, structural estimation, strategic interaction, and preference learning. Adjacent topics (algorithmic collusion (Calvano et al., 2020; Rawat, 2026), macroeconomic solution methods (Fernández-Villaverde et al., 2024), portfolio optimization (Hambly et al., 2023), and inverse reinforcement learning (Rust and Rawat, 2026)) are treated in companion work and dedicated surveys.

The chapter addresses the forward problem, that is, computing optimal policies given a known or simulated environment. Section 2 develops reinforcement learning algorithms. Section 3 develops unified theory connecting planning and learning. Section 4 applies reinforcement learning to structural estimation. Section 5 examines strategic games. Section 6 addresses offline reinforcement learning and human feedback. Section 7 examines offline RL under unobserved confounding. Section 8 concludes.

1.1 Two Cultures of Sequential Decision-Making

Two intellectual traditions study sequential decision-making under uncertainty. The economics tradition is fundamentally an *inference culture*. Its central task is to understand the world. A "model" in this tradition is a specification of preferences, beliefs, constraints, and an equilibrium concept. The RL tradition is instead a *control culture*. Its central task is to act in the world. An RL researcher's "model" is a transition kernel $P(s'|s,a)$ and a reward function $r(s,a)$. These are different mathematical objects serving different scientific purposes.

The inference tradition concentrates its effort on specifying and estimating the objective function and the law of motion that governs the environment. The entire structural econometrics enterprise (demand estimation, dynamic discrete choice) is devoted to recovering these primitives from data, and the optimal policy is a byproduct that falls out once the primitives have been identified. The control tradition inverts this emphasis. Engineers typically take the objective and the dynamics as given (the cost function is specified, the plant physics are known or measurable) and focus on whether the optimal policy can be computed, approximated, and deployed under real-time and robustness constraints. The entire controls enterprise (PID, LQR, MPC, RL) is about computing and implementing the policy under different assumptions about what the agent knows and what it can compute.

The two cultures maintain different relationships with data. Econometricians work primarily with observational data, where endogeneity is the central obstacle; agents sort, markets clear, and unobservables correlate with the variables of interest. Identification, the question of whether the data can distinguish the true model from observationally equivalent alternatives, is the defining challenge, and it disciplines every modeling choice (functional forms, equilibrium definitions, instrumental variables, regression discontinuities, natural experiments). RL researchers have traditionally enjoyed what might be called *simulator omnipotence*. They own the data-generating process, can inject arbitrary variation through exploration policies, and can generate millions of on-policy rollouts at negligible marginal cost. Their binding constraint is computational (can the algorithm converge before the compute budget runs out?) rather than statistical (is the estimator consistent given the endogeneity structure of the data?). This asymmetry shapes everything downstream, from what counts as a valid result to what the word "model" means. To an econometrician, a model is a set of falsifiable restrictions on the joint distribution of observables and primitives. To an RL researcher, a model is a simulator you can call.

The two fields also share a vocabulary ("agent," "learning," "model," "policy") whose meanings diverge in ways that create persistent confusion. The subsections below provide a systematic translation.

1.1.1 Core Distinctions

In RL, *prediction* refers to estimating $V^\pi(s)$ or $Q^\pi(s,a)$ for a fixed policy π (policy evaluation), not forecasting observable variables. *Control* refers to finding the policy π^* that maximizes expected discounted return (policy optimization), not the inclusion of regressors. Because prediction concerns evaluating a specific policy, a closely related question is whether the data was generated by the policy being studied. *On-policy* methods evaluate and improve the same policy that generates the data. *Off-policy* methods learn about a target policy π from data generated by a different behavioral policy μ. This distinction is central to causal inference, where off-policy evaluation is precisely counterfactual policy evaluation.[2]

[2] "Counterfactual" here is used in the interventional sense, asking "what would happen if we deployed policy π instead of μ?" This is distinct from the structural counterfactual in Oberst and Sontag (2019), which conditions on the specific realized trajectory and asks what would have happened to *this* individual under a different action,

Online RL learns while interacting with the environment, collecting new data as a consequence of its own actions. *Offline* RL (also called batch RL) learns exclusively from a fixed dataset of previously collected transitions, with no ability to gather additional samples. The offline setting is closer to standard empirical work, where the dataset is given and the analyst cannot run new experiments; the online setting corresponds more closely to adaptive experimental design or sequential decision problems. Note that "online" in RL carries no timing constraint; it means only that the agent generates fresh experience. *Real-time* RL, by contrast, imposes hard deadlines on the perception-action loop, as in robotics or mechatronics. Every real-time RL system is online, but most online RL (games, recommender systems) are not real-time.

In RL, the word *model* refers strictly to the environment's dynamics, the transition kernel $P(s'|s,a)$ and reward function $r(s,a)$. A *model-based* RL algorithm explicitly constructs or is given a mathematical representation of P and r, then computes a policy by planning through that representation (for example, using a simulator or the known rules of a game, as in AlphaZero). A *model-free* algorithm computes the value function or policy directly from experienced transitions without ever building an explicit representation of the transition probabilities. "Model-free" need not mean the algorithm lacks access to any model of the environment. A model-free algorithm could interact with a simulator that internally implements a complete computerized model of the environment; the distinction is that the algorithm never extracts or plans through the transition probabilities, treating the simulator as a black box that merely returns sample transitions. However, it is possible that a "model-free" algorithm could also be implemented "in-field" where there is genuinely no access to a "model" of the environment but only direct access to the environment itself (for reasons discussed later, this is rarely done).

Neither "model-free" nor "model-based" maps onto the reduced-form versus structural distinction in econometrics. Both labels refer to whether the algorithm uses an explicit representation of P and r, not to whether the analyst makes structural assumptions about preferences or equilibrium. A "model-based" RL algorithm needs only a representation of P and r, regardless of whether those objects arise from structural assumptions about human behavior. Conversely, "model-free" does not mean "assumption-free"; both variants operate within an MDP, which itself imposes the Markov assumption on the state. In the inference tradition, "model" refers to a set of agents, preferences, exclusion restrictions, and equilibrium concepts. It is therefore possible to specify a rich structural model but solve for its equilibrium using a model-free RL algorithm as a computational tool, as in Section 4.

Every RL system passes through two phases. In the *training phase*, the agent interacts with an environment (simulated or real) and updates its parameters. In the *execution phase* (also called the deployment phase), the policy is frozen and used to make decisions without further updates. This distinction is critical for interpreting "online." Online training in RL almost always takes place inside a simulator (and not in the "real world"). AlphaGo Zero trained online through millions of self-play games; when it faced Lee Sedol, its weights were frozen and it was purely executing its trained policy.[3]

The word *inference* is overloaded. In machine learning, "inference" refers to executing a frozen model, a forward pass producing outputs from inputs. In statistics, "inference" means statistical inference, the construction of standard errors, confidence intervals, and hypothesis tests. This chapter uses "inference" exclusively in the statistical sense and "execution" or "deployment" when referring to applying a trained model (whether in a simulator or in the field). Established terms such as "Bayesian inference" and "variational inference," which refer to inference about parameters or distributions, are used where appropriate. Most RL convergence results and sample complexity guarantees refer to the training phase; interpreting them as claims about deployed performance requires additional argument. Table 1 summarizes these

requiring a fully specified structural causal model rather than just the observational distribution under μ.

[3] "In-field" is not standard RL vocabulary. I use it here to distinguish live-market online learning, where exploration has real financial cost, from the far more common case of online learning inside a simulator.

distinctions.

Table 1: The reinforcement learning lifecycle grid. Most RL research operates in the top-left cell. Readers from the inference tradition typically picture the bottom-left when they hear "online."

	Training (parameters updated)	Execution (parameters frozen)
Simulator	· AlphaGo Zero self-play · RL solver in structural estimation	· Policy benchmarks on synthetic environments
Historical data	· RLHF reward model from preference logs	· Off-policy evaluation of a target policy
Live market ("in-field")	· Adaptive pricing experiments	· AlphaGo vs. Lee Sedol

A typical applied RL pipeline moves through the grid sequentially, from pre-training on historical logs (middle-left) to refinement in a simulator (top-left) to deployment with frozen weights (bottom-right).

1.1.2 Overlapping Terminology

In the inference tradition, an *agent* is always a human decision-maker (a consumer, worker, or firm) or a social planner whose choices are the object of study. In RL, "the agent" is the learning algorithm itself, or more precisely an algorithm deployed on behalf of a human decision-maker, and the human, if present at all, is part of the environment providing reward signals.

In RL the *environment* is a formal object encompassing everything outside the agent (the DGP, other agents, market clearing conditions), whereas in the inference tradition "environment" refers more loosely to market structure or institutional rules.[4]. RL speaks of *rewards* where the other tradition speaks of *utility*. The mapping is not exact: a reward $r(s, a)$ is a known, externally specified function that the algorithm maximizes, whereas utility $u(x, d)$ in the inference tradition is a primitive of preferences that the analyst must recover or estimate from observed choices.

In RL, the return $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$ is the discounted sum of future rewards from time t onward, the random variable whose expectation defines the value function. The RL usage is closer to what is called the "present discounted value" of a stream of payoffs. A single complete sequence of interactions from an initial state to termination, $(s_0, a_0, r_1, s_1, \ldots, s_T)$, is called an *episode* (synonymously, *trajectory* or *rollout*).[5] Where a statistician speaks of the *outcome*, meaning the dependent variable Y in a regression, RL has no single analog: the reward r_t is the per-period outcome, the return G_t is the cumulative outcome, and the value function $V^\pi(s)$ is the expected cumulative outcome conditional on state.

Learning has several distinct meanings. In decision theory (Bayesian learning, adaptive expectations), learning refers to agents forming and refining beliefs about unknown parameters of their environment. In supervised machine learning, learning means statistical estimation, fitting the weights of a parameterized model to minimize a loss function over data. In reinforcement learning, "learning" is primarily computation. When an RL agent "learns" a Q-function, it is executing a recursive stochastic approximation algorithm to find the fixed point of the Bellman operator. The algorithm is said to "learn" because it improves its policy iteratively through

[4]A source of confusion is *generative model*. In machine learning broadly, this means a model of the joint distribution $P(X, Y)$ or a synthetic data generator (GANs, diffusion models). In RL theory, a generative model is a simulator oracle that, given any (s, a), returns a sample $s' \sim P(\cdot|s, a)$ and reward $r(s, a)$; the usage implies random-access simulation, stronger than sequential online interaction but weaker than knowing P analytically.

[5]The closest statistical analogs are a single panel unit's time series, a realization of a stochastic process, or one "history" in a dynamic model.

simulated or real experience, but mathematically it is solving a fixed-point problem. In many applications throughout this chapter, particularly Section 4, the RL algorithm is simply a numerical method for solving the Bellman equation; no actual human-like learning from experience is taking place.

Adaptive learning in macroeconomics has used ideas formally analogous to reinforcement learning for decades, though under different names and with different motivations. In the adaptive learning literature initiated by Marcet and Sargent (1989), boundedly rational agents update their beliefs about equilibrium parameters using recursive least-squares and other Robbins-Monro-type stochastic approximation algorithms. The convergence criterion in that literature, E-stability, evaluates whether the ordinary differential equation (ODE) associated with the mapping from the perceived to the actual law of motion is locally asymptotically stable at the rational expectations equilibrium. This fixed-point stability requirement is analogous to the contraction conditions governing the convergence of temporal-difference and Q-learning algorithms in RL. Borkar and Meyn (2000) make this mathematical connection explicit: they prove the convergence of Q-learning and actor-critic methods using the ODE method, explicitly citing Sargent (1993) as a parallel application of the exact same framework to boundedly rational agents.

Exploration is ubiquitous in RL but rarely named as such in the inference tradition. In RL, exploration is a procedure for choosing informative actions that may be crude (ε-greedy) or principled (UCB, Thompson sampling). The closest analog is *optimal experimentation* in the Bayesian tradition (Rothschild 1974), where the value of information is derived endogenously from the agent's dynamic program rather than bolted on as a separate heuristic.

Bootstrapping in statistics refers to Efron's resampling method (Efron, 1979); in RL, it means updating a value estimate using another value estimate rather than a complete realized return, as when a TD algorithm uses the target $r_{t+1} + \gamma V(s_{t+1})$ that depends on the current, uncertain estimate $V(s_{t+1})$ (Sutton, 1988). *Function approximation* in RL refers to representing value functions or policies using parameterized function classes (linear combinations of basis functions, kernel methods, or neural networks), which statisticians will recognize as sieve estimation or nonparametric series estimation, the approximation of an unknown function by projection onto a finite-dimensional basis. *Calibration* carries unrelated meanings. In the inference tradition, calibration means choosing model parameters to match a set of empirical moments or stylized facts (Kydland and Prescott 1982). In machine learning, calibration refers to probability calibration, ensuring that predicted probabilities match observed frequencies, a statistical property of a classifier's outputs.

The word *policy* is overloaded. In the inference tradition, a "policy" is a rule set by a government, central bank, or regulator, a tax schedule, a subsidy, an interest rate rule, a licensing requirement. These rules are part of the *environment* in which private agents (consumers, firms) optimize. "Policy evaluation" in this tradition means changing some aspect of the environment and asking how agents would respond: if the earned income tax credit were expanded, how would labor supply shift? The standard workflow proceeds in two steps: first estimate or calibrate the structural model (preferences, technology, market interactions), then simulate counterfactuals under the alternative rule. In RL, "policy" means the agent's own decision rule $\pi(a|s)$, and "policy evaluation" means computing $V^{\pi}(s)$, the expected return from following that rule in a fixed environment. RL typically assumes access to a high-fidelity simulator, a physics engine, a game, a digital twin, whose rules do not change; the interesting question is how the agent should behave within those rules, not what happens if the rules change. When the environment does shift, RL treats it as a nuisance (sim-to-real transfer, domain adaptation) rather than the object of study; in offline RL (Section 6), distributional shift between the training data and the deployed environment becomes a central concern, bringing RL closer to standard counterfactual reasoning.[6]

[6]A notable exception is Tomasev et al. (2020), where AlphaZero was used to evaluate alternative chess rule

Identification means different things in the two fields. In statistics, a parameter θ is identified if no two distinct parameter values can generate the same distribution of observable data (Lewbel, 2019). Identification is a logical property of the model, not a statistical one; if it fails, no amount of additional data will recover the parameter. In RL, there is little general identification discourse. The closest analog is reward identifiability in inverse reinforcement learning, treated in Rust and Rawat (2026). In model-based RL, system identification refers to learning the transition dynamics well enough for the resulting policy to perform well, a usage inherited from control theory (Ross and Bagnell, 2012).

Regret diverges across fields. In decision theory, regret is either an emotion that modifies preferences under uncertainty (Loomes and Sugden 1982) or a static minimax decision criterion for choosing among actions when probabilities are unknown (Savage 1951, Manski 2004). In RL and computer science, regret is a dynamic performance metric, the cumulative gap between the agent's returns and those of the best fixed policy in hindsight, and sublinear growth of this quantity is the standard benchmark for online learning algorithms. In the other tradition, *efficiency* concerns welfare: Pareto efficiency asks whether any reallocation could improve one agent's utility without harming another, and allocative efficiency asks whether goods flow to their highest-valued uses. In RL, efficiency concerns resources: sample efficiency measures how many environment interactions an algorithm needs to find a good policy, and computational efficiency measures the operations required per timestep.

In the inference tradition, the *discount factor* is a structural parameter encoding time preference, an agent's intrinsic willingness to trade present for future consumption. Koopmans (1960) derived it axiomatically from preference postulates, principally stationarity and impatience, showing that these behavioral axioms uniquely imply an exponential discounted utility representation with a single discount factor strictly between zero and one (Bleichrodt et al., 2008). This parameter is treated as a deep primitive of preferences, to be estimated from data on intertemporal choices, and deviations from constant discounting (such as the quasi-hyperbolic present bias of Laibson 1997) are studied as substantive behavioral phenomena. In RL, the discount factor has historically served a more pragmatic role, ensuring that infinite-horizon returns remain finite and that the Bellman operator is a contraction, thereby guaranteeing convergence of dynamic programming algorithms. It is typically treated as a hyperparameter to be tuned for computational performance rather than a structural claim about the agent's preferences. Pitis (2019) bridges this gap, axiomatically deriving discounting in RL from rationality postulates and showing that the fixed discount factor is best understood as an optimizing representation rather than a literal description of time preference.

1.1.3 Structural Equivalences

Beyond terminological differences, several formal objects in RL and the inference tradition are mathematically identical. The softmax (or Boltzmann) policy used throughout RL is the multinomial logit model of McFadden (1974). The RL softmax policy selects actions according to

$$\pi(a \mid s) = \frac{\exp(Q(s,a)/\tau)}{\sum_{a' \in \mathcal{A}} \exp(Q(s,a')/\tau)}, \tag{1}$$

where $\tau > 0$ is a temperature parameter. In the discrete choice framework, $Q(s,a)$ plays the role of the deterministic component of utility $v(a \mid x)$, and τ is the scale parameter of the Type I extreme value (Gumbel) taste shocks ε_a. As $\tau \to 0$, the policy converges to the greedy (deterministic) policy, just as the logit choice probability concentrates on the utility-maximizing alternative as the variance of taste shocks vanishes.

The entropy regularization commonly added to RL objectives is the inclusive value (or log-

sets proposed by Kramnik, asking how optimal play changes under modified game rules. This is precisely an environment counterfactual.

sum-exp) from the discrete choice literature. The soft value function

$$V^{\text{soft}}(s) = \tau \log \sum_{a \in \mathcal{A}} \exp(Q(s,a)/\tau) \tag{2}$$

is identical to the McFadden surplus function $W(x) = \tau \log \sum_a \exp(v(a|x)/\tau) + C$, where C is Euler's constant. In the structural estimation literature, this object appears as the Emax function in dynamic discrete choice models following Rust (1987).

The action-value function $Q^\pi(s,a)$ is the choice-specific value function $v_\theta(x,d)$ of the DDC literature (Table 2). The advantage function $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ is therefore the choice-specific value net of the ex-ante value, a quantity that appears in the Hotz and Miller (1993) CCP estimator for dynamic discrete choice models.[7]

The two fields arrived at this shared mathematics from opposite directions. In RL, entropy regularization began as a pragmatic trick to prevent premature convergence and encourage exploration (Williams and Peng, 1991), and only decades later did Ziebart (2010) and Levine (2018) provide decision-theoretic foundations showing that the resulting policies are robust to model misspecification. In the inference tradition, the softmax emerged not from any concern about exploration but from the random utility framework of McFadden (1974), where agents are fully informed and choose deterministically; the apparent randomness arises entirely from the econometrician's inability to observe all relevant taste variation. The RL agent randomizes because it is ignorant of the environment; the economic agent appears to randomize because the observer is ignorant of the agent's preferences.

1.1.4 Notation

Table 2 maps the notation used throughout this chapter to the most common econometric equivalents.

Table 2: Notation mapping between reinforcement learning and the inference tradition.

RL Term	Symbol	Inference Tradition	Symbol
State	$s \in \mathcal{S}$	State variable, covariate	x_t, Ω_t
Action	$a \in \mathcal{A}$	Choice, control variable	d_t, u_t[8]
Reward	$r(s,a)$	Per-period utility, payoff	$u(x,d)$
Discount factor	γ[9]	Discount factor	β
Policy	$\pi(a\|s)$	Decision rule, CCP	$P(d\|x)$
Value function	$V^\pi(s)$	Ex-ante value function	$\bar{V}_\theta(x)$
Q-function	$Q^\pi(s,a)$	Choice-specific value function	$v_\theta(x,d)$
Return	G_t	Present discounted value	$\sum_{k=0}^\infty \beta^k u_{t+k}$
Transition	$P(s'\|s,a)$	State transition law	$f(x_{t+1}\|x_t,d_t)$
Learning rate	α	Step size	α_n
TD error	δ_t[10]	Bellman residual at sample	–

[7] These equivalences reflect a common convex-analytic structure. The log-sum-exp value function and the negative Shannon entropy are Fenchel conjugates of each other, so maximizing expected payoffs with an entropy bonus and recovering utilities from observed choice probabilities are two views of the same optimization. Chiong et al. (2016) build on this conjugate duality to develop identification and estimation methods for dynamic discrete choice models, and Fosgerau and Sørensen (2022) show that the underlying structure extends well beyond the logit case.

2 Reinforcement Learning Algorithms

2.1 The Classical Synthesis

2.1.1 Monte Carlo Estimation

When $P(s'|s,a)$ and $r(s,a)$ are unknown, the obvious approach is to estimate value functions directly from sampled *episodes* $(s_0, a_0, r_1, s_1, a_1, r_2, \ldots, s_T)$. The realized *return* from state s_t is

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1} \tag{3}$$

First-visit MC prediction averages G_t over episodes for each state s, counting only its first occurrence per episode. Each first-visit return is an independent draw from the return distribution, so the sample mean converges almost surely to $V^\pi(s)$ by the strong law of large numbers (Sutton and Barto, 2018). An incremental update is,

$$V(s) \leftarrow V(s) + \alpha[G_t - V(s)]$$

For MC control, one estimates action-values $Q(s,a)$ by averaging first-visit returns from each state-action pair, then improve the policy greedily: $\pi(s) = argmax_a\, Q(s,a)$. Under exploring starts (every (s,a) pair begins an episode infinitely often), this alternation converges to Q^*. [11]

2.1.2 Sutton (1988)

Monte Carlo has two limitations. The agent must wait for episode termination to compute G_t, ruling out continuing tasks. And G_t is unbiased ($\mathbb{E}[G_t \mid S_t = s] = V^\pi(s)$) but high-variance, because it sums random rewards over the entire *trajectory*.

Sutton (1988) proposed temporal difference (TD) learning to fix both problems. TD(0) replaces the full return G_t with a one-step target:

$$V(s_t) \leftarrow V(s_t) + \alpha\left[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)\right] \tag{4}$$

The target $r_{t+1} + \gamma V(s_{t+1})$ depends on one random reward and one random transition, so its variance is low. The cost is bias. The bootstrap target $V(s_{t+1})$ is the agent's current estimate, not the true value. This is "*bootstrapping*".[12] As V improves, the bias shrinks; the low variance persists regardless. Sutton demonstrated this tradeoff on a five-state random walk where TD(0) converged faster than Monte Carlo with less data.

The general TD(λ) update interpolates between these extremes through an eligibility trace.[13]

$$V(s_t) \leftarrow V(s_t) + \alpha\, \delta_t\, e_t(s) \tag{5}$$

where $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$ is the TD error and $e_t(s) = \gamma\lambda\, e_{t-1}(s) + \mathbb{1}\{s = s_t\}$ is the eligibility trace for state s. Setting $\lambda = 0$ yields TD(0); setting $\lambda = 1$ recovers Monte Carlo returns. Intermediate λ trades off variance against bias. [14]

[11] Tsitsiklis (2002) proved convergence even when the policy improves after every episode rather than waiting for complete evaluation. In practice, exploring starts is infeasible, so on-policy variants use ε-greedy exploration instead. These converge to Q^* provided the exploration schedule satisfies the greedy-in-the-limit with infinite exploration (GLIE) condition: every state-action pair is visited infinitely often, and $\varepsilon_t \to 0$ so the policy converges to greedy in the limit.

[12] See Section 1.1 for the distinction between RL bootstrapping and Efron's resampling procedure.

[13] The eligibility trace records which states were recently visited, allowing credit assignment to propagate backward in time. States visited more recently receive stronger updates when the TD error is observed.

[14] Dayan (1992) proved convergence of TD(λ) for general λ in the tabular case; Jaakkola et al. (1994) gave a unified stochastic approximation proof covering both TD and Q-learning; Tsitsiklis and Van Roy (1997) extended

2.1.3 Watkins (1989)

TD(0) learns value functions $V(s)$, but converting these to actions (the control problem) still requires knowing transition probabilities. Given $V(s')$ for all successor states, the agent needs to know which action leads to which successor.

Watkins and Dayan (1992), formalizing Watkins's 1989 PhD thesis, provided the solution. Instead of learning $V(s')$ learn $Q(s,a)$ (the action value function or the "quality" of actions function) directly, the expected return from taking action a in state s and then behaving optimally. The optimal policy is then $\pi^*(s) = \mathit{argmax}_a\, Q^*(s,a)$, requiring no model to act. The Bellman optimality equation provides the fixed-point condition:

$$Q^*(s,a) = \mathbb{E}\left[r + \gamma \max_{a'} Q^*(s',a') \mid s, a\right] \tag{6}$$

Q-learning achieves this via the update

$$Q(s_t,a_t) \leftarrow Q(s_t,a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1},a') - Q(s_t,a_t)\right] \tag{7}$$

Q-learning converges to Q^* under standard regularity conditions (Section 3.2). The maximization over a' makes Q-learning *off-policy*:[15] the update target uses the greedy action at the next state regardless of the action actually taken. Therefore the agent can follow an ε-greedy exploration strategy or even a fully random policy, while learning about the optimal policy directly.

2.1.4 Williams (1992)

Value-based methods learn an action-value function and derive a policy from it. Williams (1992) derived an alternative, policy gradients, that optimize the policy directly by gradient ascent on expected return

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t)\, G_t\right] \tag{8}$$

where $G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k+1}$ is the discounted return from time t. The log-derivative trick allows the gradient to be estimated from sampled trajectories without differentiating through the environment dynamics.

When the action space is continuous, Q-learning requires computing $\max_a Q(s,a)$ at every update, a nested optimization problem.[16] Policy gradients sidestep the issue. Parameterize the policy as a Gaussian $\pi_\theta(a|s) = \mathcal{N}(\mu_\theta(s), \sigma^2_\theta(s))$, sample an action, observe the return, and update θ by REINFORCE. Continuous-control results in reinforcement learning descend from this framework.

2.1.5 Tesauro (1994)

Tesauro (1994)'s TD-Gammon demonstrated that temporal difference learning with neural network function approximation could achieve expert-level play in backgammon, a domain with approximately 10^{20} legal positions. A feedforward neural network estimated the probability of winning from any board position, trained by self-play using TD(λ) with $\lambda = 0.7$. At each turn,

the analysis to linear function approximation.

[15] See Section 1.1 for the on-policy/off-policy distinction. Q-learning learns about the greedy policy $\pi^*(s) = \mathit{argmax}_a\, Q(s,a)$ while collecting data with an exploratory policy. This exploratory policy needs only to be sufficiently "exploratory" and admits a wide range of policies; including a fully random policy.

[16] In continuous action spaces, $\max_a Q(s,a)$ has no closed-form solution in general and must be solved numerically at every Bellman update. Discretizing a d-dimensional action space on a grid of m points per dimension costs $O(m^d)$ evaluations per update step.

the current player selected the legal move maximizing the network's value estimate. As the network improved, it generated stronger opponents for itself, producing harder training games that drove further improvement. Backgammon's dice rolls provided natural exploration without requiring an explicit mechanism.[17]

2.1.6 SARSA (1994)

Q-learning learns the optimal action-value function regardless of the policy generating experience. This off-policy property is useful but introduces complications when combined with function approximation. Rummery and Niranjan (1994) introduced SARSA as an *on-policy*[18] alternative that learns the value of the policy actually being followed. The name derives from the quintuple $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$ used in each update:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)\right] \tag{9}$$

The key difference from Q-learning is that SARSA bootstraps from the action a_{t+1} actually taken at the next state, rather than the greedy action $argmax_{a'} Q(s_{t+1}, a')$. This makes the algorithm on-policy, since the target depends on the behavior policy generating the data.

If the agent follows an ε-greedy policy, SARSA converges to $Q^{\varepsilon\text{-greedy}}$, not Q^*[19] policies and standard step-size conditions (Section 3.2). This distinction matters when exploration is costly. Consider the cliff-walking problem, where an agent must traverse a gridworld with a cliff along one edge. The optimal path runs along the cliff edge (shortest route), but the ε-greedy policy occasionally falls off. Q-learning learns the optimal path because it evaluates the greedy policy; the agent falls off during learning but the Q-values reflect the optimal route. SARSA learns a safer path further from the cliff because it evaluates the actual exploratory policy; it accounts for the fact that exploration sometimes leads to catastrophic states.

2.1.7 Baird (1995)

Baird (1995) constructed a six-state star MDP demonstrating divergence of Q-learning with linear function approximation. The MDP has five outer states that all transition to a single inner state under the target policy. The off-policy behavior samples states uniformly. With linear function approximation, the weights grow without bound under repeated Q-learning updates. The source of instability is the interaction of three components, namely bootstrapping (updating from estimated values rather than observed returns), off-policy learning (training on data from a different policy than the target), and function approximation (representing the value function with a parameterized model). Sutton and Barto (2018) later named this the deadly triad. Any two components can be combined safely; all three together permit divergence. The mechanism underlying this instability is analyzed in Section 3.3.

This result explains the asymmetry between Tesauro's success and Baird's failure. TD-Gammon used bootstrapping and function approximation but was on-policy, with training data coming from the same self-play policy whose value was being estimated. Baird's counterexample used all three components and diverged. Baird also proposed a constructive solution, namely residual gradient algorithms that perform gradient descent on the mean-squared Bellman residual, guaranteeing convergence at the cost of a different fixed point.

[17] Version 2.1, trained on 1,500,000 self-play games, achieved near-parity with former world champion Bill Robertie. The logistic sigmoid activation $\sigma(x) = 1/(1 + e^{-x})$ used in the network is identical to the binary logit link function in discrete choice.

[18] See Section 1.1 for the on-policy/off-policy distinction.

[19] SARSA converges to Q^* under GLIE (greedy in the limit with infinite exploration). A GLIE schedule explores all state-action pairs infinitely often but converges to the greedy policy asymptotically. The standard ε-greedy policy with $\varepsilon_t \to 0$ is one such schedule.

2.1.8 Actor-Critic Methods (2000)

The idea of maintaining both a policy (*actor*) and a value function (*critic*) dates to Barto et al. (1983), who used a two-component system to solve the pole-balancing task. Actor-critic methods address the high variance of REINFORCE by replacing Monte Carlo returns with bootstrapped TD targets as the learning signal. The critic learns a value function $V(s)$ by TD updates.

$$V(s_t) \leftarrow V(s_t) + \alpha_c\, \delta_t, \quad \delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t) \tag{10}$$

The actor updates the policy using the TD error as a sample of the advantage.

$$\theta \leftarrow \theta + \alpha_a \nabla_\theta \log \pi_\theta(a_t|s_t)\, \delta_t \tag{11}$$

The TD error δ_t estimates $A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t)$, the advantage of action a_t over the average action.[20] Positive advantages indicate the action was better than expected; negative advantages indicate it was worse.

Konda and Tsitsiklis (2000) provided the first convergence proof for actor-critic algorithms with function approximation, showing convergence to a stationary point under two-timescale learning rates ($\alpha_c \gg \alpha_a$) and a compatibility condition on the critic architecture (Section 3.5).

2.1.9 Natural Policy Gradient (2001)

Standard gradient descent treats all parameter directions equally, but policy parameters define probability distributions whose natural geometry is not Euclidean. Kakade (2001) introduced the natural policy gradient, which measures progress in distribution space rather than parameter space. The update uses the Fisher information matrix $F(\theta)$:

$$F(\theta) = \mathbb{E}_{s\sim d^\pi, a\sim\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a|s) \nabla_\theta \log \pi_\theta(a|s)^\top\right] \tag{12}$$

The natural gradient is

$$\tilde{\nabla}_\theta J(\theta) = F(\theta)^{-1} \nabla_\theta J(\theta) \tag{13}$$

This direction is invariant to reparameterization of the policy. In the tabular softmax[21] case, a single natural gradient step with unit step size recovers one step of exact policy iteration (Section 3.4).

The computational bottleneck is inverting $F(\theta) \in \mathbb{R}^{d\times d}$. Practical implementations use conjugate gradient methods to solve $F(\theta)x = \nabla_\theta J(\theta)$ without forming F explicitly. This approach was later scaled to deep neural networks by TRPO and PPO.

2.1.10 Fitted Value Iteration and Fitted Q-Iteration (2005)

Tabular Q-learning maintains a separate entry for every state-action pair. When $|\mathcal{S}|$ is large or the state space is continuous, as in most applied problems, this is infeasible. *Fitted Q-Iteration* (FQI) (Ernst et al., 2005) replaces the tabular update with a supervised regression step: given a batch of transitions, fit a function approximator to the Bellman targets.

Let $\mathcal{F}$ be a function class mapping $\mathcal{S} \times \mathcal{A} \to \mathbb{R}$. Initialize $Q_0 \equiv 0$. At each iteration $k = 0, 1, \ldots, K-1$: (i) draw N transitions (s_i, a_i, r_i, s'_i) from a generative model; (ii) construct regression targets $y_i^{(k)} = r_i + \gamma \max_{a'} Q_k(s'_i, a')$; (iii) set $Q_{k+1} \leftarrow \arg\min_{f\in\mathcal{F}} \frac{1}{N}\sum_{i=1}^N (f(s_i, a_i) - y_i^{(k)})^2$. The output is the greedy policy $\pi_K(s) = \arg\max_a Q_K(s, a)$.

[20] That δ_t is an unbiased estimate of the advantage follows from the policy gradient theorem (Sutton et al., 2000).

[21] The softmax function maps a vector $\mathbf{z}$ to a probability distribution: $\mathrm{softmax}(z_i) = \exp(z_i)/\sum_j \exp(z_j)$. A softmax policy parameterizes action probabilities as $\pi_\theta(a|s) = \mathrm{softmax}(\theta_{s,a})$.

The regression step replaces the exact Bellman application with a projection onto $\mathcal{F}$: $Q_{k+1} = \Pi_{\mathcal{F}}\mathcal{T}Q_k$, where $\mathcal{T}$ is the Bellman optimality operator and $\Pi_{\mathcal{F}}$ is the L^2-projection under the sample distribution. Fitted Value Iteration (FVI) (Munos and Szepesvári, 2008) applies the same idea to the value function directly: $V_{k+1} = \Pi_{\mathcal{F}}\mathcal{T}^*V_k$, where $(\mathcal{T}^*V)(s) = \max_a\{r(s,a) + \gamma\sum_{s'} P(s'|s,a)V(s')\}$.

With feature matrix $\Phi \in \mathbb{R}^{|\mathcal{S}|\times d}$ (rows $\phi(s)^\top$) and per-action weight vectors $\theta_a \in \mathbb{R}^d$, each FQI regression step for action a reduces to the normal equations:

$$\theta_a^{(k+1)} = \left(\Phi^\top\Phi\right)^{-1}\Phi^\top y_a^{(k)}, \tag{14}$$

where $y_a^{(k)}(s) = r(s,a) + \gamma\sum_{s'} P(s'|s,a)\max_{a'}\phi(s')^\top\theta_{a'}^{(k)}$. Computation is $O(d^2|\mathcal{S}| + d^3)$ per action per iteration. The FVI update takes the same form with a single weight vector θ_V:

$$\theta_V^{(k+1)} = \left(\Phi^\top\Phi\right)^{-1}\Phi^\top V_{\text{target}}^{(k)}, \tag{15}$$

where $V_{\text{target}}^{(k)}(s) = \max_a\{r(s,a) + \gamma\sum_{s'} P(s'|s,a)\phi(s')^\top\theta_V^{(k)}\}$. The finite-sample error theory for these methods is developed in Section 3.2.4.

2.2 The Deep Learning Era

2.2.1 Deep Q-Networks (2015)

Mnih et al. (2015) trained a single convolutional neural network[22] to play 49 Atari 2600 games directly from pixel inputs ($210 \times 160 \times 3$) and a scalar score, using no game-specific features.

The architecture processed four consecutive frames through three convolutional layers and a fully connected layer.[23] The network $Q(s,a;\theta)$ was trained to minimize the squared temporal difference error

$$L(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(r + \gamma\max_{a'} Q(s',a';\theta^-) - Q(s,a;\theta)\right)^2\right] \tag{16}$$

Two innovations stabilized learning. *Experience replay* (Lin, 1992) stored transitions (s,a,r,s') in a buffer $\mathcal{D}$ and sampled uniformly for training, breaking the temporal correlation between consecutive updates. A *target network* used a frozen copy of parameters θ^-, updated periodically,[24] so the regression target does not shift with each gradient step.

DQN exceeded human-level performance on 29 of 49 games using a single architecture and hyperparameters.[25] Games requiring long-horizon planning or sparse rewards, such as Montezuma's Revenge, remained difficult.

2.2.2 TRPO and PPO (2015, 2017)

Policy gradient methods suffer from a practical instability: a single large gradient step can move the policy into a region where performance collapses and recovery is slow.

[22] A convolutional neural network applies learned spatial filters to detect local patterns in grid-structured data, commonly used for image inputs.

[23] A fully connected layer computes $y = Wx + b$ where every input unit is connected to every output unit with learned weights W and biases b; it is the affine transformation familiar from linear regression, followed by a nonlinear activation.

[24] The buffer held 10^6 transitions; the target network θ^- was synchronized to θ every $C = 10{,}000$ steps.

[25] *Hyperparameters* are design choices fixed before training begins, such as network depth, learning rate, and replay buffer size; they are not estimated by gradient descent.

Schulman et al. (2015) addressed this with Trust Region Policy Optimization (TRPO). TRPO solves the constrained optimization problem

$$\max_{\theta} \; L_{\theta_{\text{old}}}(\theta) \quad \text{subject to} \quad \bar{D}_{\text{KL}}(\theta_{\text{old}}, \theta) \leq \delta \tag{17}$$

where L is a surrogate objective based on the advantage function $A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s)$.[26]

Schulman et al. (2017) proposed Proximal Policy Optimization (PPO) as a simpler alternative. PPO replaces the KL constraint with a clipped surrogate objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\left[\min\Big(r_t(\theta)\hat{A}_t,\; \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t\Big)\right] \tag{18}$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ is the probability ratio. The clipping removes the incentive for $r_t(\theta)$ to move outside the interval $[1-\varepsilon, 1+\varepsilon]$, penalizing large policy updates without explicitly computing a divergence measure.

PPO outperformed A2C, TRPO, and the cross-entropy method on continuous control benchmarks and achieved the highest average reward on 30 of 49 Atari games among the methods tested. PPO is widely used in large-scale RL applications; constraining the magnitude of policy updates is essential for stable optimization.

2.2.3 Soft Actor-Critic (2018)

Haarnoja et al. (2018) introduced Soft Actor-Critic (SAC), which adds entropy regularization to the actor-critic framework. The agent maximizes expected return plus an entropy bonus:

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty} \gamma^t \left(r_t + \tau \mathcal{H}(\pi_\theta(\cdot|s_t))\right)\right] \tag{19}$$

where $\mathcal{H}(\pi) = -\sum_a \pi(a)\log\pi(a)$ is the entropy and $\tau > 0$ is a temperature parameter. The entropy bonus encourages exploration by penalizing deterministic policies. The optimal policy under this objective is softmax in the Q-values: $\pi^*(a|s) \propto \exp(Q^*(s,a)/\tau)$, connecting to discrete choice models in econometrics.

SAC maintains two Q-networks (to reduce overestimation bias) and a policy network. The soft Bellman operator for the critic is:

$$(T^\pi Q)(s,a) = r(s,a) + \gamma \mathbb{E}_{s'}\left[V(s')\right], \quad V(s) = \mathbb{E}_{a\sim\pi}[Q(s,a) - \tau\log\pi(a|s)] \tag{20}$$

SAC is off-policy (using experience replay), handles continuous actions naturally, and achieves state-of-the-art sample efficiency on continuous control benchmarks. The entropy regularization provides automatic exploration without ε-greedy schedules.

2.2.4 Control as Probabilistic Inference

The *control-as-inference* framework (Todorov, 2006; Levine, 2018) recasts RL as probabilistic inference in a graphical model. Appending a binary optimality variable $\mathcal{O}_t$ to the standard MDP, with $P(\mathcal{O}_t = 1 \mid s_t, a_t) = \exp(r(s_t,a_t)/\tau)$, turns policy optimization into posterior inference over actions conditional on future optimality. Structured variational inference (restricting the variational family to the true dynamics) yields the soft Bellman equations already given in the SAC subsection, with the soft value function $V(s) = \tau\log\sum_a \exp(Q(s,a)/\tau)$ identical to McFadden's social surplus function from discrete choice theory (McFadden, 1978). Different approximation strategies recover SAC, soft Q-learning, and max-ent policy gradients as special

[26] The Kullback-Leibler divergence $D_{\text{KL}}(p\|q) = \sum_x p(x)\log(p(x)/q(x))$ measures statistical distance between distributions. The bar denotes expectation over states: $\bar{D}_{\text{KL}} = \mathbb{E}_s[D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot|s)\|\pi_\theta(\cdot|s))]$.

cases.[27] Reading the model in reverse, treating the reward as unknown and observed behavior as evidence of optimality, yields maximum entropy inverse reinforcement learning (Ziebart et al., 2008), connecting to the companion survey (Rust and Rawat, 2026).

2.2.5 AlphaGo Zero (2017)

The game of Go has approximately 10^{170} legal positions and a branching factor of roughly 250, far beyond brute-force search. Silver et al. (2016) combined reinforcement learning via self-play with Monte Carlo tree search (MCTS) and learned neural networks; the resulting system defeated Lee Sedol in 2016. Silver et al. (2017) then showed that human training data was unnecessary.

AlphaGo Zero uses a single deep residual network $f_\theta(s) = (\mathbf{p}, v)$ that takes a board position s (encoded as raw binary planes without hand-crafted features) and outputs a policy vector $\mathbf{p}$ over legal moves and a scalar value v estimating the probability of winning.[28]

During play, each move is selected by running MCTS from the current position. Each of 1,600 simulations traverses the search tree by choosing at each node the action maximizing

$$Q(s,a) + c_{\text{puct}} \cdot P(s,a) \cdot \frac{\sqrt{\sum_b N(s,b)}}{1 + N(s,a)}$$

where $Q(s,a)$ is the mean value, $P(s,a)$ is the network's prior, and $N(s,a)$ is the visit count.[29] When the traversal reaches an unexplored position, the neural network evaluates it in a single forward pass; the resulting value v propagates back up the path, updating Q and N at each edge. The move with the highest root visit count is selected.

The training loop generates self-play games. At each position s_t, MCTS produces improved move probabilities $\boldsymbol{\pi}_t$ (proportional to visit counts). At game end, the outcome $z \in \{-1, +1\}$ is recorded. Each position yields a training triple $(s_t, \boldsymbol{\pi}_t, z)$, and the network minimizes

$$\ell(\theta) = (z - v)^2 - \boldsymbol{\pi}^\top \log \mathbf{p} + c\|\theta\|^2 \tag{21}$$

combining value prediction loss, policy cross-entropy loss, and L_2 regularization. MCTS acts as a policy improvement operator, since the search probabilities $\boldsymbol{\pi}$ are stronger than the raw network outputs $\mathbf{p}$; training the network to match $\boldsymbol{\pi}$ distills search improvements back into the network.[30]

Go was well-suited to this architecture. Its fixed 19×19 board maps naturally to convolutional networks, its perfect information and deterministic transitions make MCTS's tree structure exact, and the binary game outcome provides an unambiguous training signal. Igami (2020) interprets the architecture in econometric terms, where the policy network is a conditional choice probability (CCP) estimator, the value network is a conditional value function (CVF) estimator, and the system performs CCP estimation and forward simulation jointly, connecting to the approach of Hotz and Miller (1993) in dynamic discrete choice.

[27] The max-ent objective $\max_\pi \sum_t \mathbb{E}[r(s_t, a_t) + \tau\mathcal{H}(\pi(\cdot|s_t))]$ equals the evidence lower bound (ELBO) of the graphical model. Trust-region methods (TRPO, PPO) do not optimize the max-ent objective globally, but each policy update step solves a max-ent subproblem (Schulman et al., 2015, 2017). The $\tau \to 0$ limit recovers standard hard-max algorithms.

[28] The input consists of 17 binary planes on the 19×19 board (eight planes each for Black and White stone positions over the last eight moves, plus one for the current player). The architecture uses 40 residual blocks (79 parameterized layers).

[29] This is a continuous analogue of the upper confidence bound (UCB) strategy, balancing exploitation (Q) and exploration (P/N).

[30] The system that defeated Lee Sedol used fixed network weights throughout the match; the months of self-play constituted the training phase, while the five-game match was purely execution (Section 1.1).

3 The Theory of Reinforcement Learning

3.1 The Geometry of Dynamic Programming

Value iteration (VI) and policy iteration (PI) are the workhorses of dynamic programming. VI applies the Bellman operator repeatedly until convergence; PI alternates between *policy evaluation* (solving a linear system) and *policy improvement* (taking the greedy action). PI converges faster. Why? The answer reveals a connection between dynamic programming and numerical optimization. Policy iteration is Newton's method applied to the Bellman equation.

3.1.1 Value Iteration as Picard Iteration

Consider the Bellman optimality operator T acting on value functions.

$$(TV)(s) = \max_{a \in \mathcal{A}} \left\{ r(s,a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s,a)V(s') \right\}. \tag{22}$$

This operator is nonlinear due to the max. Value iteration applies T repeatedly: $V_{k+1} = TV_k$. Since T is a γ-contraction in the supremum norm (Denardo, 1967), Banach's fixed-point theorem guarantees $\|V_k - V^*\|_\infty \leq \gamma^k \|V_0 - V^*\|_\infty$.[31] This is Picard iteration, with linear convergence at rate γ.[32] The iteration count to reduce error by a factor of δ is $k = \log(\delta)/\log(1/\gamma)$ (Bertsekas, 1996).

3.1.2 Policy Iteration as Newton's Method

Policy iteration takes a different approach. At the current value estimate $\tilde{V}$, define the greedy policy $\tilde{\pi}(s) = \mathit{argmax}_a\{r(s,a) + \gamma \sum_{s'} P(s'|s,a)\tilde{V}(s')\}$. The *policy evaluation* step solves the linear fixed-point equation $V = T^{\tilde{\pi}}V$ exactly, where $T^{\tilde{\pi}}$ is the policy-specific Bellman operator

$$(T^{\tilde{\pi}}V)(s) = r(s,\tilde{\pi}(s)) + \gamma \sum_{s'} P(s'|s,\tilde{\pi}(s))V(s'). \tag{23}$$

The geometric structure, formalized by Puterman and Brumelle (1979), is as follows: the linear operator $T^{\tilde{\pi}}$ is a supporting hyperplane to the nonlinear operator T at the current iterate.[33] Specifically, the operators satisfy tangency: $T^{\tilde{\pi}}\tilde{V} = T\tilde{V}$, so the linearization agrees with the nonlinear operator at the current iterate. They also satisfy support: $T^{\tilde{\pi}}V \leq TV$ for all V, meaning the linear operator lies weakly below the nonlinear one everywhere, just as a tangent line lies below a convex function. Policy evaluation solves for the fixed point of this linearization exactly. This is precisely the structure of Newton's method; linearize the nonlinear equation at the current point, solve the linearized system, and iterate.[34][35]

[31] This is the Contraction Mapping Theorem. The identical mathematical structure governs convergence of value function iteration in consumption-savings models, competitive equilibrium computation, and Bellman equation solution.

[32] Picard iteration is $x_{k+1} = f(x_k)$ for finding roots of $x = f(x)$. When f is a contraction, convergence is geometric. The rate γ means each iteration reduces the error by a fixed proportion; more patient agents (higher γ) face slower convergence because the operator contracts less per step.

[33] A supporting hyperplane to a convex function at x_0 is a linear function ℓ with $\ell(x_0) = f(x_0)$ and $\ell(x) \leq f(x)$ everywhere. In plainer terms: $T^{\tilde{\pi}}$ is the tangent-line approximation to T from elementary calculus, extended to function spaces. The policy operator $T^{\tilde{\pi}}$ plays this role for the Bellman operator.

[34] The Newton interpretation of policy iteration has precursors in Kleinman (1968) for Riccati equations in linear-quadratic control and Pollatschek and Avi-Itzhak (1969) for stochastic games.

[35] Algebraically: consider finding the root of $G(V) = V - TV = 0$. The Bellman operator T is piecewise affine, not smooth: T is affine on each region where the greedy policy is constant, with kinks at boundaries where the optimal action switches. This makes G a semismooth function in the sense of Qi and Sun (1993). At any iterate V_k where the greedy policy $\tilde{\pi}$ is unique (a generic condition), T is locally affine: $TV = r^{\tilde{\pi}} + \gamma P^{\tilde{\pi}}V$, so

Theorem 1 (Policy Improvement, Howard (1960)). *Let π_k be the current policy with value V^{π_k}, and let π_{k+1} be the greedy policy with respect to V^{π_k}:*

$$\pi_{k+1}(s) = \underset{a \in \mathcal{A}}{\mathit{argmax}} \left\{ r(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V^{\pi_k}(s') \right\}. \tag{24}$$

Then $V^{\pi_{k+1}}(s) \geq V^{\pi_k}(s)$ for all $s \in \mathcal{S}$, with strict inequality at some state unless π_k is already optimal.

The consequence is finite termination. Since there are at most $|\mathcal{A}|^{|\mathcal{S}|}$ deterministic policies and each PI step strictly improves the value function (Theorem 1), PI reaches the exact optimum in finitely many iterations. While VI requires $k = \log(100)/\log(1/\gamma)$ iterations to reduce error by a factor of 100 (Bertsekas, 1996), PI typically converges in 5–10 iterations regardless of γ.[36][37] At $\gamma = 0.90$, VI needs 44 iterations while PI needs only 5–8; at $\gamma = 0.95$, VI requires 90 versus 5–8; at $\gamma = 0.99$, VI needs 459 iterations while PI still converges in 5–10.

Bertsekas (2022b) extends this interpretation to a broad class of dynamic programming problems. The Newton structure applies whenever the Bellman operator can be written as a pointwise maximum over linear operators: $T = \max_\pi T^\pi$. This includes not only infinite horizon problems with discounting but also optimal stopping problems (job search, option exercise), average cost optimization (inventory, queueing), and minimax formulations for adversarial settings.[38][39] The practical implication is that algorithms with policy-improvement structure (evaluate a policy exactly or approximately, then improve) inherit Newton-like convergence behavior, while pure value-iteration methods (apply T directly) are limited to linear convergence.[40]

3.1.3 Simulation Study: The Brock–Mirman Economy

The Brock and Mirman (1972) optimal growth model provides a concrete demonstration. A planner chooses capital k' to maximize $\sum_{t=0}^{\infty} \beta^t \log(c_t)$ subject to the resource constraint $c_t + k_{t+1} = z_t k_t^\alpha$, where productivity $z_t \in \{0.9, 1.1\}$ follows a Markov chain with persistence 0.8. I set $\alpha = 0.36$, $\beta = 0.96$, and discretize capital on a 500-point grid covering two productivity

$G'(V_k) = I - \gamma P^{\tilde{\pi}}$. The Newton step $V_{k+1} = V_k - [G'(V_k)]^{-1} G(V_k) = (I - \gamma P^{\tilde{\pi}})^{-1} r^{\tilde{\pi}}$ is exactly the policy evaluation solution. At the non-smooth boundary points where two actions tie, any element of the B-subdifferential yields the same iterate because the two candidate linearizations produce the same fixed point.

[36] Ye (2011) proves PI is strongly polynomial with iteration count $O\left(\frac{|\mathcal{S}||\mathcal{A}|}{1-\gamma} \log \frac{|\mathcal{S}|}{1-\gamma}\right)$, resolving a long-standing conjecture. This bound is for fixed γ; Fearnley (2010) constructs examples requiring exponentially many iterations when γ is allowed to vary with $|\mathcal{S}|$.

[37] For continuous-state problems discretized on a grid (the norm in economics), the finite-termination argument still applies to the discretized problem, but the number of grid points n enters the bound. Santos and Rust (2004) establish a three-tier convergence result for PI applied to discretized dynamic programs: order ≈ 1.5 globally for general interpolation schemes, quadratic convergence locally when the value function approximation is concave and piecewise linear, and superlinear convergence for general smooth interpolation. The formal error constants $C(h)$ in their quadratic bound degrade as the grid mesh $h \to 0$, but the iteration count is empirically independent of grid size.

[38] Rust (1996) surveys successive approximation and policy iteration methods for economic models, comparing their performance on the bus engine replacement problem. Zhang (2023) extends randomized policy iteration to multi-agent problems where the control is m-dimensional, reducing per-iteration complexity from exponential to linear in m.

[39] These problems appear under different names such as "stochastic shortest path" for optimal stopping, "average-cost MDP" for long-run average optimization, and "model predictive control" (or receding-horizon control) for finite-horizon replanning.

[40] Blackwell (1965) proves that for discounted MDPs with finite state and action spaces, a stationary deterministic policy $\pi : \mathcal{S} \to \mathcal{A}$ exists that is optimal for all initial states simultaneously. This "uniform optimality" has three implications: (1) the search space reduces from history-dependent or stochastic policies to static maps, justifying neural networks that condition only on current state; (2) the optimal policy is independent of the initial distribution d_0, so changing where episodes start does not require retraining; (3) PI and VI are guaranteed to converge to the same globally optimal policy regardless of initialization.

Table 3: Brock–Mirman Economy: VI vs PI vs LP

Regime	Method	Iterations	Time (s)	$\|V - V^*\|_\infty$
1. Contraction	VI	567	32.66	9.7e-11
	PI	11	0.62	0.0e+00
2. LP dual	VI	—	0.006	—
	LP	—	0.023	2.3e-09
3. Rate (n_k=10)	VI	567	0.003	—
	PI	7	0.000	—
3. Rate (n_k=200)	VI	567	3.097	—
	PI	10	0.067	—

states (1,000 states). This model admits the closed-form policy $k'(k,z) = \alpha\beta z k^\alpha$, providing an exact benchmark.

The discretized model makes the PI–Newton equivalence concrete. Define the Bellman residual $G(V) = V - TV$ on $\mathbb{R}^n$ where $n = 1{,}000$ (500 capital grid points $\times$ 2 productivity states). At iterate V_k, let π_k denote the greedy policy. Since π_k is unique at V_k (generically), T is locally affine: $TV = r^{\pi_k} + \gamma P^{\pi_k} V$, so the residual becomes $G(V) = (I - \gamma P^{\pi_k})V - r^{\pi_k}$ with Jacobian $G'(V_k) = I - \gamma P^{\pi_k}$. The Newton update is

$$V_{k+1} = V_k - [G'(V_k)]^{-1}\, G(V_k) = (I - \gamma P^{\pi_k})^{-1}\, r^{\pi_k}, \tag{25}$$

which is exactly the policy evaluation step: solving $V = r^{\pi_k} + \gamma P^{\pi_k} V$ for V. Each PI iteration is one Newton step on the Bellman residual, explaining the 11-iteration convergence on a 1,000-state problem.[41]

The VI iteration count follows from the contraction bound: each iteration reduces the Bellman residual by the factor $\beta = 0.96$, requiring $k = \lceil \log(\epsilon/\|TV_0 - V_0\|_\infty)/\log\beta \rceil = 567$ iterations for tolerance $\epsilon = 10^{-10}$.[42]

The following geometric interpretation applies to a scalar Bellman equation with three policies. Each policy operator $T^{\pi_i}V = r^{\pi_i} + \gamma_i V$ is affine; the Bellman operator $T = \max_i T^{\pi_i}$ is their upper envelope, a convex piecewise-linear function. VI proceeds by staircase iterates $V_{k+1} = TV_k$, alternating between the T curve and the 45° line, converging at the linear rate γ. PI identifies the active policy operator at each iterate and solves for its fixed point on the 45° line, jumping directly to the intersection. This is a Newton step, where each T^{π_k} is a supporting hyperplane to T at V_k, and the fixed point of the linearization is the Newton iterate. The scalar picture extends to $\mathbb{R}^n$: the affine operator $T^{\pi_k}V = r^{\pi_k} + \gamma P^{\pi_k} V$ supports T at V_k, and its fixed point $(I - \gamma P^{\pi_k})^{-1} r^{\pi_k}$ is the Newton iterate from equation (25). Finite termination follows because T has finitely many affine pieces; the iteration count depends on the number of policy switches, not the state-space dimension.

Table 3 confirms the theory. VI requires 567 iterations at rate $\beta^n = 0.96^n$; PI converges in 11, a 50× reduction predicted by the Newton interpretation. The Manne (1960) LP recovers the same value function to solver precision ($\|V_{\mathrm{LP}} - V_{\mathrm{VI}}\|_\infty < 10^{-8}$).[43]

[41] Santos and Rust (2004) is the definitive reference on PI convergence for discretized economic models of precisely this type. Their analysis of the Brock–Mirman growth model establishes that PI iteration counts are empirically independent of grid resolution (7–11 iterations across all grid sizes in Table 3), consistent with the Newton interpretation: Newton's method converges in a number of steps determined by the nonlinearity of the operator, not the dimension of the discretization.

[42] At $\beta = 0.99$, the weaker contraction yields approximately four times as many iterations, since $\log(0.96)/\log(0.99) \approx 4$.

[43] PI wall-clock time scales favorably: at $n_k = 200$, PI is roughly 50× faster than VI (Table 3), because the $O(n^3)$ per-iteration cost of policy evaluation is offset by 7–10 total iterations versus 567.

3.2 Value Learning Methods

3.2.1 Stochastic Approximation Foundations

When P is unknown, a single sampled transition (s, a, r, s') can replace the expectation $\mathbb{E}_{s' \sim P(\cdot|s,a)}[V(s')]$. The mathematical foundation is stochastic approximation, developed by Robbins (1952).[44] Consider the problem of finding x^* such that $g(x^*) = 0$, where g cannot be evaluated directly but one can observe noisy samples $g(x) + \epsilon$. The Robbins-Monro iteration is:

$$x_{t+1} = x_t - \alpha_t[g(x_t) + \epsilon_t], \tag{26}$$

where ϵ_t is zero-mean noise. Under two conditions on the step sizes, this converges to x^* with probability one. The conditions are $\sum_{t=0}^{\infty} \alpha_t = \infty$ (sufficient exploration, ensuring that learning never ceases) and $\sum_{t=0}^{\infty} \alpha_t^2 < \infty$ (diminishing noise, ensuring the variance of cumulative updates is finite). The canonical choice $\alpha_t = 1/(t+1)$ satisfies both conditions.

3.2.2 Q-Learning and SARSA

Q-learning (Watkins and Dayan, 1992) is Robbins-Monro applied to the Bellman equation for action-value functions.[45] Define the Q-factor Bellman operator:

$$(FQ)(s, a) = r(s, a) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q(s', a'). \tag{27}$$

The Q-learning update, upon observing transition (s_t, a_t, r_t, s_{t+1}), is

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha_t \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]. \tag{28}$$

The logic of Q-learning is best understood as a Monte Carlo approximation of the Bellman contraction. The true Bellman operator involves an integral over the transition distribution, $(FQ)(s, a) = r + \gamma \int \max_{a'} Q(s', a')\, dP(s'|s, a)$. Since P is unknown, this integral cannot be computed analytically. However, a sample transition (s, a, r, s') acts as a single-point Monte Carlo estimate of this integral. The Q-learning update is simply an exponential moving average (with weight α_t) between the current estimate and this noisy Monte Carlo target. Because F is a γ-contraction in the supremum norm, the expected update drives the estimate toward the fixed point Q^*, provided the noise in the Monte Carlo sample averages out over time (which the Robbins-Monro conditions ensure) (Tsitsiklis, 1994).[46]

Convergence requires two conditions. Exploration (visiting all state-action pairs infinitely often) ensures identification. The Robbins-Monro step-size conditions ($\sum_t \alpha_t = \infty$, $\sum_t \alpha_t^2 < \infty$) balance tracking versus noise suppression. Watkins and Dayan (1992) and Jaakkola et al. (1994) formalize these; Tsitsiklis (1994) provides the general framework.

[44] The modern theory of stochastic approximation, including convergence rates and the ODE (ordinary differential equation) method for analyzing iterates, is developed in Kushner and Clark (1978) and Borkar and Meyn (2000). The ODE method shows that the expected trajectory of the stochastic iterates tracks the solution of a deterministic differential equation $\dot{x} = -g(x)$, providing stability conditions via Lyapunov theory.

[45] The "Q" in Q-learning stands for "quality," following Watkins and Dayan (1992), who used $Q(s, a)$ to denote the quality (expected return) of taking action a in state s. The term "Q-factor" is used interchangeably with "action-value function" throughout the RL literature.

[46] Q-learning can be viewed as root-finding for the expected Bellman residual: the goal is to find parameters such that $\mathbb{E}_{(s,a,r,s')}[\delta_t] = 0$, where $\delta_t = r + \gamma \max_{a'} Q(s', a') - Q(s, a)$ is the temporal difference error. The update is a stochastic gradient step on the mean-squared Bellman error, but with a critical distinction: the gradient is "semi-gradient" because the target $\max_{a'} Q(s', a')$ is treated as a fixed constant rather than a function of the parameters being updated. This simplifies computation but disconnects the update from true gradient descent, requiring the specific stability conditions of Tsitsiklis (1994).

The choice of step-size schedule has quantitative consequences: Even-Dar and Mansour (2003) show that polynomial schedules $\alpha_t = 1/t^\omega$ with $\omega \in (1/2, 1)$ achieve convergence rate $O(1/t^{1-\omega})$, creating an explicit tradeoff between speed and stability. Recent work by Li et al. (2024a) establishes that Q-learning with variance-reduced updates achieves minimax-optimal sample complexity $\tilde{O}(|\mathcal{S}||\mathcal{A}|/(1-\gamma)^3\epsilon^2)$, matching information-theoretic lower bounds.[47] Model-free learning is possible. The optimal value function can be found without ever estimating the transition probabilities.[48]

SARSA provides an on-policy variant.[49] Instead of taking the maximum over next actions, SARSA uses the action actually taken:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha_t \left[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right]. \tag{29}$$

This solves for the value function of the behavior policy π rather than the optimal policy. Singh et al. (2000) prove convergence under the same step-size conditions, provided the behavior policy converges to a stationary distribution. Q-learning is noisy value iteration on Q-factors; SARSA is noisy *policy evaluation*.[50] The Robbins-Monro conditions ensure that the noise averages out faster than the signal decays, allowing asymptotic convergence despite using only single-sample estimates.

3.2.3 Multi-Step Returns and TD(λ)

The updates in (28) bootstrap from a single successor state. More generally, the n-step return $G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n V(S_{t+n})$ bootstraps from n steps ahead, interpolating between TD(0) ($n = 1$) and Monte Carlo ($n \to \infty$). The λ-return (Sutton, 1988) averages all n-step returns with geometrically decaying weights, $G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$, where $\lambda \in [0, 1]$ controls the bias-variance tradeoff. Lower λ gives higher bias (more bootstrapping) but lower variance; higher λ relies more on stochastic returns. An equivalent backward-view implementation uses eligibility traces to distribute each TD error to recently visited states, enabling online $O(|\mathcal{S}|)$ per-step credit assignment.[51] Under linear function approximation, TD(λ) converges to a unique fixed point with approximation error bounded by $\frac{1-\lambda\gamma}{\sqrt{1-\gamma^2}}$ times the best-in-class error (Tsitsiklis and Van Roy, 1997).

3.2.4 Finite-Sample Theory of Fitted Methods

Fitted Q-Iteration and Fitted Value Iteration (Definition 2.1.10, Section 2.1.10) replace exact Bellman applications with projected regression steps. The projection introduces approximation error that compounds across iterations.

[47] Vanilla Q-learning (without variance reduction) has tight complexity $\tilde{\Theta}(|\mathcal{S}||\mathcal{A}|/(1-\gamma)^4\epsilon^2)$, worse by a factor of $1/(1-\gamma)$ due to maximization bias inflating variance. The optimal cubic rate requires variance-reduced updates (Wainwright, 2019; Sidford et al., 2018). By contrast, naive model-based RL (estimate $\hat{P}$ from samples, then solve by planning) achieves the optimal $(1-\gamma)^{-3}$ rate with no special tricks (Agarwal et al., 2020b), illustrating the statistical cost of discarding transition structure.

[48] Model-free methods are essential when (a) the environment is a physical system with dynamics too complex to write down, or (b) the agent learns directly from interaction. Note that "model-free" does not mean "atheoretical." The agent does not store $P(s'|s, a)$ explicitly, but the Q-function serves as an implicit model encoding long-run consequences.

[49] SARSA is named for the quintuple $(S_t, A_t, R_t, S_{t+1}, A_{t+1})$ used in each update (Rummery and Niranjan, 1994). Convergence requires the GLIE (Greedy in the Limit with Infinite Exploration) condition: the behavior policy must explore all actions infinitely often while converging to a greedy policy. ε-greedy with $\varepsilon_t \to 0$ satisfies this.

[50] Bhandari et al. (2021) provide finite-time analysis of TD learning, showing that the convergence rate depends on the mixing time of the Markov chain under the behavior policy. Faster mixing (less serial correlation in the state sequence) yields faster convergence.

[51] The forward and backward views produce identical total weight changes over a complete episode (Sutton, 1988). Off-policy extensions include Retrace(λ) (Munos et al., 2016) and V-trace (Espeholt et al., 2018).

Define the inherent Bellman approximation error

$$\varepsilon_{\text{approx}} = \inf_{f \in \mathcal{F}} \|\mathcal{T}f - f\|_{p,\mu}, \tag{30}$$

the smallest residual achievable when the Bellman operator maps any element of $\mathcal{F}$ back to itself. Munos and Szepesvári (2008) show that after K iterations with N i.i.d. samples per iteration,

$$\|V_K - V^*\|_{p,\rho} \;\leq\; C_{\rho,\mu} \left[\gamma^K \|V_0 - V^*\|_{p,\rho} + \frac{\varepsilon_{\text{approx}}}{(1-\gamma)^2} + O\Big(\frac{1}{\sqrt{N}}\Big) \right], \tag{31}$$

where $C_{\rho,\mu}$ is a *concentrability coefficient* bounding the ratio of future-state distributions under the evaluation distribution ρ relative to the data distribution μ.[52] Three terms drive the error: the geometric decay γ^K (initialization bias), the approximation error $(1-\gamma)^{-2}\varepsilon_{\text{approx}}$ (bias from function class), and the estimation error $O(1/\sqrt{N})$ (variance from finite samples). When $\mathcal{F}$ contains V^* exactly, $\varepsilon_{\text{approx}} = 0$ and the bound recovers exact convergence as $K \to \infty$. The $(1-\gamma)^{-2}$ amplification, one factor of $(1-\gamma)^{-1}$ more than in tabular Q-learning, reflects error accumulation across approximate DP steps: each regression step introduces bias, and this bias compounds over K iterations.

Both algorithms solve projected Bellman equations. When $V^* \in \text{span}(\Phi)$ exactly, FVI converges to V^* in a single projected iteration, since the normal equations (15) recover θ_V^* satisfying $\Phi\theta_V^* = V^*$. For FQI, the per-action Q-functions satisfy $Q^*(s, a^*(s)) = V^*(s)$ at the optimal action $a^*(s)$, so when $Q^*(\cdot, a)$ is also representable in $\text{span}(\Phi)$ for each a, FQI recovers consistent value estimates $\phi(s)^\top \theta^*_{a^*(s)} = \phi(s)^\top \theta_V^*$ at convergence. Whether FQI succeeds therefore depends on the geometry of the problem: when $Q^*(\cdot, a) \notin \text{span}(\Phi)$, as on the Brock–Mirman economy, where the per-action Q-function requires fractional-power terms $k^{-n\alpha}$ outside the log-polynomial span, FQI stalls at error 1.65 while FVI converges to 0.001; when Q^* lies exactly in $\text{span}(\Phi)$, both FVI and FQI converge to near-zero error. Section 3.2.5 shows that replacing the linear basis with a nonlinear parametric model that matches the log-Cobb–Douglas structure of Q^* restores FQI convergence on the Brock–Mirman economy.

3.2.5 Simulation Study: Basis Representability on the Brock–Mirman Economy

The Brock–Mirman stochastic growth model (Brock and Mirman, 1972) provides the negative case. The economy has $|\mathcal{S}| = 100$ states ($N_K = 50$ capital grid points, $N_Z = 2$ productivity levels) and $|\mathcal{A}| = 50$ actions. We use the same log-polynomial basis $\phi(k, z) = [1, \log k, k/\bar{k}, (k/\bar{k})^2, (k/\bar{k})^3] \otimes [\mathbb{1}_{z=z_\ell}, \mathbb{1}_{z=z_h}]$ from the theory discussion above, which contains V^* up to residual $\|\Pi_\Phi V^* - V^*\|_\infty = 0.0002$. To test whether the failure is inherent to FQI or to the basis, we add two methods that use the structurally correct per-action feature $\log(zk^\alpha - k')$, the log-consumption implied by the Cobb–Douglas technology. Oracle-FQI treats $\alpha = 0.36$ as known and runs standard OLS per action with three parameters $[\mathbb{1}_{z=z_\ell}, \mathbb{1}_{z=z_h}, \log(zk^\alpha - k')]$. NLLS-FQI estimates α jointly via concentrated least squares: for each candidate α, it solves conditional OLS for intercepts and slope, then optimizes α to minimize total residual sum of squares, initialized at the deliberately wrong value $\alpha_0 = 0.5$.[53]

Table 4 confirms the diagnosis: basis representability, not algorithmic failure. FVI converges near the projection floor of the log-polynomial basis; linear FQI stalls at error 1.65, confirming $Q^*(\cdot, a) \notin \text{span}(\Phi)$. Oracle-FQI and NLLS-FQI, using the structurally correct log-consumption

[52] The concentrability coefficient $C_{\rho,\mu}$ measures how well the data distribution μ covers future states reachable under optimal policies from ρ. When μ is the state-action distribution of the optimal policy itself, $C_{\rho,\mu} = 1$. Distribution mismatch, common when batch data comes from a sub-optimal behavior policy, inflates $C_{\rho,\mu}$ and worsens the bound. Antos et al. (2008) extend these results to continuous action spaces and single-trajectory data.

[53] Observations where $zk^\alpha - k' \leq 0$ (infeasible consumption) contribute a penalty equal to the mean squared target, preventing the optimizer from improving RSS by shrinking the feasible set.

feature, match exact VI with error below 10^{-4}. NLLS-FQI recovers $\hat{\alpha} = 0.3600$ in a single iteration, demonstrating that the same FQI algorithm succeeds when the function class contains Q^*.

Method	$\|V - V^*\|_\infty$	$\|V - V^*\|_{\text{rms}}$	Policy agreement (%)	Iterations
Exact VI	0.0000	0.0000	98.0	—
FVI (linear)	0.0010	0.0008	99.0	341
FQI (linear)	1.6521	1.4805	4.0	339
Oracle-FQI	0.0000	0.0000	98.0	341
NLLS-FQI ($\hat{\alpha} = 0.3600$)	0.0000	0.0000	98.0	341

Table 4: Convergence metrics for five methods on the Brock–Mirman economy ($N_K = 50$, $N_Z = 2$, $\gamma = 0.96$). Policy agreement is measured against the closed-form optimal policy $k' = \alpha\beta z k^\alpha$.

3.2.6 Rollout, Lookahead, and AlphaZero

Two constructions bridge the Newton interpretation of Section 3.1 to practical algorithms. Given a base policy μ with value function V^μ, the *rollout* policy selects

$$\mu_R(s) = \underset{a \in \mathcal{A}}{argmax} \left\{ r(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V^\mu(s') \right\}. \tag{32}$$

This is one step of policy iteration starting from μ: the Policy Improvement Theorem (Theorem 1) guarantees $V^{\mu_R}(s) \geq V^\mu(s)$ for all s, with strict inequality unless μ is already optimal (Bertsekas, 2021).[54] Given an arbitrary approximate value function $\tilde{V}$ (not necessarily the value of any policy), the *one-step lookahead* policy selects

$$\tilde{\pi}(s) = \underset{a \in \mathcal{A}}{argmax} \left\{ r(s,a) + \gamma \sum_{s'} P(s'|s,a)\, \tilde{V}(s') \right\}. \tag{33}$$

When $\tilde{V} = V^\mu$, lookahead and rollout coincide. The distinction matters because rollout inherits the monotone improvement guarantee (it starts from the value of a policy), while lookahead from an arbitrary $\tilde{V}$ has no such monotonicity. The Newton interpretation from Section 3.1 explains why lookahead nevertheless helps: both constructions solve the linearized Bellman equation at the current iterate.[55]

An ℓ-step lookahead extends this by applying $(\ell - 1)$ steps of value iteration before the final greedy selection. The first $(\ell - 1)$ steps are ordinary Bellman contractions, each shrinking the approximation error by a factor of γ. Only the final step, the greedy policy improvement, constitutes the Newton step.[56] The resulting error bound is

$$\|V^{\tilde{\pi}} - V^*\|_\infty \leq \gamma^\ell\, \|\tilde{V} - V^*\|_\infty, \tag{34}$$

where the γ^ℓ factor reflects ℓ total contractions (Bertsekas, 2022a, Prop. 2.3.1). Deep lookahead compensates for poor approximation through repeated contraction, not through repeated Newton steps.

[54] Bertsekas uses cost-minimization notation throughout his work, writing min where standard RL uses max and defining value as accumulated cost rather than reward. I translate to the reward-maximization convention used elsewhere in this chapter; the mathematics are equivalent with reversed inequalities.

[55] Rollout requires a simulator (generative model) that can be queried from arbitrary states, a stronger assumption than the trajectory-based access of Q-learning and SARSA.

[56] Bertsekas (2021) states this explicitly: "whatever follows the first step of the lookahead is preparation for the Newton step." The preceding value iteration steps have linear convergence at rate γ; only the terminal improvement step has superlinear character.

Recall the AlphaGo Zero system from Section 2.2.5, where a neural network $f_\theta(s) = (\mathbf{p}, v)$ outputs a prior policy $\mathbf{p}$ and a value estimate v for any board position s. During play, the network does not act alone: Monte Carlo Tree Search runs simulated games from the current position, using v to evaluate leaf nodes and $\mathbf{p}$ to guide which branches to explore.[57] The network provides $\tilde{V}$; MCTS applies multi-step lookahead through selective tree expansion. Table 5 makes the correspondence explicit.

Step	Policy Iteration	AlphaZero
Initialize	Arbitrary π_0	Random network f_θ
Evaluate	Solve $V = T^{\pi_k} V$ exactly	Network value head $v \approx V^{\pi_k}$
Improve	$\pi_{k+1} = argmax_a \{r + \gamma P V^{\pi_k}\}$	MCTS simulations from v
Iterate	Repeat until π is stationary	Retrain f_θ on self-play outcomes

Table 5: Policy iteration and AlphaZero follow the same evaluate-improve loop. The network provides approximate policy evaluation; MCTS provides approximate policy improvement via selective tree search.

The gap between network-only play and network-plus-search is the contraction factor γ^H, where H is the effective search depth.[58]

3.3 The Central Challenge: The Deadly Triad

State spaces are too large for lookup tables (Go has 10^{170} states; most economic models have continuous state variables). Practitioners must combine three ingredients: *function approximation* (to handle large state spaces), bootstrapping (to learn from single transitions rather than complete episodes), and off-policy learning (to learn about the optimal policy while exploring, or to reuse old data). Each is desirable in isolation. Their interaction, known as the *deadly triad* (Sutton and Barto, 2018, Ch. 11), is the central open problem in reinforcement learning theory.

Off-policy learning is preferred for three reasons. First, sample efficiency: transitions collected under any behavioral policy can be reused to evaluate or improve a different target policy, amortizing the cost of data collection. Discarding data because it was generated by a superseded policy is wasteful. Second, exploration and exploitation separate cleanly: the agent can follow an exploratory policy (e.g., ε-greedy) to ensure adequate state-space coverage while simultaneously learning the optimal deterministic policy. On-policy methods such as SARSA entangle the two, learning the value of the exploratory policy rather than the optimal one. Third, off-policy evaluation answers "what would have happened under policy π?" from data generated by policy μ, the counterfactual question at the heart of policy comparison.

3.3.1 The Projected Bellman Operator

With function approximation $V(s) \approx \phi(s)^\top \theta$, the parameter vector θ is shared across states. Updating θ to improve the value estimate at one state simultaneously changes the estimate at every other state. The algorithm can no longer apply the Bellman operator T^π to each state independently; instead, it applies T^π to compute a target, then *projects* the result back onto the function space (the span of the features ϕ). The composed operator is ΠT^π, the *projected Bellman operator*, where Π denotes this projection (Tsitsiklis and Van Roy, 1997). Convergence of the approximate iteration $\theta_{k+1} = \Pi T^\pi \theta_k$ requires ΠT^π to be a contraction.

[57] AlphaGo Zero uses 1,600 MCTS simulations per move for Go; the generalized AlphaZero algorithm uses 800 simulations per move across chess, shogi, and Go (Silver et al., 2018, Table S3).

[58] MCTS adds UCB exploration and selective tree expansion beyond the literal lookahead framework. The Newton interpretation explains why lookahead helps (the final greedy selection is a policy improvement step) but does not explain the specific mechanisms that make MCTS computationally efficient.

In the on-policy setting, Π minimizes squared error weighted by d^π, the stationary distribution of the policy being evaluated, so $\Pi V = \arg\min_{\hat{V}\in\text{span}(\Phi)} \|V - \hat{V}\|_{d^\pi}$, where $\|V\|^2_{d^\pi} = \sum_s d^\pi(s)V(s)^2$. The Bellman operator T^π is a γ-contraction in the same d^π-norm. Because both operators use the same norm, the projection Π is *orthogonal*, meaning the residual $V - \Pi V$ is perpendicular to the approximation subspace. The Pythagorean theorem then gives $\|\Pi V\|^2_{d^\pi} + \|V - \Pi V\|^2_{d^\pi} = \|V\|^2_{d^\pi}$, so $\|\Pi V\|_{d^\pi} \leq \|V\|_{d^\pi}$.[59] The projection cannot expand distances. The composition therefore contracts:

$$\|\Pi T^\pi V_1 - \Pi T^\pi V_2\|_{d^\pi} \leq \underbrace{\|\Pi\|}_{\leq 1} \cdot \underbrace{\|T^\pi V_1 - T^\pi V_2\|_{d^\pi}}_{\leq \gamma\|V_1 - V_2\|_{d^\pi}} < \|V_1 - V_2\|_{d^\pi}. \tag{35}$$

A unique fixed point $\Phi\theta^*$ exists, and TD(λ) converges to it with probability one. The resulting approximation error satisfies $\|\Phi\theta^* - V^\pi\|_{d^\pi} \leq \frac{1-\lambda\gamma}{\sqrt{1-\gamma^2}}\|\Pi V^\pi - V^\pi\|_{d^\pi}$, bounding the TD solution's error by a multiple of the best possible approximation error (Tsitsiklis and Van Roy, 1997, Theorem 1).

3.3.2 Why Off-Policy Learning Diverges

In the off-policy setting, samples come from a behavior distribution $\mu \neq d^\pi$. The projection now minimizes error under μ, but the Bellman operator T^π still contracts in the d^π-norm. The two operators measure distance in different norms. The projection is no longer orthogonal in the d^π-norm; it is *oblique*. Unlike orthogonal projections, oblique projections can expand distances, with $\|\Pi_\mu\|_{d^\pi}$ exceeding 1 in the worst case. If $\|\Pi_\mu\|_{d^\pi} > 1/\gamma$, the expansion from projection overwhelms the γ-contraction from the Bellman operator, and the fixed-point iteration diverges.

This divergence is not overfitting. Overfitting occurs when the approximator memorizes training data at the expense of generalization; collecting more data helps. Divergence means the parameter vector θ grows without bound, producing arbitrarily large value estimates that bear no relation to the true values. More data does not help; the algorithm itself is unstable. The distinction matters because the remedies are entirely different. Regularization and early stopping address overfitting, while the deadly triad requires structural changes to the algorithm.

Baird (1995) constructed a six-state star MDP that makes this failure concrete. All rewards are zero, so the true value is $V^*(s) = 0$ for every state. A lookup table learns this immediately. The MDP has a star topology: states 1 through 5 each transition to state 6, and state 6 transitions to itself. Linear function approximation uses a shared weight w_1 across all states plus a state-specific weight, with $V(s) = 2w_1 + w_s$ for $s \in \{1, \ldots, 5\}$ and $V(6) = 2w_1 - w_6$. Training samples all transitions equally often (uniform distribution, not d^π). The dynamics are as follows.[60] When $V(6)$ is large and positive, the TD target $\gamma V(6)$ exceeds $V(s)$ for states 1 through 5, producing positive TD errors that push w_1 upward. At state 6, the TD target is $\gamma V(6) < V(6)$, producing a negative TD error that pushes w_1 downward. But states 1 through 5 are each visited as often as state 6, so w_1 receives five upward pushes for every one downward push. The shared weight diverges to $+\infty$. The on-policy distribution would concentrate mass on state 6 (the absorbing state), counterbalancing the upward pressure; uniform sampling destroys this balance.[61]

[59] The same argument holds in $\mathbb{R}^n$. Projecting a vector onto a subspace never makes it longer. This is the geometric content of the Cauchy-Schwarz inequality. In the function-approximation setting, the "vector" is a value function, the "subspace" is the span of features, and "length" is the d^π-weighted L^2 norm.

[60] Baird (1995) also presents an MDP variant with two actions per state, demonstrating that Q-learning diverges under the same mechanism. The mechanism is identical: shared parameters create cross-state coupling that uniform sampling cannot counterbalance.

[61] The gradient of $\|Q - TQ\|^2$ requires two independent next-state samples from the same (s, a), since $\nabla\mathbb{E}[(Q - \mathbb{E}[r + \gamma V(s')])^2]$ involves $\mathbb{E}[\cdot] \cdot \nabla\mathbb{E}[\cdot]$ and $\mathbb{E}[XY] \neq \mathbb{E}[X]\mathbb{E}[Y]$. This "double-sampling" requirement is impractical (Baird, 1995), so practitioners use semi-gradient TD, treating the bootstrap target as a constant. This semi-gradient structure makes off-policy TD vulnerable to projection mismatch.

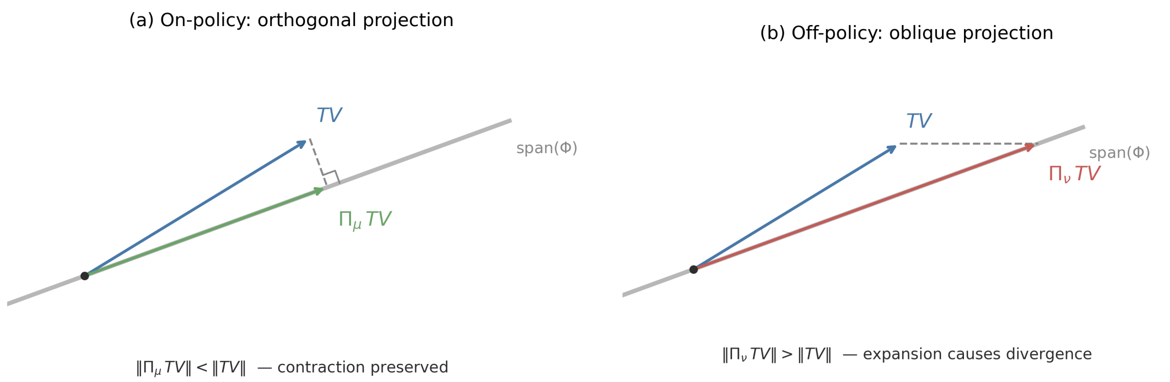

Figure 1: Geometry of the projected Bellman operator in $\mathbb{R}^2$. The gray line is the function approximation subspace span(Φ); the blue arrow is TV, the Bellman update. (a) On-policy: the orthogonal projection Π_μ drops TV perpendicularly onto the subspace, preserving the contraction. (b) Off-policy: the oblique projection Π_ν (under the behavior distribution) reaches a point further from the origin than TV itself, causing expansion.

Each element of the triad is individually necessary for divergence. Without function approximation (tabular), the projection is the identity and Q-learning's contraction applies directly. Without bootstrapping (Monte Carlo returns), targets are independent of current value estimates and the problem reduces to supervised regression. Without off-policy learning, samples come from d^π, the projection is orthogonal, and the Tsitsiklis-Van Roy convergence guarantee holds.

3.3.3 Resolutions

Three classes of algorithms restore convergence, each neutralizing a different component of the triad.

Target networks weaken bootstrapping. Instead of updating toward $r + \gamma Q(s'; \theta)$, where the target moves with each parameter update, DQN (Mnih et al., 2015) updates toward $r + \gamma Q(s'; \theta^-)$, where θ^- is a slowly-updated copy of the parameters.[62] The regression target becomes quasi-static, converting the coupled fixed-point problem into a sequence of supervised learning problems. Zhang et al. (2021) prove that this two-timescale scheme converges to a regularized TD fixed point with linear function approximation. Fellows et al. (2023) show that target networks recondition the Jacobian of the TD update: the spectral radius of the composed update operator depends on the target network update frequency k, and for sufficiently large k the spectral radius drops below 1 even in off-policy settings with nonlinear function approximation.

Gradient TD methods fix the projection mismatch. Sutton et al. (2009) reformulate the projected Bellman error as a saddle-point problem $\min_\theta \max_y L(\theta, y)$, yielding algorithms (GTD, GTD2, TDC) that perform true stochastic gradient descent on the mean-squared projected Bellman error.[63] These methods converge off-policy with linear function approximation because they eliminate the semi-gradient approximation that causes the norm mismatch.

Regularization shrinks the projection operator. Lim and Lee (2024) add an ℓ_2 penalty $-\eta\theta$ to the Q-learning update. This changes the projection from $\Pi = X(X^\top DX)^{-1}X^\top D$

[62] Experience replay (Lin, 1992) complements target networks by breaking temporal correlation in the training data. The two mechanisms address different sources of instability: target networks stabilize the bootstrap target, while replay stabilizes the sampling distribution.

[63] The saddle-point formulation introduces auxiliary variables y of the same dimension as θ, doubling the parameter count and requiring a second learning rate. Bhandari et al. (2021) provide finite-time convergence rates for these two-timescale algorithms.

to $\Pi_\eta = X(X^\top DX + \eta I)^{-1}X^\top D$. As the regularization strength η increases, the projection "shrinks" toward the origin. For sufficiently large η, $\gamma\|\Pi_\eta\| < 1$, restoring the contraction property. The algorithm converges to a biased but stable fixed point, with the bias controlled by η.

3.4 Policy Learning Methods

Value-based methods find fixed points of the Bellman operator. Policy-based methods parameterize the policy directly as $\pi_\theta(a|s)$ and maximize expected return $J(\theta) = \mathbb{E}_{\pi_\theta}[\sum_{t=0}^\infty \gamma^t R_t]$ by gradient ascent. This formulation sidesteps the Bellman equation entirely and frames reinforcement learning as constrained optimization.

3.4.1 The Policy Gradient Theorem

The policy gradient theorem, proved independently by Williams (1992) for the episodic case and Sutton et al. (2000) for the general discounted setting, provides a tractable expression for the gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{s\sim d^{\pi_\theta}, a\sim\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a|s)\, Q^{\pi_\theta}(s,a)\right], \tag{36}$$

where $d^{\pi_\theta}(s) = (1-\gamma)\sum_{t=0}^\infty \gamma^t \mathbb{P}(s_t = s|\pi_\theta)$ is the discounted state visitation distribution. The fundamental econometric challenge in optimizing $J(\theta)$ is that this distribution depends on θ through the environment's dynamics. A naive derivative would require $\nabla_\theta d^{\pi_\theta}(s)$, which implies differentiating the unknown transition matrix $P(s'|s,a)$.

The policy gradient theorem sidesteps this entirely via a likelihood ratio (or score function) trick.[64] The theorem transforms a sensitivity analysis problem (how does the system evolve?) into a simpler expectation problem (what is the correlation between the score $\nabla \log \pi$ and the value Q?). The gradient can be written as an expectation under the current policy, weighted by action-values, without requiring $\nabla_\theta d^{\pi_\theta}$. The transition dynamics $P(s'|s,a)$ do not appear; the gradient is estimable via sample averages from trajectories alone.

3.4.2 REINFORCE and Variance Reduction

REINFORCE (Williams, 1992) is the simplest policy gradient algorithm. Sample a trajectory $(s_0, a_0, r_0, s_1, \ldots)$, compute the return $G_t = \sum_{k=0}^\infty \gamma^k r_{t+k}$ from each time step, and update:

$$\theta \leftarrow \theta + \alpha \sum_t \nabla_\theta \log \pi_\theta(a_t|s_t)\, G_t. \tag{37}$$

This is an unbiased estimator of $\nabla_\theta J(\theta)$, but its variance is high because a single trajectory provides a noisy estimate of Q^{π_θ}. Despite high variance, REINFORCE converges to a globally optimal policy in the tabular setting.[65]

3.4.3 Natural Policy Gradient and Gradient Domination

Standard gradient descent treats all parameter directions equally. But small changes in θ can cause large changes in the policy distribution π_θ. The natural policy gradient (Kakade, 2001), building on the natural gradient framework of Amari (1998), accounts for this curvature by

[64] The score function $\nabla_\theta \log \pi_\theta(a|s)$ is the same mathematical object that appears in the Cramér-Rao bound and the score test in maximum likelihood estimation. The policy gradient is a covariance, $\nabla J = \mathrm{Cov}_{d^{\pi_\theta}\times\pi_\theta}(\nabla \log \pi_\theta, Q^{\pi_\theta})$, measuring how sensitive the log-likelihood of the policy is to parameter changes, weighted by action quality.

[65] The baseline $b(s)$ subtracted from G_t reduces variance while preserving unbiasedness, since $\mathbb{E}[\nabla_\theta \log \pi_\theta(a|s) b(s)] = 0$ for any baseline independent of a.

preconditioning with the Fisher information matrix.[66] The relationship between NPG and standard PG parallels that between Fisher scoring and gradient ascent in MLE. Both precondition with the inverse Fisher information matrix $F(\theta)^{-1}$, achieving parameterization invariance and quadratic convergence near the optimum.

$$\tilde{\nabla}_\theta J(\theta) = F(\theta)^{-1} \nabla_\theta J(\theta), \quad F(\theta) = \mathbb{E}_{s,a}\left[\nabla_\theta \log \pi_\theta(a|s) \nabla_\theta \log \pi_\theta(a|s)^\top\right]. \tag{38}$$

Why does NPG recover policy iteration? Standard gradient ascent is sensitive to parameterization: it takes the steepest step in Euclidean parameter space, where units depend on how the policy is parameterized. NPG takes the steepest step in distribution space (measured by KL-divergence), which is invariant to reparameterization. In the tabular case, Kakade (2001, Theorem 2) proves that this geometric correction aligns the gradient exactly with the greedy policy $\tilde{\pi}$ from policy iteration. With step size 1 (and exact estimation), NPG performs one full Newton step; with smaller step sizes, it performs damped Newton updates. This explains its rapid convergence: NPG approximates the quadratic convergence of finding a fixed point rather than the linear convergence of hill-climbing.

The RL objective $J(\theta)$ is non-convex in θ. For researchers trained to distrust gradient methods on non-convex objectives, the natural concern is convergence to spurious local optima. For tabular softmax policies (one free parameter per state-action pair), this concern is unfounded. The landscape is "benign" in a precise sense. Agarwal et al. (2021) prove that $J(\theta)$ satisfies a *gradient domination* condition (also called Polyak-Łojasiewicz, or PL). The PL condition has the same functional form as the strong convexity condition for guaranteeing linear convergence of gradient descent, but it applies to non-convex functions: whenever $\|\nabla J(\theta)\|$ is small, the policy must be near-optimal. Formally, the sub-optimality $J(\pi^*) - J(\pi_\theta)$ is bounded by a constant times $\|\nabla J(\theta)\|^2$. The implication is immediate: any point where the gradient vanishes is globally optimal. The non-convex landscape has no false peaks, no spurious local maxima. Gradient ascent cannot get trapped.

Mei et al. (2020) sharpen this result for softmax parameterization, proving explicit convergence rates. These guarantees are specific to the tabular parameterization. With function approximation, the PL condition does not hold. Agarwal et al. (2021, Theorem 6.2) show that NPG with log-linear or smooth policy classes (including neural networks) converges to a neighborhood of the optimum whose radius depends on the approximation error of the policy class, not to the global optimum itself.[67]

In the tabular setting, NPG achieves more: *dimension-free* convergence. Standard gradient ascent on $J(\theta)$ has a convergence rate that depends on the smoothness constant, which scales with $|\mathcal{S}|$. NPG circumvents this by preconditioning with the Fisher information matrix $F(\theta)^{-1}$. The mechanism is that the state-visitation distribution $d^{\pi_\theta}(s)$ appears in both $\nabla J(\theta)$ and $F(\theta)$; when computing $F^{-1}\nabla J$, these terms cancel analytically. The resulting update rule is equivalent to soft policy iteration and converges at rate $O(1/(1-\gamma)^2\epsilon)$, independent of $|\mathcal{S}|$ and $|\mathcal{A}|$ (Xiao, 2022).

[66] The Fisher information $F(\theta) = \mathbb{E}[\nabla \log p \nabla \log p^\top]$ measures curvature of the log-likelihood and appears in the Cramér-Rao bound. Here it measures how policy distributions change with parameters.

[67] However, "no spurious local optima" does not imply "easy optimization." The landscape is dominated by vast plateaus (saddle points) where gradients vanish. Without sufficient exploration, the probability of visiting relevant states decays exponentially with the horizon, rendering the gradient exponentially small. Global convergence requires the starting distribution to have adequate coverage relative to the optimal policy's visitation distribution, formalized as the "distribution mismatch coefficient" by Agarwal et al. (2021). The Natural Policy Gradient addresses this by preconditioning with the Fisher Information Matrix, making the update direction covariant: invariant to invertible linear transformations of the parameter space. This standardizes units across parameters, preventing stalling on plateaus caused by poor parameter scaling. Li et al. (2022) make this quantitatively precise: vanilla softmax policy gradient requires iterations doubly exponential in the effective horizon $1/(1-\gamma)$ because score functions are exponentially small in directions corresponding to suboptimal actions.

3.4.4 Trust Region Methods

NPG requires computing and inverting the Fisher information matrix $F(\theta)$, which scales as $O(d^2)$ in parameters and is impractical for neural networks. TRPO (Schulman et al., 2015) approximates the natural gradient using conjugate gradient methods[68] without forming F explicitly, and enforces trust regions via line search.[69] Shani et al. (2020) prove convergence for adaptive trust region methods that adjust the constraint radius dynamically. PPO (Schulman et al., 2017) simplifies further by replacing the hard KL constraint with a clipped surrogate objective, trading theoretical guarantees for computational simplicity. The geometric foundation of trust region methods lies in information geometry. The space of policies $\{\pi_\theta : \theta \in \mathbb{R}^d\}$ forms a statistical manifold, and the natural distance between two nearby policies is the KL divergence, not the Euclidean distance between their parameters (Amari, 1998). To second order, $\mathrm{KL}(\pi_\theta \| \pi_{\theta+\Delta\theta}) \approx \frac{1}{2}\Delta\theta^\top F(\theta)\Delta\theta$, where $F(\theta)$ is the Fisher information matrix. Two parameter vectors θ and θ' that are far apart in Euclidean distance may correspond to nearly identical distributions, while nearby parameters may produce radically different policies. The natural gradient corrects for this by measuring steepest ascent in KL-divergence rather than Euclidean norm. On the policy manifold, the Euclidean gradient $\nabla_\theta J$ points in a direction that ignores curvature, while the natural gradient $F^{-1}\nabla_\theta J$ follows the manifold's intrinsic geometry toward the optimum.

TRPO formalizes this insight as a constrained optimization problem. At each iteration, TRPO maximizes the importance-weighted surrogate

$$\max_\theta \; L(\theta) = \mathbb{E}_{s\sim d^{\pi_{\theta_{\text{old}}}}}\left[\sum_a \frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} A^{\pi_{\theta_{\text{old}}}}(s,a)\right] \quad \text{s.t.} \quad \mathrm{KL}(\pi_{\theta_{\text{old}}} \| \pi_\theta) \le \delta, \tag{39}$$

where $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ is the advantage function. Linearizing $L(\theta)$ around θ_{old} and applying the quadratic KL approximation yields a Lagrangian whose closed-form solution is

$$\theta_{\text{new}} = \theta_{\text{old}} + \sqrt{\frac{2\delta}{g^\top F^{-1} g}}\, F^{-1} g, \qquad g = \nabla_\theta L(\theta)\big|_{\theta_{\text{old}}}. \tag{40}$$

This is precisely the natural gradient direction, scaled so that the step saturates the KL budget δ. The step size is determined entirely by the trust region geometry, not by a learning rate hyperparameter. In practice, TRPO solves the linear system $Fv = g$ via conjugate gradient and performs a backtracking line search to enforce the KL constraint exactly.[70]

The theoretical guarantee underlying TRPO is a majorization-minimization (MM) argument. The surrogate $L(\theta)$ is a local lower bound on $J(\theta)$ that is tight at θ_{old}: $L(\theta_{\text{old}}) = J(\theta_{\text{old}})$ and $L(\theta) \le J(\theta)$ within the trust region.[71] Maximizing L within the trust region therefore guarantees monotonic improvement: $J(\theta_{\text{new}}) \ge L(\theta_{\text{new}}) \ge L(\theta_{\text{old}}) = J(\theta_{\text{old}})$. This is the same pattern as the EM algorithm in statistics, where the E-step constructs a surrogate (the ELBO) and the M-step maximizes it.[72] Shani et al. (2020) prove convergence for adaptive trust region

[68]Conjugate gradient is an iterative method for solving linear systems $Ax = b$ without forming A explicitly, requiring only matrix-vector products Av. With k iterations it costs $O(kd)$ versus $O(d^3)$ for direct inversion, making it feasible for neural networks with millions of parameters.

[69]Trust region methods build on the performance difference lemma: for any two policies π and π', $J(\pi') - J(\pi) = \frac{1}{1-\gamma}\mathbb{E}_{s\sim d^{\pi'}}[\sum_a \pi'(a|s)A^\pi(s,a)]$, where $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ is the advantage function. This identity bounds how much policy improvement is possible and motivates constraining updates to regions where advantage estimates remain accurate (Kakade, 2002).

[70]Conjugate gradient solves $Fv = g$ iteratively using only matrix-vector products Fv, which can be computed via automatic differentiation without forming F explicitly. With k iterations it costs $O(kd)$ versus $O(d^3)$ for direct inversion, making it feasible for neural networks with millions of parameters.

[71]The bound follows from the performance difference lemma: $J(\pi') - J(\pi) = \frac{1}{1-\gamma}\mathbb{E}_{s\sim d^{\pi'}}[\sum_a \pi'(a|s)A^\pi(s,a)]$. Replacing $d^{\pi'}$ with d^π introduces error controlled by the KL divergence between the two policies (Kakade, 2002).

[72]In EM, $Q(\theta|\theta^{(t)})$ lower-bounds the log-likelihood and is tight at $\theta^{(t)}$. Each M-step guarantees $\ell(\theta^{(t+1)}) \ge$

methods that adjust δ dynamically. Each surrogate is a lower bound that touches J at the current iterate, and sequential maximization produces monotonically improving iterates.

PPO (Schulman et al., 2017) replaces the hard KL constraint with a clipped surrogate objective. Let $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ denote the importance ratio. PPO maximizes

$$L^{\text{clip}}(\theta) = \mathbb{E}_t\left[\min\big(r_t(\theta)A_t,\ \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)A_t\big)\right], \tag{41}$$

where ε (typically 0.1–0.2) bounds the ratio. When $A_t > 0$, clipping prevents r_t from exceeding $1+\varepsilon$; when $A_t < 0$, it prevents r_t from falling below $1-\varepsilon$. The resulting feasible region is not an ellipsoid in parameter space but rather a non-convex set determined by the ratio constraint at each sampled state-action pair. PPO requires no Fisher information computation and uses only first-order gradients, making it widely used in large-scale applications. The trust region framework connects naturally to econometric optimization. The Levenberg-Marquardt algorithm for nonlinear least squares uses a similar trust region mechanism, interpolating between gradient descent and Gauss-Newton steps.[73] More broadly, the Fisher information matrix that defines TRPO's trust region is the same object that appears in the Cramér-Rao bound: it measures the statistical precision of the policy parameterization. The natural gradient adapts step sizes to this precision, taking large steps in well-identified directions and small steps where the data provide little information about the policy.

3.5 Hybrid Methods

REINFORCE estimates policy gradients from sample returns (unbiased, high variance); TD methods use bootstrapped targets $r + \gamma V(s')$ (lower variance, biased when V is approximate). Actor-critic methods combine both. The critic estimates the value function, the actor updates the policy using the critic's estimates.

3.5.1 Actor-Critic Architecture and Two-Timescale Convergence

The theoretical foundation is two-timescale stochastic approximation (Konda and Tsitsiklis, 2000), building on the two-timescale ODE convergence theory of Borkar (1997).[74] Run two concurrent learning processes:

$$\text{Critic (fast):}\quad \theta_{t+1} = \theta_t + \alpha_t^{(c)}\delta_t \nabla_\theta \hat{V}(s_t;\theta_t), \tag{42}$$

$$\text{Actor (slow):}\quad \omega_{t+1} = \omega_t + \alpha_t^{(a)}\delta_t \nabla_\omega \log \pi_\omega(a_t|s_t), \tag{43}$$

where $\delta_t = r_t + \gamma\hat{V}(s_{t+1};\theta_t) - \hat{V}(s_t;\theta_t)$ is the TD error. The critic updates the value function parameters θ; the actor updates the policy parameters ω.

Convergence requires the critic to learn faster than the actor.

$$\lim_{t\to\infty} \frac{\alpha_t^{(a)}}{\alpha_t^{(c)}} = 0, \quad \text{with both satisfying Robbins-Monro conditions.} \tag{44}$$

Under this separation, the actor sees a quasi-stationary critic: from the actor's perspective,

$Q(\theta^{(t+1)}|\theta^{(t)}) \geq Q(\theta^{(t)}|\theta^{(t)}) = \ell(\theta^{(t)})$. The TRPO bound has the same structure with L playing the role of Q and J playing the role of ℓ.

[73] Levenberg-Marquardt solves $\min_\theta \|r(\theta)\|^2$ by adding a damping term λI to the Gauss-Newton Hessian approximation $J^\top J$, which is equivalent to constraining the step to a trust region whose radius decreases with λ. TRPO replaces $J^\top J$ with the Fisher information matrix $F(\theta)$.

[74] The original analysis of Konda and Tsitsiklis (2000) uses the average-cost formulation with TD error $\delta_t = c(X_t, U_t) - \Lambda + V(X_{t+1}) - V(X_t)$, where Λ is the average cost. The discounted variant presented here follows by replacing the average-cost baseline with $\gamma V(s')$. A related but distinct ODE stability framework for single-timescale stochastic approximation, including Q-learning and TD learning, appears in Borkar and Meyn (2000).

the critic provides approximately correct value estimates at each step.[75] The actor's updates are then approximately unbiased policy gradient steps. Konda and Tsitsiklis (2000) prove convergence to a stationary point of $J(\omega)$ (i.e., $\nabla J(\omega) \to 0$).

Convergence requires a structural condition on the critic. The critic's feature vectors must span the actor's score functions $\nabla_\omega \log \pi_\omega(a|s)$, so that the critic's approximation error lies orthogonal to the policy gradient direction. Under this compatibility condition, the critic's projection error does not bias the actor's gradient estimates.[76]

A2C (Advantage Actor-Critic) is the synchronous variant: collect a batch of transitions, compute TD errors, and update both networks. A3C (Mnih et al., 2016) parallelizes this across multiple workers updating a shared parameter server asynchronously.[77]

3.5.2 Entropy Regularization and Soft Actor-Critic

SAC (Soft Actor-Critic) (Haarnoja et al., 2018) extends the actor-critic framework with entropy regularization, building on the soft Q-learning algorithm of Haarnoja et al. (2017). The agent maximizes the entropy-augmented objective:

$$J_\tau(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty} \gamma^t \left(R_t + \tau \mathcal{H}(\pi_\theta(\cdot|s_t))\right)\right], \tag{45}$$

where $\mathcal{H}(\pi) = -\sum_a \pi(a) \log \pi(a)$ is the entropy and $\tau > 0$ is the temperature parameter. Geist et al. (2019) later provided the unifying theoretical framework, showing that entropy regularization converts the Bellman optimality operator's non-smooth hard max into a smooth log-sum-exp, and that the resulting soft Bellman operator remains a γ-contraction, preserving the convergence guarantees of standard dynamic programming.[78]

Entropy regularization also addresses the deadly triad directly. By maintaining policy stochasticity, the behavior policy used for data collection remains close to the target policy being optimized. This reduces the distribution mismatch between the stationary distribution under the behavior policy and the update targets, mitigating the off-policy instability leg of the triad. Cen et al. (2022) formalize a second benefit: entropy regularization accelerates convergence of the natural policy gradient from $O(1/\epsilon)$ to $O(\log(1/\epsilon))$, providing a precise sense in which smoothing the policy landscape aids optimization. The actor-critic structure separates identification from optimization: the critic solves a regression problem (estimate V^π from data), while the actor solves an optimization problem (improve π using the estimated values).

[75] The two-timescale structure is analogous to nested optimization in structural estimation, where an inner loop solves for equilibrium given parameters and an outer loop searches over parameters. The critic's inner loop (policy evaluation) must converge before the actor's outer loop (policy improvement) takes a step. Wu et al. (2020) provide finite-time convergence rates ($\tilde{O}(\epsilon^{-2.5})$ sample complexity) for two-timescale actor-critic with linear approximation, and Tian et al. (2023) establish analogous rates for single-timescale actor-critic with multi-layer neural networks.

[76] The compatible function approximation theorem first appears in Sutton et al. (2000) and is the key structural requirement in Konda and Tsitsiklis (2000). It constrains the critic architecture to be "compatible" with the actor parameterization, the same condition that makes the natural policy gradient equal to the critic's weight vector in Kakade (2002).

[77] Parallel workers decorrelate gradient estimates by exploring different parts of the state space simultaneously, removing the need for an experience replay buffer. Rigorous convergence theory for A3C's lock-free asynchronous parameter updates remains an open problem; existing analyses of asynchronous stochastic approximation (Qu and Wierman, 2020) address classical asynchrony (different state-action pairs updated at different times on a single trajectory), not the parallel-worker gradient setting.

[78] The optimal policy under entropy regularization is $\pi^*(a|s) \propto \exp(Q^*(s,a)/\tau)$, which is precisely the McFadden (1974) logit choice probability with systematic utility $Q^*(s,a)$ and scale parameter τ. The entropy-regularized value function is the log-sum-exp of Q-values, corresponding to the inclusive value (log-sum) operator in nested logit models. This equivalence between the soft-control framework and dynamic discrete choice models with EV1 taste shocks is developed formally in Rust and Rawat (2026), Appendix A.

3.5.3 Error Amplification Under Approximate Value Functions

Two questions remain important: how does approximation error propagate to policy quality, and how does computational complexity scale with problem size? Singh and Yee (1994) bound the policy degradation from value function errors (an independent derivation appears in Bertsekas and Tsitsiklis 1996, Proposition 6.1). If $\hat{V}$ approximates V^* with error $\|\hat{V} - V^*\|_\infty \leq \epsilon$, and $\hat{\pi}$ is the greedy policy with respect to $\hat{V}$, then:

$$\|V^* - V^{\hat{\pi}}\|_\infty \leq \frac{2\gamma}{1-\gamma}\epsilon. \tag{46}$$

At $\gamma = 0.99$, the amplification factor is $2 \cdot 0.99/0.01 = 198$. A 1% error in value function approximation yields at most 198% error in policy value.[79] This bound is pessimistic but finite: approximate value functions do not cause unbounded policy degradation.

3.5.4 Sample Complexity of Planning

Classical dynamic programming complexity scales with the state space size $|\mathcal{S}|$. For problems like Go, where $|\mathcal{S}| \approx 10^{170}$, exact computation is impossible. Kearns et al. (2002) prove that with access to a generative model[80] (a simulator that samples transitions from any state-action pair), near-optimal planning is possible with *no dependence on* $|\mathcal{S}|$. The cost is exponential dependence on the effective horizon $H = \log(R_{\max}/(\epsilon(1-\gamma)))/\log(1/\gamma)$: the sparse sampling algorithm requires $O((|\mathcal{A}|/\epsilon)^H)$ simulator calls.[81] For γ near 1, $H \approx (1-\gamma)^{-1}\log(R_{\max}/\epsilon)$, so the method is practical only for short effective horizons or moderate discount factors. Classical DP scales linearly in $|\mathcal{S}|$ but polynomially in H; sparse sampling eliminates state-space dependence at the cost of exponential horizon dependence. This explains why MCTS succeeds in large state spaces with bounded lookahead.

The minimax-optimal sample complexity for planning with a generative model, when queries to arbitrary state-action pairs are permitted, is $\Theta(|\mathcal{S}||\mathcal{A}|/((1-\gamma)^3\epsilon^2))$ (Azar et al., 2013). This bound scales linearly in $|\mathcal{S}|$ but polynomially in $1/(1-\gamma)$, the opposite regime from sparse sampling. Agarwal et al. (2020a) show that the plug-in model-based approach (learn $\hat{P}$ from samples, then plan with $\hat{P}$) achieves this minimax rate, establishing that model-based RL is statistically optimal. Li et al. (2024b) further tighten this result by breaking the $|\mathcal{S}||\mathcal{A}|/(1-\gamma)^2$ sample-size barrier, showing that the minimax rate is achievable with total sample size as low as $|\mathcal{S}||\mathcal{A}|/(1-\gamma)$.[82]

[79] The Singh-Yee bound is worst-case and not tight in general; massoud Farahmand et al. (2010) derive tighter bounds under smoothness assumptions on the MDP. The amplification factor $2\gamma/(1-\gamma)$ is a sensitivity analysis: it quantifies how errors in the "inputs" (value estimates) propagate to "outputs" (policy quality), analogous to errors-in-variables bias in regression. The discount factor γ controls sensitivity; more patient agents face larger amplification.

[80] A "generative model" in RL is a simulator that, given any state-action pair (s, a), returns a sampled next state $s' \sim P(\cdot|s, a)$ and reward r. This is unrelated to "generative models" in machine learning (GANs, diffusion models) or "generative processes" in Bayesian econometrics. The distinction matters: planning with a generative model is strictly easier than learning from a single trajectory, because the agent can query arbitrary states rather than following a sequential path.

[81] The sparse sampling algorithm builds a random tree of depth H from the current state, sampling C successor states per action at each node, then estimates values by averaging leaf rewards back up the tree. Its running time is $O((C|\mathcal{A}|)^H)$, which is exponential in H but entirely independent of $|\mathcal{S}|$. Kearns et al. (2002) also prove a lower bound of $\Omega(2^H)$ generative model calls for any planning algorithm, so exponential horizon dependence is unavoidable in the worst case.

[82] Recent extensions push these results beyond standard MDPs. Clavier et al. (2024) study the robust MDP setting where the agent must plan under model uncertainty with sa-rectangular or s-rectangular uncertainty sets, establishing minimax rates for robust policy optimization. Wang et al. (2025) extend the analysis to risk-sensitive objectives under the iterated CVaR criterion.

3.6 Fundamental Tradeoffs

The choice between methods involves distinct tradeoffs rooted in DP structure. Value-based methods target Q^* via the Bellman contraction; function approximation introduces the deadly triad. Policy-based methods optimize π_θ directly with global convergence guarantees for softmax policies (Agarwal et al., 2021), but high variance. Actor-critic methods combine both (Konda and Tsitsiklis, 2000). Four cross-cutting tradeoffs recur throughout. (i) Exploration versus exploitation, with the Lai and Robbins (1985) lower bound establishing that regret must grow at least logarithmically. (ii) Model-based versus model-free, where model-based methods gain sample efficiency at the cost of asymptotic bias under misspecification (Moerland et al., 2023). (iii) On-policy versus off-policy (Sutton and Barto, 2018, Ch. 5–7), trading stability for data reuse. (iv) Bias versus variance in advantage estimation, with GAE (Schulman et al., 2015) interpolating between Monte Carlo returns ($\lambda = 1$) and one-step TD ($\lambda = 0$).

3.7 Conclusion

Reinforcement learning algorithms are asymptotic approximations to classical dynamic programming operators, justified by the mathematics of contractions, stochastic approximation, and gradient domination. Value iteration becomes Q-learning (Watkins and Dayan, 1992; Tsitsiklis, 1994) when expectations are replaced by single samples. Policy iteration becomes the natural policy gradient (Kakade, 2001; Agarwal et al., 2021) when the greedy improvement step is approximated by gradient ascent, and NPG recovers PI exactly in the tabular case. The stochastic approximation framework, from the foundational work of Robbins (1952) through the ODE method of Borkar and Meyn (2000), guarantees that under appropriate step-size conditions, noisy iterates converge to the same fixed points as their deterministic counterparts. Reinforcement learning is not a departure from dynamic programming but an extension of it. Tabular RL and RL with linear function approximation rest on solid theoretical foundations. Deep RL lacks comparable guarantees: convergence remains an open problem, and empirical successes remain case-specific.

4 Structural Estimation with Reinforcement Learning

Several recent papers have used RL training loops,[83] namely Q-learning, temporal-difference learning, policy gradient, and actor-critic methods, to solve structural economic models at scales where conventional dynamic programming fails.[84] Throughout this chapter I adopt a unified notation. An MDP is a tuple $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s'|s, a)$ is the transition kernel, $r(s, a)$ is the per-period reward, and $\gamma \in [0, 1)$ is the discount factor.[85] A policy $\pi : \mathcal{S} \to \Delta(\mathcal{A})$ maps states to distributions over actions. The value function under π is $V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s\right]$, and the action-value function is $Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)}[V^\pi(s')]$. The optimal value function satisfies $V^*(s) = \max_{a \in \mathcal{A}} Q^*(s, a)$.

The canonical structural estimation framework for MDPs was formulated by Rust (1994), whose nested fixed-point (NFXP) algorithm embeds the Bellman equation inside a maximum likelihood estimator. The methods reviewed in this chapter replace the inner fixed-point computation with RL-based approximations.

[83] Throughout this chapter, the RL training loop runs entirely inside the econometrician's computational model; the agent never interacts with real economic agents or markets but serves as a numerical method for solving the Bellman equation within a structural estimation procedure (Section 1.1). There is no execution phase in the usual sense.

[84] I exclude papers that use neural network function approximation without an RL training mechanism. Inverse reinforcement learning is treated in the sister survey (Rust and Rawat, 2026).

[85] Several of the papers reviewed here use β for the discount factor, following economics convention. I translate all results to γ for consistency with the RL literature and the rest of this survey.

4.1 Single-Agent Structural Estimation

4.1.1 TD Learning for CCP Estimation

Adusumilli et al. (2022) adapt temporal-difference (TD) learning to estimate the recursive terms that arise in CCP-based estimation, entirely avoiding specification or estimation of transition densities.

The CCP approach requires computing two functions $h : \mathcal{A} \times \mathcal{S} \to \mathbb{R}^d$ and $g : \mathcal{A} \times \mathcal{S} \to \mathbb{R}$ that solve the recursive equations

$$h(a, s) = z(s, a) + \gamma\, \mathbb{E}[h(a', s') \mid a, s], \tag{47}$$

$$g(a, s) = e(a, s) + \gamma\, \mathbb{E}[g(a', s') \mid a, s], \tag{48}$$

where $e(a, s) = \gamma_{\mathrm{E}} - \ln P(a|s)$ under logit errors (γ_{E} denoting the Euler constant), and the expectation is over the next-period state-action pair (s', a') given the transition kernel $P(s'|s, a)$ and the observed policy $P(a|s)$. Both h and g satisfy a Bellman-like recursion under the observed (data-generating) policy, not the optimal policy, so standard TD learning applies directly. Adusumilli et al. (2022) propose two methods.

The first is the linear semi-gradient method.[86] Approximate $h(a, s) \approx \phi(a, s)^\top w$ where $\phi : \mathcal{A} \times \mathcal{S} \to \mathbb{R}^p$ is a vector of basis functions (e.g., polynomials in state variables) and $w \in \mathbb{R}^p$ are the weights to be estimated. The TD(0) fixed-point equation is

$$\mathbb{E}\left[\phi(a, s)\left(\phi(a, s) - \gamma\phi(a', s')\right)^\top\right] w = \mathbb{E}\left[\phi(a, s)\, z(s, a)\right]. \tag{49}$$

The sample analog replaces population expectations with averages over the observed panel.

$$\hat{w} = \left(\frac{1}{n(T-1)} \sum_{i=1}^{n} \sum_{t=1}^{T-1} \phi_{it}\left(\phi_{it} - \gamma\phi_{i,t+1}\right)^\top\right)^{-1} \left(\frac{1}{n(T-1)} \sum_{i=1}^{n} \sum_{t=1}^{T-1} \phi_{it}\, z_{it}\right), \tag{50}$$

where $\phi_{it} = \phi(a_{it}, s_{it})$ and $z_{it} = z(s_{it}, a_{it})$. This requires inverting a $p \times p$ matrix, where p is the number of basis functions, making computation trivial in most settings. No transition density estimation is needed, since the method uses only observed sequences of current and next-period state-action pairs.

The second method is approximate value iteration (AVI), which iterates the Bellman-like operator using nonparametric regression. At iteration k, one constructs pseudo-outcomes

$$Y_{it}^{(k)} = z_{it} + \gamma\, \hat{h}^{(k-1)}(a_{i,t+1}, s_{i,t+1}) \tag{51}$$

and then fits $\hat{h}^{(k)}$ by regressing $Y_{it}^{(k)}$ on (a_{it}, s_{it}) using any machine learning method, including LASSO, random forests, or neural networks.[87] This is the first DDC estimator compatible with arbitrary ML prediction methods, enabling application to very high-dimensional state spaces.

With $\hat{h}$ and $\hat{g}$ in hand, structural parameters are recovered by pseudo-maximum likelihood estimation (PMLE). For continuous state spaces, Adusumilli et al. (2022) derive a locally robust correction to the PMLE criterion that accounts for the nonparametric first-stage estimation of value terms, restoring $\sqrt{n}$-convergence of $\hat{\theta}$.[88] The PMLE score is $m(a, s; \theta, h, g) =$

[86] The method is called "semi-gradient" because it computes the gradient of only the prediction $\phi(a, s)^\top w$ with respect to w, not the full TD error including the bootstrap target $\phi(a', s')^\top w$. This avoids differentiating through the target but sacrifices guaranteed convergence in some settings.

[87] *LASSO* (Tibshirani, 1996) adds an ℓ_1 penalty to a regression loss, shrinking many coefficients to zero for sparse solutions; *random forests* average many decision trees fit to random subsamples and feature subsets for nonparametric estimation; *neural networks* compose affine transformations with elementwise nonlinearities across multiple layers.

[88] An estimator has $\sqrt{n}$-convergence if its error shrinks at rate $n^{-1/2}$ where n is the sample size. This is the

$\partial_\theta \ln \pi(a, s; \theta, h, g)$, where π is the logit choice probability with continuation value $V(a, s) = h(a, s)^\top \theta + g(a, s)$. The naive estimator solves $\mathbb{E}_n[m(a, s; \theta, \hat{h}, \hat{g})] = 0$, but with continuous states this moment condition is not orthogonal to the first-stage estimates and converges slower than $\sqrt{n}$. The locally robust moment adds a debiasing correction:

$$\zeta = m(a, s; \theta, h, g) - \lambda(a, s; \theta) \left\{ z(s, a)^\top \theta + \gamma\, e(a', s') + \gamma\, V(a', s') - V(a, s) \right\}, \tag{52}$$

where $\lambda(a, s; \theta)$ solves a backward recursion that propagates the influence of estimation error through the dynamic structure.[89] The corrected estimator $\hat{\theta}_{LR}$ solves $\mathbb{E}_n[\zeta_n] = 0$ and is computationally no harder than the naive PMLE, since the correction is constant in θ.

Theorem 2 (Adusumilli et al. (2022), Theorem 1)**.** *Under regularity conditions, the linear semi-gradient estimator $\hat{h}$ satisfies $\|\hat{h} - h\|_2 = O_P(n^{-1/2}(T-1)^{-1/2})$, where $\|\cdot\|_2$ denotes the $L^2(P)$ norm.*[90]

Theorem 3 (Adusumilli et al. (2022), Theorem 5)**.** *Under regularity conditions on the ML method used in AVI, the locally robust PMLE estimator $\hat{\theta}_{LR}$ satisfies $\sqrt{n}(\hat{\theta}_{LR} - \theta^*) \xrightarrow{d} \mathcal{N}(0, \Sigma)$ for an explicit variance Σ, even when the state space is continuous.*

Monte Carlo experiments on a dynamic firm entry model with seven structural parameters and five continuous state variables show that the TD-based estimators achieve a 4- to 11-fold reduction in mean squared error compared to CCP estimators using state-space discretization.

For dynamic discrete games, the method extends naturally. Standard CCP-based estimation of games requires integrating out other players' actions, which becomes intractable with many players or continuous states. TD learning avoids this entirely, since it works directly with the joint empirical distribution of states and their successors. The "integrating out" is done implicitly within sample expectations.

4.1.2 Policy Gradient for DDC Estimation

Hu and Yang (2025) combine policy gradient methods with the Simulated Method of Moments (SMM) to estimate DDCs, with particular focus on models with unobserved state variables.

The outer loop is SMM.[91] Define a vector of data moments $\mathbf{M}_d$ computed from the observed panel. For candidate structural parameters θ and transition parameters ξ, simulate the model to produce simulated moments $\mathbf{M}_s(\theta, \xi)$. The estimator minimizes

$$(\hat{\theta}, \hat{\xi}) = \arg\min_{\theta, \xi} \left(\mathbf{M}_d - \mathbf{M}_s(\theta, \xi)\right)^\top \mathbf{W} \left(\mathbf{M}_d - \mathbf{M}_s(\theta, \xi)\right), \tag{53}$$

where $\mathbf{W}$ is a positive definite weight matrix. Computing $\mathbf{M}_s(\theta, \xi)$ requires solving for the optimal policy under (θ, ξ), which is the inner-loop problem.

The inner loop parametrizes the choice probability directly as a logistic function of state variables. For a binary choice $J_t \in \{0, 1\}$, the general form is $\Pr(J_t = 1 \mid \boldsymbol{X}_t; \boldsymbol{\gamma}(\theta)) = \text{logistic}(\boldsymbol{X}_t \boldsymbol{\gamma}(\theta))$, where $\boldsymbol{\gamma}(\theta)$ are policy parameters that depend on the structural parameters.[92]

standard parametric rate; slower rates (e.g., $n^{-1/4}$) indicate efficiency loss from nonparametric first stages.

[89] The term in braces is the temporal-difference error of the continuation value V. The adjoint λ weights this TD error by its marginal impact on the PMLE score. See Online Appendix B.3 of Adusumilli et al. (2022) for the derivation.

[90] The $L^2(P)$ norm is $\|f\|_2 = (\int f(x)^2\, dP(x))^{1/2}$, measuring average squared deviation under the probability measure P. This is the natural norm for mean-squared-error analysis.

[91] SMM (Gourieroux et al., 1993) estimates structural parameters by matching moments from simulated data to moments from observed data, avoiding direct evaluation of the likelihood function.

[92] The linear index can be replaced by higher-order terms of $\boldsymbol{X}_t$ or deep neural networks; the method requires only that the gradient $\nabla_{\boldsymbol{\gamma}} \log \pi_{\boldsymbol{\gamma}}$ has a closed form.

In their application to a Rust bus engine model with unobserved bus condition S_t^*, this takes the form

$$\Pr(J_t = 1 \mid X_t, S_t^*, t; \boldsymbol{\gamma}) = \frac{\exp(\gamma_0 + \gamma_1 t + \gamma_2 X_t + \gamma_3 S_t^*)}{1 + \exp(\gamma_0 + \gamma_1 t + \gamma_2 X_t + \gamma_3 S_t^*)}, \tag{54}$$

where t enters the index directly to capture time-varying replacement incentives. The policy parameters are updated by REINFORCE-style gradient ascent, applying the policy gradient theorem (Sutton et al., 1999):

$$\nabla_\gamma V(\boldsymbol{\gamma}) = \mathbb{E}\left[\sum_{t=0}^{T} \nabla_\gamma \log \pi_\gamma(J_t \mid X_t, S_t^*, t)\, Q^{\pi_\gamma}(X_t, S_t^*, J_t)\right], \tag{55}$$

where Q^{π_γ} is the action-value function under the current policy, estimated by Monte Carlo returns from forward-simulated trajectories.

The main contribution is handling unobserved state variables. When state variables are only partially observed, the policy in (54) is parametrized as a function of both X_t and S_t^*, and the algorithm forward-simulates trajectories of both observed and unobserved variables.

Building on the nonparametric identification results of Hu and Shum (2012), the outer-loop SMM targets moments from five consecutive periods of observed data, which suffice to separately identify the structural parameters θ and transition parameters ξ without requiring the econometrician to observe S_t^*. No discretization of continuous unobserved states is needed; the same algorithm handles both discrete and continuous unobserved heterogeneity.

For each candidate (θ, ξ) in the outer minimization (53), the inner loop runs policy gradient until convergence, producing optimal policy parameters $\boldsymbol{\gamma}^*(\theta, \xi)$. These are used to simulate data and compute $\mathbf{M}_s(\theta, \xi)$.

On an extended Rust bus engine model with a continuous unobserved bus condition following an AR(1) process, estimates of seven structural parameters are centered around their true values across 400 simulations. On a discrete-unobservable variant, the method matches the precision of Arcidiacono and Miller (2011)'s two-step EM algorithm at comparable computation times, though the advantage diminishes as more inner-loop iterations are used for precision.

4.2 Dynamic Oligopoly and Strategic Interaction

Dynamic oligopoly models combine game theory and dynamic programming: firms choose actions strategically while anticipating competitors' strategies, and the state space grows combinatorially in the number of firms.

4.2.1 Q-Learning in Dynamic Procurement Auctions

Asker et al. (2020) develop a computational framework for analyzing dynamic procurement auctions with serially correlated asymmetric information. Their approach builds on the Experience-Based Equilibrium (EBE) concept of Fershtman and Pakes (2012), which computes equilibria by simulating industry trajectories and updating strategies toward best responses. EBE evaluates values only on recurrently visited states rather than the full state space, making it feasible for large state spaces. While Fershtman and Pakes (2012) use a stochastic approximation algorithm to update continuation values from simulated industry trajectories, Asker et al. (2020) add explicit value-function updates via stochastic approximation.

The model is a repeated first-price sealed-bid auction with two firms. Each firm i maintains a private inventory state $\omega_{i,t}$ (stock of unharvested timber, in their application). The state evolves endogenously, as winning an auction increases inventory while harvesting depletes it. Each firm's private state is not observed by its competitor except at periodic revelation events.

The key computational innovation is the use of Q-learning to compute equilibrium strategies. Each firm i maintains a Q-function $Q_i : \mathcal{S}_i \times \mathcal{A}_i \to \mathbb{R}$, where $\mathcal{S}_i$ encodes firm i's information set (its own inventory, beliefs about the competitor's inventory, public history) and $\mathcal{A}_i$ is its action set (participation decision and bid level). The Q-function satisfies

$$Q_i(s,a) = \mathbb{E}\left[r_i(s,a,a_{-i}) + \gamma \max_{a'\in\mathcal{A}_i} Q_i(s',a') \;\middle|\; s,a\right], \tag{56}$$

where $r_i(s,a,a_{-i})$ is firm i's per-period profit given the state, its own action a, and the competitor's action a_{-i}, and the expectation is taken over the competitor's strategy and the stochastic transitions.

Firms update Q-values using sample averaging.

$$Q_i(s,a) \leftarrow Q_i(s,a) + \frac{1}{h_k(s,a)}\left[r_i + \gamma \max_{a'\in\mathcal{A}_i} Q_i(s',a') - Q_i(s,a)\right], \tag{57}$$

where $h_k(s,a)$ is the number of times state-action pair (s,a) has been visited.[93] The equilibrium computation proceeds iteratively, with firms simultaneously updating their Q-functions based on simulated play against each other's current strategies. Strategies are derived from Q-values using an ε-greedy rule or Boltzmann exploration.[94]

Asker et al. (2020) add a boundary consistency condition to the EBE concept that restricts behavior at the boundary of the recurrent state class, reducing the multiplicity of equilibria. Their numerical analysis reveals that information sharing between firms can, through increased precision of beliefs about competitor states, induce firms to spend more time in states where competition is less intense. The dynamic RL-computed equilibrium yields qualitatively different predictions from both static analysis and myopic ($\gamma = 0$) benchmarks. With dynamics, information sharing decreases average bids and increases average profits, while the myopic benchmark shows negligible effects.

The limitation of this approach is the tabular representation; the Q-function is stored as a lookup table over discretized states and actions, restricting applicability to models with moderate state-space dimension.

4.2.2 TD Learning for Merger Analysis with Innovation

Hollenbeck (2019) uses RL to solve a dynamic oligopoly model with endogenous mergers, entry, exit, and quality investment. The model extends the Ericson-Pakes framework to study how horizontal mergers affect innovation incentives.

The industry state is $\Omega = (\omega_1, \ldots, \omega_n)$ where $\omega_i \in \{1, \ldots, \omega_{\max}\}$ is firm i's product quality. Firms produce differentiated goods and compete in prices (Bertrand competition with logit demand). In each period, firms simultaneously choose investment levels, entry/exit decisions, and potentially initiate merger negotiations.

Each firm i computes its continuation value $V_i(\Omega)$ from the industry state. Because the state space is the product of all firms' quality levels plus industry structure (number of active firms, recent mergers), exact dynamic programming is infeasible for industries with more than two or three firms. Hollenbeck (2019) instead uses temporal-difference learning to estimate values from simulated industry trajectories.

[93] This sample-averaging rule ($\alpha_k = 1/h_k$) is equivalent to maintaining the running mean of observed returns, as distinct from fixed-α Q-learning which gives exponentially decaying weight to older observations. The update is applied for all actions, including counterfactual actions not taken, using the observed state transition.

[94] ε-greedy selects the greedy action $argmax_a\, Q(s,a)$ with probability $1-\varepsilon$ and a uniformly random action otherwise. Boltzmann exploration selects action a with probability $\propto \exp(Q(s,a)/\tau)$ where $\tau > 0$ is a temperature parameter.

The value function update for firm i is[95]

$$V_i(\Omega) \leftarrow V_i(\Omega) + \alpha \left[\Pi_i(\Omega, \mathbf{a}) + \gamma V_i(\Omega') - V_i(\Omega)\right], \tag{58}$$

where $\Pi_i(\Omega, \mathbf{a})$ denotes firm i's per-period profit given industry state Ω and the joint action profile $\mathbf{a}$ (investment, entry/exit, merger decisions),[96] γ is the discount factor, and Ω' is the realized next-period state. The algorithm uses ε-decreasing exploration to prevent convergence to locally suboptimal equilibria.

The equilibrium computation follows the Pakes-McGuire iterative scheme.[97] The algorithm simulates long industry histories, updates each firm's value function via (58), re-derives best-response strategies from updated values, and repeats.

The central finding is that horizontal mergers, while reducing static consumer surplus in the short run, create a strong incentive for entry and investment. Firms enter with negative static profits because the prospect of a lucrative buyout justifies the initial investment. The result is substantially higher long-run innovation and consumer welfare with mergers than without. This finding reverses the standard static antitrust prediction and can only emerge in a dynamic model where firms are forward-looking.

Two related papers merit brief mention. Lomys and Magnolfi (2024) develop structural estimation methods for strategic settings where agents use learning algorithms (specifically regret-minimizing rules) rather than playing a fixed equilibrium. They impose an "asymptotic no-regret" condition as a minimal rationality requirement and derive identification results for payoff parameters. Covarrubias (2022) uses deep RL to study oligopolistic pricing in a New Keynesian framework, representing firms' pricing policies as neural networks $\pi_\phi(a|s)$. The method uncovers multiple equilibria ranging from competitive to collusive pricing.[98]

4.3 Auction Equilibria and Mechanism Design

Closed-form equilibrium bidding strategies exist only for narrow families of valuation distributions and auction formats, and numerical methods scale poorly with the number of bidders and items.

4.3.1 RL for Sequential Price Mechanisms

Brero et al. (2021) use RL to design optimal sequential price mechanisms (SPMs), a class of indirect auction mechanisms where agents are approached in sequence and offered menus of items at posted prices. SPMs generalize both serial dictatorship and posted-price mechanisms and essentially characterize all strongly obviously strategyproof (SOSP) mechanisms (Pycia and Troyan, 2023).[99]

The mechanism design problem is formulated as a partially observable Markov decision

[95] This stochastic approximation update, introduced by Pakes and McGuire (1994) for dynamic oligopoly computation, is closely related to temporal-difference learning in the RL literature. The original algorithm uses visit-count averaging ($\alpha_k = 1/k$ where k counts visits to each state) rather than a fixed learning rate.

[96] I use Π_i for firm i's per-period profit to avoid confusion with policy π.

[97] The Pakes-McGuire algorithm (Pakes and McGuire, 1994) computes Markov perfect equilibria by iterating: (1) given current value functions, compute best-response strategies; (2) given strategies, update value functions via simulation. Convergence is not guaranteed but works well in practice.

[98] The most prominent example of algorithmic collusion is Calvano et al. (2020), who showed that independent Q-learning agents in a repeated Bertrand pricing game learn to sustain supra-competitive prices and punish deviators without explicit communication. This result and its implications for competition policy are treated in the companion thesis chapter (Rawat, 2026).

[99] A mechanism is strategyproof if truthful reporting is a dominant strategy. It is obviously strategyproof if this dominance is apparent even to boundedly rational agents; strongly obviously strategyproof (SOSP) adds that dominance holds at every information set (Pycia and Troyan, 2023).

process (POMDP).[100] The state at round t includes the set of remaining items $\rho_t^{\text{items}} \subseteq [m]$, remaining agents $\rho_t^{\text{agents}} \subseteq [n]$, the partial allocation $\mathbf{x}_t$, and the agents' (unobserved) valuation functions. The action $a_t = (i_t, \{p_j^t\}_{j \in \rho_{t-1}^{\text{items}}})$ specifies which agent to visit next and what prices to offer. The observation is the agent's purchase decision, from which the mechanism can update beliefs about valuations. The reward is the objective function evaluated at the final allocation:

$$r = g(\mathbf{x}_T, \boldsymbol{\tau}_T; \mathbf{v}), \tag{59}$$

where g can be social welfare $\sum_i v_i(x_i)$, revenue $\sum_i \tau_i$, or max-min fairness $\min_i v_i(x_i)$.

A key theoretical result establishes when adaptive mechanisms outperform static ones.

Theorem 4 (Brero et al. (2021), Propositions 1–4, informal)**.** *Each feature of adaptive mechanisms is necessary for welfare optimality, even in simple settings. Personalized prices are needed with one item and two i.i.d. agents (Proposition 1); adaptive prices are needed with two identical items and three i.i.d. agents (Proposition 2); adaptive ordering is needed with six agents whose valuations are correlated (Proposition 3); both adaptive prices and ordering are needed with four agents whose valuations are independently but non-identically distributed (Proposition 4).*

The policy maps from a sufficient statistic of the observation history to actions. Brero et al. (2021) show that this statistic can be represented compactly. The set of remaining items and agents suffices for independent valuations, while the full allocation matrix is needed for correlated valuations. They train the mechanism policy using Proximal Policy Optimization (PPO),[101] which handles the discrete action space (agent selection) and continuous action space (price setting) of the POMDP.

Experimental results show that the learned SPMs achieve near-optimal welfare across settings with up to 20 agents and 5 items (with similar results noted for up to 30 of each), significantly outperforming static pricing benchmarks. The improvement is largest when agent valuations are correlated, since adaptive prices allow the mechanism to infer information about remaining agents from earlier purchases.

The limitation is that the POMDP formulation requires knowledge of the prior distribution over valuations, which in practice must be estimated from data. The method also does not scale easily to very large numbers of items due to the combinatorial action space.

4.3.2 Fitted Policy Iteration for Combinatorial Auctions

Ravindranath et al. (2024) address revenue-maximizing mechanism design for combinatorial auctions with multiple items and strategic bidders. Their innovation is integrating differentiable auction structure into a fitted policy iteration framework, enabling analytical gradient computation where standard RL methods struggle with high variance.

The mechanism visits agents one at a time in sequence. Each agent i, upon being visited, selects a bundle of items from those still available, given posted prices. Valuations are drawn once from distributions V_i and remain fixed throughout the mechanism. Complementarities arise from the structure of the valuation function over bundles, not from dynamic evolution. The MDP state at step t is $s_t = (i_t, S_t)$, where i_t is the current bidder and S_t is the set of remaining items.

The mechanism's policy maps from state to a price vector over available items. The key technical contribution is making the auction clearing differentiable. In a standard auction, the allocation is an argmax over bids (non-differentiable), and payments depend discontinuously on

[100] In a POMDP, the agent cannot directly observe the full state. It maintains a belief distribution over possible states, updated via Bayes' rule as new observations arrive. This captures the mechanism designer's uncertainty about bidder valuations.

[101] Proximal Policy Optimization (Schulman et al., 2017) is a policy gradient algorithm that constrains each update to a trust region, preventing large destabilizing policy changes. It is described in Chapter 2.

the allocation. Ravindranath et al. (2024) replace the hard bundle selection with a softmax relaxation,[102] enabling analytical gradient computation through the mechanism. The actor loss is the negative expected revenue, and gradients flow directly through the softmax-relaxed allocation and payment rules. The paper explicitly avoids REINFORCE-style estimators, noting that analytical gradients overcome the sample inefficiency and high variance of score-function methods.

The method follows fitted policy iteration (Bertsekas and Tsitsiklis, 1996), alternating between evaluating the current policy (computing expected revenue) and improving it via gradient ascent on the actor loss. This differs from model-free RL approaches (PPO, SAC) that the paper uses as baselines.

Experiments on settings with additive and combinatorial valuations show that the learned mechanisms achieve up to 13% higher revenue than item-wise Myerson optimal auctions, with the largest gains in combinatorial settings where bundle complementarities make item-wise pricing suboptimal. Standard RL baselines (PPO, SAC) are also outperformed, confirming the advantage of exploiting differentiable structure.[103]

4.4 Macroeconomic Models and Optimal Policy Design

Reinforcement learning and deep learning methods extend to high-dimensional macroeconomic models where grid-based dynamic programming is infeasible. Maliar et al. (2021) approximate policy and value functions in models with dozens of state variables. Fernández-Villaverde et al. (2024) provide a systematic treatment covering both single-agent and equilibrium settings. Atashbar and Shi (2023) apply model-free RL (DDPG) to a real business cycle model, recovering near-optimal policies without deriving Euler equations. Moll (2025) argues that rational expectations equilibria in heterogeneous agent models are computationally intractable and proposes reinforcement learning as a tractable alternative; see also Zhao (2025). In a related direction, Zheng et al. (2022) use multi-agent RL for automated tax policy design, training worker and government agents simultaneously in a Stackelberg game where the learned tax policies improve upon several analytical baselines on both equality and productivity dimensions.

4.5 Simulation Study: DDC Estimation at Scale

The test bed is a multi-component extension of the Rust (1987) bus engine replacement model. In the original formulation a maintenance superintendent observes discretized mileage $m \in \{0, \ldots, M-1\}$ and makes a binary keep-or-replace decision, facing running cost $c(s; \theta) = \theta_1 x + \theta_2 x^2$ with $x = m/M$, replacement cost RC, and Type I extreme value additive errors that yield logit conditional choice probabilities. We extend the model to K independent wear components, each evolving in $\{0, \ldots, M-1\}$, with aggregate normalized wear $x(s) = \sum_k m_k/M$ entering the same cost function. The state space is $|\mathcal{S}| = M^K$, so increasing K produces a controlled scaling experiment in which the data-generating process is identical across scales but the computational burden grows exponentially.

We compare four estimation methods on a multi-component bus engine replacement problem (Rust, 1987) with $K \in \{1, 2, 3, 4\}$ independent wear components, producing state spaces from 20 to 160,000 states. Panel data consist of $N = 500$ agents observed for $T = 100$ periods. The four methods are NFXP (nested fixed-point MLE), CCP (Hotz-Miller inversion), TD-CCP Linear (semi-gradient TD with polynomial basis), and TD-CCP Neural (approximate value iteration with a two-layer MLP), where the two TD-CCP variants follow Adusumilli et al. (2022). Each configuration is replicated across 5 seeds; Table 6 reports means.

[102]The softmax relaxation replaces the hard allocation $argmax_i\, b_i$ with a soft allocation $\exp(b_i/\tau)/\sum_j \exp(b_j/\tau)$, which is differentiable and approaches the hard allocation as $\tau \to 0$.

[103]DDPG was also tested but found to be unstable. DQN is not applicable to continuous price-setting.

Table 6: DDC estimation results across state-space scales. Rows show method, number of components K, state-space size $|\mathcal{S}|$, mean wall-clock time (seconds), RC bias, and root mean squared error for each structural parameter. Averages over 5 seeds; dashes indicate method failure (sparse state coverage).

Method	K	$\|S\|$	Time (s)	RC Bias	RC RMSE	θ_1 RMSE	θ_2 RMSE
NFXP	1	20	0.2	+0.074	0.102	0.363	0.618
CCP	1	20	0.1	+0.076	0.103	0.374	0.639
TD-CCP Linear	1	20	0.1	-0.083	0.096	0.178	0.222
TD-CCP Neural	1	20	33.7	+0.118	0.133	0.487	0.819
NFXP	2	400	0.4	-0.023	0.077	0.218	0.290
CCP	2	400	0.1	+0.042	0.085	0.461	0.775
TD-CCP Linear	2	400	0.1	-0.211	0.217	0.131	0.844
TD-CCP Neural	2	400	39.3	-0.040	0.070	0.099	0.168
NFXP	3	8,000	4.2	+0.004	0.043	0.290	0.454
CCP	3	8,000	—	—	—	—	—
TD-CCP Linear	3	8,000	0.1	-0.251	0.253	0.163	1.171
TD-CCP Neural	3	8,000	39.2	-0.009	0.046	0.266	0.417
NFXP	4	160,000	172.6	-0.036	0.070	0.130	0.113
CCP	4	160,000	—	—	—	—	—
TD-CCP Linear	4	160,000	0.2	-0.333	0.337	0.217	1.119
TD-CCP Neural	4	160,000	44.1	-0.029	0.069	0.167	0.144

NFXP scales from 0.2s at K=1 ($|\mathcal{S}|$=20) to 179s at K=4 ($|\mathcal{S}|$=160,000), reflecting the cost of repeated value iteration inside each likelihood evaluation. CCP becomes infeasible beyond K=2 due to sparse state coverage in the panel data. TD-CCP Neural maintains near-constant runtime (∼28–44s) across all K levels with competitive accuracy (RC RMSE 0.077 at K=4), consistent with the AVI approach of Adusumilli et al. (2022). TD-CCP Linear runs in under 0.3s at all scales but suffers from basis misspecification at higher K (RC RMSE 0.337 at K=4); see Table 6.

5 Reinforcement Learning in Games

With multiple agents adapting simultaneously, each agent's environment includes the others' changing policies, so the stationary-transition assumption behind single-agent convergence results fails. Shoham et al. (2007) enumerate five desiderata for multi-agent learning: (1) convergence to a stationary strategy in self-play; (2) rationality (best-responding against stationary opponents); (3) equilibrium attainment; (4) safety (guaranteeing at least the Nash-value payoff); (5) social welfare. No existing algorithm satisfies all five in general games.

Two paradigms emerged. Value-based methods generalize the Bellman operator to games, replacing the max with game-theoretic solution concepts (minimax, Nash), targeting stochastic games with simultaneous moves and observable payoffs. Regret-based methods (CFR) accumulate counterfactual regrets and let the time-averaged strategy converge, targeting extensive-form games with sequential moves and private information. Computing Nash equilibria is PPAD-complete (Daskalakis et al., 2009), so neither approach escapes the fundamental hardness, but both achieve convergence in the game classes they target.

5.1 Stochastic Games and Equilibrium Learning

5.1.1 The Stochastic Game Framework

An n-player stochastic game $\Gamma = (n, \mathcal{S}, \mathcal{A}_1, \ldots, \mathcal{A}_n, P, r_1, \ldots, r_n, \gamma)$ consists of a finite state space $\mathcal{S}$; finite action sets $\mathcal{A}_i$ for each player i; a transition function $P : \mathcal{S} \times \mathcal{A}_1 \times \cdots \times \mathcal{A}_n \to \Delta(\mathcal{S})$; reward functions $r_i : \mathcal{S} \times \mathcal{A}_1 \times \cdots \times \mathcal{A}_n \to \mathbb{R}$; and a common discount factor $\gamma \in [0, 1)$. At each stage, all players simultaneously choose actions, receive individual rewards, and the game transitions to a new state.[104]

A Markov decision process is a stochastic game with $n = 1$; a matrix game is a stochastic game with $|\mathcal{S}| = 1$. Each player i seeks a policy $\pi_i : \mathcal{S} \to \Delta(\mathcal{A}_i)$ maximizing discounted return $\mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_i(s_t, a_{1,t}, \ldots, a_{n,t})]$.

Standard Q-learning convergence (Watkins and Dayan, 1992) requires stationary transition and reward dynamics; with multiple learners, this assumption fails. Bowling and Veloso (2002) formalized two properties a learning algorithm should satisfy. It is *rational* if, when all other players converge to stationary policies, it converges to a best response; it is *convergent* if, in self-play, it converges to a stationary policy.

If all players use rational, convergent algorithms, the resulting profile is a Nash equilibrium by construction. The challenge is achieving both properties simultaneously.

5.1.2 Minimax-Q Learning

Littman (1994) proposed the first Q-learning algorithm for stochastic games, targeting two-player zero-sum games. The key modification replaces the max operator in the standard Q-learning backup with a minimax operator. Each agent maintains $Q_i(s, a_i, a_{-i})$ over the joint action space. The update rule is

$$Q_i(s, a_i, a_{-i}) \leftarrow (1 - \alpha)\, Q_i(s, a_i, a_{-i}) + \alpha \left[r_i + \gamma\, V_i(s') \right], \tag{60}$$

where the value backup solves a linear program:

$$V_i(s) = \max_{\pi_i \in \Delta(\mathcal{A}_i)} \min_{a_{-i} \in \mathcal{A}_{-i}} \sum_{a_i \in \mathcal{A}_i} \pi_i(a_i)\, Q_i(s, a_i, a_{-i}). \tag{61}$$

This is the RL analogue of Shapley's value iteration for zero-sum stochastic games (Shapley, 1964). The resulting policy $\pi_i(s)$ is generally a mixed strategy, since deterministic policies are exploitable in adversarial settings.

Minimax-Q converges to the minimax Q-values under Robbins-Monro learning rates ($\sum_t \alpha_t = \infty$, $\sum_t \alpha_t^2 < \infty$) and infinite exploration of all state-action tuples.[105] However, the algorithm sacrifices rationality: it plays the equilibrium strategy even against exploitable opponents.[106]

5.1.3 Nash-Q Learning

Hu and Wellman (2003) extended the framework to general-sum stochastic games, where players may have aligned, opposed, or mixed incentives. Each agent i maintains a Q-function over the

[104] Shapley (1953) introduced stochastic games in 1953 for the two-player zero-sum case, proving existence of the value via a contraction argument on the Bellman operator. The general-sum extension to n players is due to Fink (1964) and Takahashi (1964).

[105] The convergence proof extends the contraction argument for standard Q-learning. Because the zero-sum minimax operator is a contraction with modulus γ under the ℓ^∞ norm, the stochastic approximation converges to the fixed point. See Littman (1994) and the general treatment in Szepesvári and Littman (1999).

[106] In the soccer game of Littman (1994), minimax-Q won 53.7% against a hand-built opponent versus 26.1% for Q-learning. Against an adversarial challenger, Q-learning won 0% (its deterministic policy was fully predictable); minimax-Q won 37.5% through mixed strategies.

joint action space $Q_i(s, a_1, \ldots, a_n)$ and updates via

$$Q_i(s, \mathbf{a}) \leftarrow (1-\alpha)\, Q_i(s, \mathbf{a}) + \alpha \left[r_i + \gamma \, \mathrm{Nash}_i(Q_1(s'), \ldots, Q_n(s'))\right], \tag{62}$$

where $\mathrm{Nash}_i(\cdot)$ denotes player i's payoff under a Nash equilibrium of the stage game defined by the current Q-values $(Q_1(s'), \ldots, Q_n(s'))$. At each backup, the algorithm treats the Q-values as payoff matrices, computes a Nash equilibrium of this matrix game, and uses the equilibrium payoffs for the value estimate.

Theorem 5 (Hu and Wellman (2003))**.** *Nash-Q converges to Nash Q-values under Robbins-Monro learning rates and infinite exploration, provided every stage game encountered during learning has either (a) a global optimal point (all agents receive their highest payoff at the same joint action) or (b) a unique saddle-point Nash equilibrium.*

These conditions are restrictive. When stage games have multiple Nash equilibria, agents may select different equilibria for their backups, causing divergence.[107] Nash-Q reduces to standard Q-learning in the single-agent case.

5.1.4 The Convergence Problem

Table 7 summarizes the trade-offs. No single algorithm achieves both rationality and convergence in general games.

Table 7: Multi-agent Q-learning algorithms: convergence and information requirements

Algorithm	Rational	Convergent	Game class	Information
Indep. Q-learning	Yes	No	Any	Own reward
Minimax-Q	No	Yes	Zero-sum	Joint actions
Nash-Q	Cond.	Cond.	General-sum	All rewards
WoLF-PHC	Yes	Yes (2×2)	General-sum	Own reward

WoLF-PHC (Win or Learn Fast, Policy Hill-Climbing) of Bowling and Veloso (2002) maintains a policy $\pi_i(a|s)$ and a running average policy $\bar{\pi}_i(a|s)$, updating Q-values as in standard Q-learning. The policy moves toward the greedy action at a variable rate:

$$\delta = \begin{cases} \delta_l & \text{if } \sum_a \pi_i(a|s)\, Q_i(s,a) < \sum_a \bar{\pi}_i(a|s)\, Q_i(s,a) \quad \text{(losing)} \\ \delta_w & \text{otherwise} \quad \text{(winning)} \end{cases} \tag{63}$$

with $\delta_l > \delta_w$. When the current policy underperforms the historical average (losing), the agent adapts quickly; when outperforming (winning), it adapts slowly to avoid destabilizing the opponent. Bowling and Veloso (2002) proved that WoLF-IGA (the infinitesimal gradient ascent variant) converges to Nash equilibrium in all two-player, two-action games. The trajectory traces piecewise ellipses around the equilibrium, shrinking by a factor of $\ell^4 < 1$ per orbit, where $\ell = \sqrt{\delta_w/\delta_l}$. WoLF-PHC requires only own-reward observations, the same information as independent Q-learning.

Two further algorithms deserve mention. Friend-or-Foe Q-learning (Littman, 2001) decomposes agents as cooperative (friend) or adversarial (foe), using max for friends and minimax

[107] In the experiments of Hu and Wellman (2003), a grid game with a unique equilibrium Q-function converged in 100% of trials under Nash-Q versus 20% under independent Q-learning; a game with three equilibrium Q-functions converged in only 68–90% of trials. Nash-Q requires each agent to observe all other agents' rewards and to maintain Q-values over the joint action space $\mathcal{A}_1 \times \cdots \times \mathcal{A}_n$, with storage $O(n|\mathcal{S}| \prod_i |\mathcal{A}_i|)$, exponential in the number of players.

for foes; it always converges but requires knowing the relationship type a priori.[108] The evolutionary perspective of Börgers and Sarin (1997) provides a deeper lens: reinforcement learning dynamics in matrix games converge to the replicator equation from evolutionary game theory. The cycling of Q-learning in games such as matching pennies is structurally identical to the cycling of replicator dynamics in Rock-Paper-Scissors games.

5.1.5 Simulation Study: Cournot and Bertrand Duopoly

Two canonical games from industrial organization serve as benchmarks. In Cournot duopoly, two firms choose quantities $q_i \in \{0, 1, \ldots, 9\}$ with inverse demand $P(Q) = 10 - Q$ and marginal cost $c = 1$; the unique Nash equilibrium is $q^* = 3$ with profit $\pi^* = 9$. In Bertrand duopoly with differentiated products, two firms choose prices $p_i \in \{0, 1, \ldots, 9\}$ with demand $d_i = 10 - 2p_i + p_j$ and marginal cost $c = 1$; the continuous Nash equilibrium is $p^* \approx 4.33$, which discretizes to $p^* = 4$ with profit $\pi^* = 18$. Both games have unique Nash equilibria in pure strategies.[109] Three algorithms compete: independent Q-learning (IQL), Nash-Q, and WoLF-PHC.

Table 8: Convergence to Nash equilibrium in Cournot and Bertrand duopoly

Game	Algorithm	Action	Profit	$\lvert a - a^* \rvert$	Conv. iter
Cournot	IQL	2.95 ± 0.05	9.1 ± 0.0	0.05	1,000
	Nash-Q	2.89 ± 0.33	8.8 ± 1.3	0.17	1,000
	WoLF-PHC	3.00 ± 0.00	9.0 ± 0.0	0.00	1,000
Bertrand	IQL	4.00 ± 0.00	18.0 ± 0.0	0.33	1,000
	Nash-Q	4.00 ± 0.00	18.0 ± 0.0	0.33	1,000
	WoLF-PHC	3.95 ± 0.05	17.8 ± 0.2	0.38	1,000

Notes: Action and Profit report mean ± standard error across 20 seeds over the final 5,000 iterations. $|a - a^*|$ is the mean distance from Nash. Conv. iter is the first iteration where the smoothed average action enters a 0.5-neighborhood of Nash.

Table 8 reports the results. All three algorithms converge to Nash in both games within the first 5,000 iterations. IQL, which lacks any game-theoretic computation, matches the game-aware methods in these well-structured games with unique pure-strategy equilibria. The advantage of Nash-Q and WoLF-PHC emerges in games requiring mixed strategies, where IQL's deterministic policy limit prevents convergence.

5.2 Counterfactual Regret Minimization

Extensive-form games require a different approach. CFR bypasses equilibrium selection: instead of computing Nash equilibria at each step, it minimizes cumulative regret and lets the time-averaged strategy converge to equilibrium.

An extensive-form game consists of a game tree with information sets $\mathcal{I}_i$ partitioning player i's decision nodes. An information set groups nodes where the player cannot distinguish between them due to hidden opponent actions or private information. A behavioral strategy $\sigma_i : \mathcal{I}_i \to \Delta(\mathcal{A})$ assigns action probabilities at each information set.

The *counterfactual value* of action a at information set I is

$$v_i^\sigma(I, a) = \sum_{h \in I} \pi_{-i}^\sigma(h) \sum_{z \sqsupset ha} \pi^\sigma(ha, z) u_i(z) \tag{64}$$

[108] Correlated-Q learning (Greenwald and Hall, 2003) generalizes both Nash-Q and Minimax-Q by using correlated equilibrium, a probability distribution over joint actions enforced by a correlating device. The set of correlated equilibria contains all Nash equilibria. Correlated-Q converges under conditions analogous to Nash-Q.

[109] Each configuration runs for 50,000 iterations across 20 seeds.

where $\pi^\sigma_{-i}(h)$ is the probability opponents reach h, and $\pi^\sigma(ha, z)$ is the probability of reaching terminal z from ha. The counterfactual formulation weights by opponent reach $\pi^\sigma_{-i}(h)$ rather than joint reach $\pi^\sigma(h)$, ensuring non-zero updates even at rarely-visited information sets. Cumulative regret for action a is $R^T(I, a) = \sum_{t=1}^T [v^{\sigma^t}(I, a) - v^{\sigma^t}(I)]$. CFR updates via *regret matching*:[110]

$$\sigma^{T+1}(I, a) = \frac{[R^T(I, a)]^+}{\sum_{a'} [R^T(I, a')]^+}, \quad [x]^+ = \max(x, 0). \tag{65}$$

The average strategy $\bar{\sigma}^T$ converges to ε-Nash with $\varepsilon = O(|\mathcal{I}|\sqrt{|\mathcal{A}|}\Delta/\sqrt{T})$, where Δ is the range of payoffs (Zinkevich et al., 2008).[111]

CFR+ (Tammelin, 2014) replaces standard regret matching with Regret Matching+, which truncates cumulative regrets to zero after each update rather than only at action selection. The update becomes $R^{T+1}(I, a) = \max(R^T(I, a) + r^{T+1}(I, a), 0)$, where $r^{T+1}(I, a) = v^{\sigma^{T+1}}(I, a) - v^{\sigma^{T+1}}(I)$ is the instantaneous counterfactual regret. CFR+ also weights iteration t by t when computing the average strategy. While vanilla CFR converges at $O(1/\sqrt{T})$, CFR+ empirically converges at $O(1/T)$.[112] CFR+ enabled Bowling et al. (2015) to essentially solve heads-up limit Texas hold'em, a game with 3.16×10^{17} states, the first non-trivial imperfect-information game played competitively by humans to be essentially solved. Their program Cepheus achieved exploitability below 1 mbb/g (milli-big-blind per game).

5.3 Neural Extensions and Poker

Tabular CFR stores regrets at every information set, infeasible when $|\mathcal{I}| > 10^{14}$. Brown et al. (2019) approximate cumulative regrets with a neural network trained on sampled (information set, iteration, regret) tuples, weighting by iteration index to prioritize recent estimates; the current strategy derives from regret matching on the network's outputs. NFSP (Heinrich and Silver, 2016) takes a different approach, combining fictitious play[113] with deep Q-learning: each player maintains a best-response network Q_θ and an average strategy network $\bar{\pi}_\phi$ trained via supervised learning.

These methods scale CFR to large imperfect-information games. Deep CFR attained exploitability[114] of 37 mbb/g in heads-up flop hold'em (Brown et al., 2019). Libratus defeated top professionals in heads-up no-limit hold'em using nested subgame solving with CFR (Brown and Sandholm, 2018). Pluribus extended to six-player no-limit hold'em (Brown and Sandholm, 2019).

5.4 The Coase Conjecture

The durable goods monopoly is a canonical extensive-form bargaining problem with private information: the seller does not know the buyer's valuation.

[110] Regret matching is an action selection rule where the probability of choosing action a is proportional to the cumulative regret for not having chosen a in the past (truncated at zero). Actions with high regret receive higher probability; actions with negative regret (having performed worse than average) receive zero probability.

[111] An ε-Nash equilibrium is a strategy profile where no player can improve her expected payoff by more than ε through unilateral deviation. As $\varepsilon \to 0$, this converges to an exact Nash equilibrium.

[112] The $O(1/T)$ rate for CFR+ is empirically observed but not yet proven in full generality. Tammelin (2014) conjectured this rate based on extensive experiments across poker variants.

[113] Fictitious play (Brown, 1951) is a classical learning rule where each player best-responds to the empirical distribution of opponents' past actions.

[114] Exploitability measures how far a strategy is from Nash equilibrium, defined as the maximum expected gain an adversary could achieve by best-responding. In poker, it is measured in milli-big-blinds per game (mbb/g).

Coase (1972) conjectured that a durable goods monopolist[115] loses market power when buyers are patient. Unable to commit to future prices, the seller competes with her future self, eroding rents. As the inter-offer interval shrinks ($\delta \to 1$), price collapses to marginal cost and all surplus is extracted by buyers. Gul et al. (1986) formalized this for the gap case, where the seller's cost is strictly below all buyer valuations, guaranteeing trade and a unique stationary equilibrium.[116]

5.4.1 Model

A seller with zero cost faces a buyer with private valuation $v \in \{v_L, v_H\}$, where $\Pr(v = v_H) = \pi$. In each period $t = 1, 2, \ldots$, the seller posts price p_t; the buyer accepts or rejects. Upon acceptance at t, the seller receives $\delta^{t-1} p_t$ and the buyer receives $\delta^{t-1}(v - p_t)$, where $\delta \in (0, 1)$ is the common discount factor. The game ends upon acceptance or after T periods. Parameters: $v_L = 100$, $v_H = 200$, $T = 2$ periods.

5.4.2 Equilibrium Analysis

The screening price $P^*(\delta)$ makes the high-type buyer indifferent between accepting now and waiting.

$$v_H - P^* = \delta(v_H - v_L) \implies P^*(\delta) = v_H - \delta(v_H - v_L). \tag{66}$$

The seller's optimal strategy depends on π. Let $\Pi_{\text{screen}} = \pi P^* + (1 - \pi)\delta v_L$ and $\Pi_{\text{pool}} = v_L$. The seller screens if $\Pi_{\text{screen}} > \Pi_{\text{pool}}$, yielding threshold

$$\pi^* = \frac{v_L(1 - \delta)}{P^* - \delta v_L} = \frac{1}{2} \quad \text{when } v_H = 2v_L. \tag{67}$$

For $\pi < \pi^*$, the seller pools (offers v_L immediately). For $\pi > \pi^*$, the seller screens (offers P^*, then v_L if rejected).[117] The Coase conjecture manifests as $\delta \to 1$, where $P^*(\delta) \to v_L$ and the seller cannot extract surplus from high types.

5.4.3 Computational Results

I model the bargaining game as an extensive-form game and apply CFR.[118]

Table 9 reports results from varying π at fixed $\delta = 0.5$. CFR recovers the sharp phase transition at $\pi^* = 0.5$: pooling below, screening above. A δ-sweep at $\pi = 0.7$ confirms the screening price formula $P^*(\delta) = 200 - 100\delta$ with zero error across $\delta \in [0.1, 0.9]$; at $\delta \approx 0.75$, the seller switches to pooling as patient buyers erode the screening premium, consistent with the Coase conjecture.[119]

[115] A durable good provides utility over multiple periods (e.g., a car, appliance, or software license) rather than being consumed immediately. Unlike non-durable goods, durable goods create intertemporal competition, as the seller at time t competes with her own future self at $t + 1$.

[116] Stokey (1981) showed that a monopolist facing rational consumers cannot price discriminate intertemporally; Bulow (1982) proved that in the no-gap case, the monopolist prefers renting to selling. The no-gap case admits multiple equilibria with more complex dynamics.

[117] In a screening equilibrium, the seller uses price to separate buyer types: high-value buyers accept immediately at a high price, while low-value buyers reject and receive a lower offer. In a pooling equilibrium, the seller offers a single price that all types accept. The gap case ($0 = c < v_L$) guarantees trade in equilibrium.

[118] The game tree has two information sets for the seller (indexed by rejection history) and two for the buyer (indexed by private type). CFR runs for 5,000 iterations per parameter configuration; NashConv (sum of exploitabilities) measures convergence.

[119] Four stress tests were conducted: (1) awkward primes $(v_L, v_H) = (37, 83)$, converging to $P^* = 55$ (theory: 55.4); (2) information leak with a single seller information set at the root; (3) grid shift with 150 removed, recovering 145 (99.4%); (4) 3-period game, finding $P_1 = 120$ versus theoretical 136. The 3-period result reflects a tie-breaking equilibrium: at $P_1 = 136$ the high buyer is exactly indifferent, so the seller prefers $P_1 = 120$ (revenue 108 versus 86.4 if rejected).

Table 9: CFR Equilibrium vs. Theory: π-sweep at $\delta = 0.5$

π	P^*	P(Screen)	Theory	NashConv	Eq. Type	Status
0.10	150	0.000	0.0	5.0258	Pooling	✓
0.15	150	0.000	0.0	7.5249	Pooling	✓
0.20	150	0.000	0.0	10.0239	Pooling	✓
0.25	150	0.000	0.0	12.5229	Pooling	✓
0.30	150	0.000	0.0	15.0223	Pooling	✓
0.35	150	0.000	0.0	17.5216	Pooling	✓
0.40	150	0.000	0.0	20.0213	Pooling	✓
0.45	150	0.000	0.0	22.5213	Pooling	✓
0.50	150	0.001	0.5	25.0221	Indifferent	✓
0.55	150	0.002	1.0	27.5256	Screening	✓
0.60	150	0.506	1.0	30.0066	Screening	✓
0.65	150	0.999	1.0	26.2553	Screening	✓
0.70	150	1.000	1.0	22.5027	Screening	✓
0.75	150	1.000	1.0	18.7531	Screening	✓
0.80	150	1.000	1.0	15.0067	Screening	✓
0.85	150	1.000	1.0	11.2576	Screening	✓
0.90	150	1.000	1.0	7.5089	Screening	✓

Notes: P(Screen) is the probability the seller offers $P^* = 150$ in period 1. Theory column gives the predicted probability (0 for pooling, 1 for screening).

6 Offline Reinforcement Learning and Human Feedback

In the preceding chapters, the agent interacts with its environment while learning: it tries a price, observes a purchase, and updates its belief. Online learning is natural in digital markets where experimentation is cheap and feedback is instant. In many economically important settings, however, experimentation is impossible or prohibitively costly. A hospital cannot randomly assign treatments to learn optimal dosing. A central bank cannot experiment with interest rate schedules to discover optimal monetary policy. A firm inheriting a decade of transaction logs wants to improve its pricing rule without conducting new experiments during the transition. In each case, the agent has access to a fixed dataset of past decisions and outcomes, collected under some historical policy, and must learn the best possible new policy from this data alone.

This is the problem of *offline reinforcement learning*, also called *batch reinforcement learning*.[120] The agent never queries the environment. All learning happens from a fixed dataset $\mathcal{D} = \{(s_i, a_i, r_i, s_i')\}_{i=1}^n$ collected by some behavioral policy π_b. The goal is to find a policy $\hat{\pi}$ whose value $V^{\hat{\pi}}$ is as close to the optimal V^* as possible.

Online RL algorithms (Q-learning, SARSA, policy gradient methods from Section 2) can in principle be applied to offline data by treating the dataset as a replay buffer. In practice, this fails. The reason is *distributional shift*: the learned policy $\hat{\pi}$ inevitably queries state-action pairs that the behavioral policy π_b never visited, and the Q-function at these unseen pairs is pure extrapolation. Because the Bellman backup propagates these extrapolation errors through the max operator, errors compound geometrically across the planning horizon, producing arbitrarily poor policies.[121]

[120] The term "batch RL" was standard in the earlier literature (Ernst et al., 2005; Lange et al., 2012). "Offline RL" became dominant after Levine et al. (2020), who distinguished it from off-policy RL (which still collects new data, just under a different policy than the target). I use "offline RL" throughout.

[121] This failure mode was first demonstrated empirically by Fujimoto et al. (2019), who showed that standard off-policy algorithms (DDPG, SAC) trained purely from a static dataset performed worse than the behavioral

6.1 The Pessimism Principle

The overestimation failure has a clean theoretical characterization. Consider tabular Q-learning with a fixed dataset. At any state-action pair (s, a) not in the dataset, the empirical Bellman backup is undefined, but the max in $\max_{a'} \hat{Q}(s', a')$ may still select it if the randomly-initialized Q-value happens to be high. With function approximation, the problem is subtler but identical in spirit: the function approximator can assign high values to out-of-distribution inputs without any corrective signal.

The solution, established simultaneously by several groups, is the *pessimism principle*: construct a lower confidence bound on the Q-function and optimize against it. Formally, given a dataset $\mathcal{D}$ and a confidence parameter δ, construct a penalty function $\Gamma(s, a)$ that is large where data coverage is poor, and define the pessimistic Q-function

$$\tilde{Q}(s, a) = \hat{Q}(s, a) - \Gamma(s, a) \tag{68}$$

where $\hat{Q}$ is the standard empirical Bellman solution. The policy $\hat{\pi}(s) = \arg\max_a \tilde{Q}(s, a)$ selects actions that are both high-value *and* well-supported by data.

Definition D1 (Pessimistic Value Iteration, PEVI (Jin et al., 2021))**.** *Given dataset $\mathcal{D}$, penalty function $\Gamma_h(s, a)$ for each stage h, and horizon H, PEVI computes*

$$\hat{Q}_h(s, a) = r_h(s, a) + \hat{P}_h \hat{V}_{h+1}(s, a) - \Gamma_h(s, a) \tag{69}$$

$$\hat{V}_h(s) = \max\{\max_a \hat{Q}_h(s, a),\, 0\} \tag{70}$$

$$\hat{\pi}_h(s) = \arg\max_a \hat{Q}_h(s, a) \tag{71}$$

where $\hat{P}_h$ is the empirical transition operator estimated from $\mathcal{D}$.

Jin et al. (2021) show that with $\Gamma_h(s, a) = c \cdot \sqrt{H^3/N_h(s, a)}$, where $N_h(s, a)$ counts visits to (s, a) at stage h and c is an absolute constant, PEVI achieves

$$V^* - V^{\hat{\pi}} \leq \tilde{O}\left(\sum_{h=1}^{H} \mathbb{E}_{\pi^*}\left[\sqrt{\frac{1}{N_h(s_h, a_h)}}\right]\right) \tag{72}$$

with high probability. The bound depends on the data coverage at states and actions visited by the *optimal* policy π^*, not the full state-action space. This is the key advantage of pessimism over uniform coverage requirements.

6.1.1 Concentrability and Coverage

The bound in (72) is instance-dependent, scaling with how well π_b covers π^*. The classical way to formalize this is through *concentrability coefficients* (Munos and Szepesvári, 2008).

Definition D2 (Single-policy concentrability)**.** *The single-policy concentrability coefficient of π^* with respect to π_b is*

$$C^* = \max_{h \in [H]} \left\| \frac{d_h^{\pi^*}}{d_h^{\pi_b}} \right\|_\infty \tag{73}$$

where $d_h^\pi(s, a)$ is the state-action occupancy measure of π at stage h.

When C^* is finite, the optimal policy visits only states and actions that π_b also visits with non-negligible probability, and the $1/\sqrt{N_h(s_h, a_h)}$ terms in (72) remain controlled. Rashidinejad

policy itself, even when that behavioral policy was a partially-trained, mediocre agent. The gap widened with dataset size, the opposite of what one expects from more data.

et al. (2021) prove that single-policy concentrability is both necessary and sufficient for offline learning: if $C^* < \infty$, then $O(|\mathcal{S}|H^2C^*/\epsilon^2)$ samples suffice for an ϵ-optimal policy; if $C^* = \infty$, no algorithm can guarantee suboptimality better than the behavioral policy.

6.1.2 Impossibility Results

Zanette (2021) establish fundamental limits on offline RL by showing that it can be exponentially harder than online RL. Specifically, there exist MDPs with S states and horizon H where online RL finds an ϵ-optimal policy in $\mathrm{poly}(S, H, 1/\epsilon)$ episodes, but any offline algorithm requires $\Omega(2^H)$ samples unless the dataset covers all reachable states. The construction uses a binary tree MDP where the optimal path visits a unique leaf, and any dataset that misses this leaf provides no information about the optimal action at the root.

The practical implication is that offline RL is not a universal replacement for online experimentation. When the behavioral policy is far from optimal, especially in long-horizon problems, offline methods provably cannot recover the optimal policy without exponentially large datasets. Pessimistic algorithms are the best one can do, but they are still fundamentally constrained by what the data contains.

6.2 Algorithms

I present four practical algorithms that instantiate different approaches to the distributional shift problem. All four can be understood as modifications of standard Q-learning (Section 2) that prevent the agent from overvaluing actions outside the data support.

6.2.1 Fitted Q-Iteration

Fitted Q-Iteration (FQI, Ernst et al., 2005) is the simplest offline RL algorithm, predating the modern pessimism framework. FQI applies the standard Bellman backup iteratively using supervised regression on the fixed dataset, as described in Section 2. It does not include any explicit pessimism mechanism. When state-action coverage is poor, the max in the Bellman backup selects the highest Q-value among all actions at s', including actions never observed in the data. If the function approximator generalizes poorly at these unseen actions, targets become noisy and biased upward, causing the overestimation cascade. FQI works well when coverage is good but degrades as coverage gaps grow.[122]

6.2.2 Conservative Q-Learning

Conservative Q-Learning (CQL, Kumar et al., 2020) adds an explicit penalty that pushes down Q-values at actions not well-represented in the data. The key idea is to add a regularizer to the Bellman error objective that minimizes Q-values under a broad distribution over actions while maximizing Q-values at the actions actually taken in the dataset.

Definition D3 (Conservative Q-Learning)**.** *CQL modifies the standard Bellman error objective by adding a conservative regularizer. At each iteration, CQL solves*

$$\hat{Q}_{k+1} = \arg\min_{Q}\ \alpha\left(\mathbb{E}_{s\sim\mathcal{D}}\left[\log\sum_{a}\exp Q(s,a)\right] - \mathbb{E}_{(s,a)\sim\mathcal{D}}[Q(s,a)]\right) + \frac{1}{2}\mathbb{E}_{\mathcal{D}}\left[\left(Q(s,a) - \hat{\mathcal{B}}^{\pi_k}\hat{Q}_k(s,a)\right)^2\right] \tag{74}$$

where $\alpha > 0$ is a hyperparameter controlling the degree of conservatism, and $\hat{\mathcal{B}}^{\pi_k}$ is the empirical Bellman operator.

[122] Munos and Szepesvári (2008) prove finite-time error bounds for FQI under approximate Bellman completeness and all-policy concentrability. Both assumptions are strong, and violation of either leads to divergence in practice.

The first term in the regularizer, $\log \sum_a \exp Q(s,a)$, is a soft maximum over all actions, pushing down Q-values uniformly. The second term, $\mathbb{E}_{\mathcal{D}}[Q(s,a)]$, pulls Q-values back up at the data actions. The net effect is a penalty on Q-values for actions that appear infrequently relative to their softmax contribution. Kumar et al. (2020) prove that the resulting Q-function is a pointwise lower bound on the true Q-function (Theorem 3.2), making it a concrete instantiation of the pessimism principle from (68) with an implicit, data-adaptive penalty Γ.

6.2.3 Alternative Pessimism Mechanisms

Other approaches achieve pessimism through different mechanisms. Implicit Q-Learning (IQL, Kostrikov et al., 2022) avoids querying unseen actions entirely by learning the value function through expectile regression, approximating the in-sample maximum without evaluating Q at out-of-distribution actions.[123] Batch-Constrained Q-Learning (BCQ, Fujimoto et al., 2019) constrains the policy to actions with high likelihood under the learned behavioral distribution, addressing distributional shift at the policy level rather than the value function level. BCQ's performance is bounded by the quality of actions in the dataset; if the behavioral policy never takes the optimal action at some state, BCQ cannot discover it regardless of sample size.

6.3 Simulation Study: Offline RL for Dynamic Pricing

The simulation study evaluates the four offline RL algorithms on a perishable inventory pricing problem with demand regime switching. A retailer with $I_{\max} = 30$ units of perishable inventory must set prices over $H = 20$ periods. The state (i, d, t) consists of current inventory $i \in \{0, \ldots, 30\}$, demand regime $d \in \{1, 2, 3, 4\}$, and time remaining $t \in \{1, \ldots, 20\}$. The action is a price $p \in \{1, \ldots, 10\}$. Demand follows $Q \sim \text{Poisson}(\lambda_0[d] \cdot e^{-0.15p})$ with base rates $\lambda_0 = (1.5, 3.0, 5.0, 8.0)$, and the reward is $r = p \cdot \min(Q, i)$. Demand regimes follow a 4-state Markov chain with diagonal persistence 0.6. Unsold inventory at the terminal period incurs a spoilage cost of \$2.00 per unit, making clearance pricing valuable near the deadline.[124] The behavioral policy represents a conservative pricing team that always sets the maximum price ($p = 10$) regardless of demand regime, inventory, or time remaining, with probability 0.85 and randomizes uniformly over all prices with probability 0.15.[125] All episodes start at full inventory ($i = 30$). All offline methods train on 500 episodes and are evaluated over 1,000 episodes against the DP optimal policy computed by backward induction.[126] Results are averaged over 20 independent seeds.

FQI achieves 81.2% of the DP optimal, substantially below the behavioral cloning baseline of 88.0% (Table 10). Without any mechanism to control extrapolation error, the $\max_{a'} Q(s', a')$ operator in FQI's Bellman backup selects overestimated Q-values at out-of-distribution actions, and these overestimates compound across 200 iterations of fitted Q-iteration. CQL and IQL both

[123] IQL uses an asymmetric squared loss $L_2^\tau(u) = |\tau - \mathbb{1}\{u < 0\}| \cdot u^2$ with expectile parameter $\tau \in (0.5, 1)$. As $\tau \to 1$, the expectile converges to the in-sample maximum; $\tau = 0.7$ is typical.

[124] The spoilage penalty creates distributional shift. The optimal policy adapts prices to inventory level and time remaining, using lower prices near the deadline when inventory is high. The behavioral policy ignores these state variables and prices at the maximum, so the Q-function at state-adapted pricing actions is extrapolation from sparse data. The \$2.00 penalty is a deliberate design choice: under harsher penalties (e.g., \$10 per unit), all methods collapse to 48–53% of optimal regardless of algorithmic sophistication, confirming that no offline correction can overcome severe distributional shift when the penalty regime amplifies consequences of the behavioral policy's suboptimality.

[125] With 500 episodes (10,000 transitions) on a state-action space of 24,800 pairs, the 85% concentration at price 10 ensures that the behavioral state-action occupancy diverges significantly from the optimal policy's occupancy, while the 15% uniform component provides sparse off-policy coverage.

[126] FQI uses the standard Bellman backup with $\max_{a'} Q(s', a')$, deliberately without a target network, to isolate the overestimation cascade as a pedagogical baseline. Adding target networks to FQI mitigates but does not eliminate extrapolation error. CQL and IQL include target networks following their original implementations (Kumar et al., 2020; Kostrikov et al., 2022); for CQL, target networks proved essential, as the conservative penalty amplifies bootstrap instability without them.

Table 10: Policy value for each offline RL method, expressed as mean return and percentage of the DP optimal. Standard errors computed over 20 seeds.

Method	Mean Return	% of Optimal
DP Oracle	192.41 ± 0.33	100.0%
BC	169.27 ± 0.60	88.0%
FQI	156.18 ± 1.68	81.2%
CQL	176.73 ± 1.13	91.9%
IQL	176.98 ± 0.56	92.0%
BCQ	169.27 ± 0.60	88.0%

exceed the behavioral baseline, achieving 91.9% and 92.0% respectively.[127] CQL's conservative penalty suppresses Q-values at actions not well-represented in the data while preserving relative ordering among data-supported actions, allowing the policy to discover lower prices that clear inventory near the deadline. IQL achieves the same improvement through a different mechanism: by replacing $\max_{a'} Q(s', a')$ with the expectile-regressed value function $V(s')$ as the Bellman continuation, IQL avoids querying Q at unseen actions during training while still extracting a policy that improves on the behavioral at states where the 15% noise component revealed better pricing actions. BCQ matches the behavioral at 88.0%; its action constraint restricts the policy to prices near 10, preventing both overestimation and improvement.[128]

These results validate several theoretical predictions from the preceding sections. FQI's degradation below the behavioral baseline (81.2% versus 88.0%) demonstrates the overestimation cascade described in Section 6.2: without pessimism, the $\max_{a'}$ operator selects overestimated Q-values at out-of-distribution actions, and 200 iterations of bootstrapping compound these errors geometrically (Fujimoto et al., 2019). CQL and IQL both exceeding BC confirms the pessimism principle (Section 6.1): CQL's conservative penalty produces a pointwise lower bound on the true Q-function (Definition D3, Theorem 3.2 of Kumar et al. 2020), while IQL's expectile regression avoids querying Q at unseen actions entirely (Kostrikov et al., 2022). BCQ matching BC exactly illustrates the action-constraint tradeoff: performance is bounded by the quality of actions in the data, and when the behavioral policy concentrates on a single action, no amount of Q-learning can overcome the constraint.

Varying the behavioral noise parameter ϵ_b confirms that both CQL and IQL remain above the behavioral cloning baseline across all coverage levels, while FQI exhibits non-monotone behavior and BCQ collapses when a nearly uniform behavioral policy renders the action constraint vacuous.

6.4 From Offline RL to Human Feedback

The offline RL algorithms above assume access to a scalar reward signal in the dataset. In many domains, however, the reward itself is unknown. Scalar rewards may be unavailable, but ordinal preferences over trajectories are easy to elicit: a human evaluator can reliably say "response A is better than response B" without assigning numerical scores. Reinforcement Learning from Human Feedback (RLHF) uses these ordinal comparisons to learn a proxy reward function r_θ, which then serves as the scalar signal for standard RL optimization. This is a two-stage forward problem: the analyst first learns a reward from human judgments and then optimizes a policy against that learned reward (Christiano et al., 2017).

[127] CQL uses $\alpha = 0.1$, the result of a search over $\alpha \in \{5.0, 2.0, 0.5, 0.1\}$. Larger values push Q-values down too aggressively, collapsing the learned policy to the behavioral action at most states; the right α is problem-specific and can vary by orders of magnitude.

[128] When the behavioral policy concentrates 85% probability at a single action, BCQ's threshold constraint permits only that action at most states, effectively reducing BCQ to behavioral cloning regardless of the learned Q-values.

6.5 Learning Rewards from Preferences

The canonical RLHF framework is built on a formal model of human preference. The observed data consists of tuples (s, y_w, y_l), where s is a context, and y_w and y_l are two outputs, with y_w being the "winner" preferred by a human. Assuming preferences follow a latent utility model, the Bradley-Terry model (Bradley and Terry, 1952) gives the probability that y_w is preferred: $P(y_w \succ y_l|s) = \sigma(r_\theta(s, y_w) - r_\theta(s, y_l))$, where $r_\theta : \mathcal{S} \times \mathcal{Y} \to \mathbb{R}$ is a learned *reward model* parameterized by θ, trained to approximate human preferences (not the ground-truth reward, which is unobserved), and $\sigma(\cdot)$ is the logistic function. This formulation is a binary logit model (Section 1.1, Equation 1).[129] The reward model parameters θ are estimated by minimizing the negative log-likelihood of the observed human choices:

$$\mathcal{L}(\theta) = -\mathbb{E}_{(s,y_w,y_l)\sim\mathcal{D}} \left[\log \sigma \left(r_\theta(s, y_w) - r_\theta(s, y_l)\right)\right], \tag{75}$$

In the LLM setting, the "outputs" y_w and y_l are token sequences, i.e., trajectories of the autoregressive policy[130], so preferences are over trajectories rather than single actions. The preference loss in Equation (75) is MLE of a choice model where the "alternatives" are trajectory segments and the "choice" is the human-preferred one.

6.6 The RLHF Pipeline and Direct Optimization

The learned r_θ then serves as a proxy objective for policy optimization. The canonical pipeline (Ouyang et al., 2022) proceeds in three stages: supervised fine-tuning (SFT) on demonstrations to produce a reference policy π^{SFT}, reward model training from preferences, and PPO fine-tuning of π_ϕ against the frozen r_θ with a KL-divergence penalty preventing drift into regions where r_θ is unreliable. The objective is

$$J(\phi) = \mathbb{E}_{s\sim\mathcal{D}, y\sim\pi_\phi(\cdot|s)}[r_\theta(s, y)] - \lambda_{KL}\mathbb{E}_{s\sim\mathcal{D}}[D_{KL}(\pi_\phi(\cdot|s) \,||\, \pi^{SFT}(\cdot|s))], \tag{76}$$

where D_{KL} is the Kullback-Leibler divergence and λ_{KL} controls the penalty strength.[131] Without this constraint, the policy exploits inaccuracies in r_θ to achieve high proxy scores with degenerate behavior ("reward hacking"), the RLHF analogue of divergence under function approximation (Section 3.3).

The KL-regularized objective in Equation (76) admits a Bayesian interpretation (Korbak et al., 2022). In this view, the reference policy π^{SFT} acts as a prior distribution over plausible responses. The reward model r_θ provides evidence, specifying which responses are more desirable. The goal of alignment is to find the posterior distribution π^* that optimally combines the prior with this evidence. This ideal posterior policy takes the form of Equation (77):

$$\pi^*(y|s) \propto \pi^{SFT}(y|s) \exp\left(\frac{r_\theta(s, y)}{\lambda_{KL}}\right). \tag{77}$$

The reward function scaled by λ_{KL} defines the log-likelihood, so the KL-regularized objective $J(\phi)$ is equivalent (up to an additive constant) to the Evidence Lower Bound (ELBO) for this Bayesian inference problem. Maximizing the RLHF objective via PPO is therefore variational inference: finding the policy π_ϕ that minimizes KL divergence to π^*. This reframes the KL

[129] Iskhakov et al. (2020) discuss the contrasts and synergies between machine learning and structural econometrics, including the shared reliance on logit-based choice models that underlies both RLHF and dynamic discrete choice estimation.

[130] An *autoregressive* language model generates text token by token: at each step it outputs a distribution over the vocabulary conditioned on all preceding tokens, then samples the next token; the full response is therefore a trajectory in token space and the model acts as a sequential policy over a vocabulary-sized action set.

[131] λ_{KL} denotes the KL penalty weight, reserving β for model parameters and γ for discount factors. The standard RLHF literature, including Rafailov et al. (2023), uses β for this parameter.

penalty as a structural component of the inference problem rather than an ad-hoc regularizer.

Despite this closed-form characterization, the three-stage RLHF pipeline is complex to implement, requiring training multiple large models and a computationally expensive RL loop. Direct Preference Optimization (DPO), introduced by Rafailov et al. (2023), collapses the pipeline into a single supervised learning objective by reparameterizing the reward function in terms of π^* and π^{SFT}, as in Equation (78):

$$r(s, y) = \lambda_{KL} \log \left(\frac{\pi^*(y|s)}{\pi^{SFT}(y|s)} \right) + \lambda_{KL} \log Z(s). \tag{78}$$

When this analytical expression for the reward is substituted into the Bradley-Terry preference loss from Equation (75), the unknown partition function $Z(s)$ cancels out. This yields a loss function that depends only on the policy π_ϕ being optimized and the fixed reference policy π^{SFT}. The DPO objective, given in Equation (79), is thus a simple binary cross-entropy loss over policy likelihoods:

$$\mathcal{L}_{DPO}(\phi; \pi^{SFT}) = -\mathbb{E}_{(s,y_w,y_l)\sim\mathcal{D}} \left[\log \sigma \left(\lambda_{KL} \log \frac{\pi_\phi(y_w|s)}{\pi^{SFT}(y_w|s)} - \lambda_{KL} \log \frac{\pi_\phi(y_l|s)}{\pi^{SFT}(y_l|s)} \right) \right]. \tag{79}$$

This objective is optimized on ϕ using standard supervised learning with a static preference dataset. The gradient increases the likelihood of preferred responses y_w and decreases the likelihood of dispreferred responses y_l, relative to the reference policy.

6.7 Simulation Study: Preference Learning in Job Search

A worker searches for jobs in a labor market with compensating differentials, following a McCall (1970)-style search model. Each job is characterized by a wage $w \in \{20, 28, 38, 50, 65, 82, 100, 125\}$ (thousands) and an amenity level $z \in \{0, 1, \dots, 6\}$ capturing commute quality, flexibility, and job security. The state space has 112 states: 56 searching states in which the worker observes a pending offer (w_i, z_j) and decides to accept or reject, plus 56 employed states (w_i, z_j) in which the worker decides to stay or quit. The offer distribution exhibits compensating differentials, with wage rank and amenity rank negatively correlated ($\rho = -0.74$): high-wage offers cluster with low amenities and vice versa. The worker's true per-period utility is $u(w, z) = \alpha \log(w) + (1-\alpha)z$ with $\alpha = 0.6$, but this function is unobserved; the worker can only compare career trajectories ("I prefer path A to path B"), exactly as in stated-preference surveys in labor economics. While searching, the worker receives the unemployment benefit $u_b = \alpha \log(b)$ where $b = 28$. Layoffs occur with probability $p = 0.05$ per period, and the discount factor is $\gamma = 0.95$. Dynamic programming gives $V^*(s_0) = 74.13$; the optimal policy accepts 25 of 56 offer types and stays employed at 25 of 56 job types. Preference data is generated by rolling out a uniform random policy from a random searching state. Each rollout produces a career segment of $L = 15$ periods recording states and actions. For each of K comparisons, two independent career segments are generated; the segment with higher cumulative discounted utility under the true (unobserved) utility function is labeled as preferred via the Bradley-Terry model. The optimal accept/reject and stay/quit boundaries cut diagonally in wage-amenity space, reflecting the compensating differential structure.

Six methods are compared across $K \in \{25, 50, 100, 200, 500, 1{,}000, 2{,}000, 5{,}000\}$, averaged over 30 seeds. The neural network RLHF reward model is a two-layer MLP trained by Bradley-Terry MLE on the K segment pairs; per-transition rewards are discount-weighted and summed to obtain a segment score, and the resulting 112-state reward table is solved by value iteration.[132] The correctly specified structural model parameterizes utility as $\hat{u} = \hat{\alpha} \log(w) + (1 - \hat{\alpha})z$, esti-

[132]The neural network has 4 inputs (normalized log-wage, normalized amenity, employment indicator, action), 32 hidden units per layer, and ∼1,200 parameters. The logistic loss from Equation (75) is applied to discount-weighted segment reward sums.

mating the single parameter $\hat{\alpha}$ via Bradley-Terry MLE, then solves the MDP. The misspecified model uses $\hat{u} = \hat{\alpha}\log(w) + (1-\hat{\alpha})\bar{z}$, where $\bar{z} = 3$ is the mean amenity level; this model ignores amenity variation across jobs, treating all amenities as identical. DPO trains a tabular softmax policy directly from trajectory comparisons, bypassing reward modeling entirely.[133] Tabular Q-learning (10,000 episodes, $\varepsilon = 0.15$, learning rate 0.1) and exact DP provide scalar-reward baselines. All four preference methods receive identical comparison data per seed; DPO receives a same-state variant generated from the same seed.

The correctly specified structural model outperforms all alternatives: even at $K = 25$ it achieves 99.9% of DP-optimal, illustrating the power of correct specification when the model has a single free parameter. Second, the neural network converges more slowly but reaches 99.9% by $K = 5{,}000$, reflecting the higher sample complexity of a flexible model relative to a one-parameter structural specification. Third, DPO plateaus at approximately 95% by $K = 500$ and does not improve further.[134] This plateau is qualitatively different from the gridworld failure (−118%): DPO recovers a reasonable policy, but the gap does not close with additional data.[135] The misspecified constant-amenity model plateaus at 91% regardless of K, because additional preference data cannot correct the omitted variable.

Table 11: Diagnostics at $K = 5{,}000$ (single seed): policy agreement with π^*, value-function correlation, and mean accepted wage and amenity for each method.

Method	Policy agree. (%)	V^π corr.	Mean amenity	Mean wage	$\hat{\alpha}$
NN RLHF	96.4	1.000	4.67	73	—
DPO	57.1	0.777	3.38	63	—
Correct	100.0	1.000	4.84	71	0.597
Misspecified	50.0	0.889	3.00	70	—
Optimal	100.0	1.000	4.84	71	—

Table 11 provides state-level diagnostics at $K = 5{,}000$. The structural model and neural network achieve near-perfect policy agreement with π^* (100% and 96% respectively). DPO agrees on only 57% of states with value-function correlation 0.78, systematically underselecting on both wage and amenity dimensions.[136] The misspecified model agrees on 50%, with disagreements concentrated where amenity variation matters.[137]

A segment length ablation at $K = 2{,}000$ confirms this pattern. At $L = 1$, both methods perform poorly because single-step comparisons carry minimal information about long-run value. The neural network recovers rapidly (by $L = 3$) because the reward model aggregates per-transition estimates over longer segments and value iteration propagates them through the full transition structure; DPO improves monotonically but plateaus at its ~95% ceiling, as

[133] DPO uses 112 logit parameters ϕ_s, one per state, trained via the DPO loss (Equation 79) with Adam optimization over a sweep of $\lambda_{KL} \in \{0.01, 0.05, 0.1, 0.5, 1.0, 5.0\}$, selecting the λ_{KL} that minimizes training loss. The reference policy is uniform: $\pi^{SFT}(a|s) = 0.5$. To match the LLM setup where both completions condition on the same prompt, DPO comparison pairs start from the same initial state.

[134] DPO learns only from (s, a) pairs visited in training trajectories generated by the random behavioral policy. It cannot propagate value to undervisited states the way value iteration does after learning a reward model, so states poorly covered by the behavioral policy remain suboptimal regardless of K.

[135] DPO fails catastrophically in gridworld because transitions are stochastic (10% slip probability) and rewards are transition-dependent. The same (s, a) pair yields different rewards depending on whether the agent slipped, so the DPO loss conflates policy quality with transition luck. In the job search model, accept/reject deterministically changes employment status, and only the 5% layoff probability introduces stochastic transitions.

[136] DPO's mean accepted amenity of 3.4 parallels the misspecified structural model's 3.0, though the mechanisms differ: the misspecified model ignores amenity variation by construction, while DPO underweights amenities because the random behavioral policy underrepresents high-amenity employed states in the training data.

[137] An online versus offline ablation for the neural network at $K = 1{,}000$ (20 seeds) shows comparable performance: online 73.92 ± 0.05, offline 73.99 ± 0.03 ($p = 0.09$). The random behavioral policy already provides diverse career trajectories covering the full wage-amenity space.

it must learn the policy directly from trajectory comparisons without access to the transition model.

Two-stage RLHF separates preference estimation (a static econometric problem) from dynamic programming (which exploits the known transition model); DPO conflates the two and forfeits the transition structure that structural modelers typically have access to.[138]

7 Offline RL under Unobserved Confounding

The offline RL algorithms of the preceding section assume that the behavioral policy μ is a known function of the observed state s_t. In many applied settings this assumption fails. A hospital's treatment decisions depend on unobserved patient severity. A firm's pricing depends on private demand signals unavailable to the analyst. When unobserved confounders simultaneously influence both the behavioral policy and the transitions or rewards, standard off-policy evaluation produces biased results, the sequential analogue of omitted variable bias in regression. This section formalizes the problem and reviews identification strategies with direct econometric analogues.

7.1 The Confounded MDP

Zhang and Bareinboim (2019), Zhang and Bareinboim (2020), and Kallus and Zhou (2020) formalize confounded sequential decision problems using Pearl's do-operator.[139]

Definition D4 (Confounded MDP (Zhang and Bareinboim, 2020))**.** *A confounded MDP is a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{U}, P, r, \gamma)$ where $\mathcal{U}$ is a space of unobserved confounders and the dynamics are governed by structural equations*

$$U_t \sim P_U(\cdot \mid S_t), \tag{80}$$
$$A_t \sim \mu(\cdot \mid S_t, U_t), \tag{81}$$
$$R_t = f_R(S_t, A_t, U_t) + \epsilon_t, \tag{82}$$
$$S_{t+1} \sim f_S(\cdot \mid S_t, A_t, U_t). \tag{83}$$

The behavioral policy μ depends on the unobserved confounder U_t through Equation (81). An evaluation policy $\pi(a \mid s)$ depends only on the observed state.

Because μ depends on U_t, conditioning on $\{A_t = a\}$ carries information about the confounder, so the observational and interventional transitions diverge: $P(s' \mid s, a) \neq P(s' \mid s, \mathrm{do}(a))$. The Bellman equation for policy evaluation must use interventional, not observational, transition probabilities.

Definition D5 (Causal Bellman Operator (Zhang and Bareinboim, 2020))**.** *The causal Bellman operator $\mathcal{T}_c^\pi$ for policy π in a confounded MDP is*

$$(\mathcal{T}_c^\pi V)(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s' \in \mathcal{S}} P(s' \mid s, \mathrm{do}(a)) \big[r(s,a) + \gamma V(s')\big], \tag{84}$$

where $r(s,a) = \mathbb{E}[R_t \mid S_t = s, \mathrm{do}(A_t = a)]$ is the interventional expected reward.

[138]This advantage is specific to settings where the transition model is known or estimable; in domains without a tractable transition model, DPO's single-stage approach avoids compounding errors from reward model estimation.

[139]The interventional distribution $P(Y \mid \mathrm{do}(X = x))$ arises when X is set externally rather than observed passively, severing all incoming causal influences on X while leaving the remaining data-generating process intact. The do-operator is formalized within the structural causal model (SCM) framework of Pearl (2009).

Lemma L1 (Bias of Naive Off-Policy Evaluation (Kallus and Zhou, 2020))**.** *In a confounded MDP, solving the Bellman equation with observational transitions $P(s' \mid s, a)$ yields $\hat{V}^{\pi}_{naive}(s) \neq V^{\pi}(s)$. The importance-sampling estimator using the marginal propensity $\mu(a \mid s)$ rather than the true propensity $\mu(a \mid s, u)$ is also biased.*[140]

7.2 Identification Strategies

The backdoor criterion (Pearl, 2009) provides the primary path to identification.

Theorem 6 (Backdoor Identification in Confounded MDPs (Pearl, 2009; Zhang and Bareinboim, 2020))**.** *If observed variables $\mathbf{Z}_t$ block all backdoor paths from A_t to S_{t+1} and no element of $\mathbf{Z}_t$ is a descendant of A_t, the interventional transition is identified:*

$$P(s' \mid s, \mathrm{do}(a)) = \sum_{\mathbf{z}} P(s' \mid s, a, \mathbf{z})\, P(\mathbf{z} \mid s). \tag{85}$$

Three alternative identification strategies apply when no backdoor variable is available, each with a direct econometric analogue. The front-door criterion (Pearl, 2009) identifies causal effects through an observed mediator M_t that intercepts all directed paths from action to outcome, the sequential analogue of mediation analysis. Instrumental variables (Liao et al., 2024) use an exogenous instrument Z_t satisfying relevance and exclusion to recover the transition function from conditional moment restrictions, directly paralleling two-stage least squares.[141] Proximal causal inference (Bennett and Kallus, 2021) uses two noisy proxy variables for the latent confounder, solving a bridge function equation to recover interventional quantities without ever observing the confounder. For comprehensive surveys of causal RL, including causal representation learning, counterfactual policy optimization, and causal transfer, see Deng et al. (2023) and da Costa Cunha et al. (2025).

7.3 Simulation Study: Confounded Retail Pricing

I construct a 5-state engagement funnel with two actions (promote, hold price) and an absorbing conversion state. A retailer manages customers through engagement stages, deciding whether to offer a promotional discount. The data-generating process embeds four distinct sources of causal variation (observed market conditions, unobserved consumer sentiment, an independent cost shock as instrument, and a mediator), enabling simultaneous validation of all four identification strategies from a single DGP.[142]

The naive estimator's bias grows monotonically with ρ because observational transitions overestimate promotion success: the promote action is more likely when $U_t = 1$, which correlates with favorable conditions, so per-step bias compounds through the Bellman recursion (Figure 2). The backdoor and front-door estimators eliminate bias at all ρ, validating Theorem 6. The IV estimator maintains low bias but higher variance due to the Wald ratio's sensitivity to instrument strength. The proximal estimator achieves low bias with moderate variance, confirming that bridge functions recover causal effects from noisy proxies.

[140] This is the sequential analogue of omitted variable bias. In the static case, regressing Y on X without controlling for U yields a biased coefficient. In the sequential case, the bias propagates through the Bellman recursion and can amplify over the horizon.

[141] Liao et al. (2024) illustrate with NICU assignment, where differential travel time to specialty care serves as an instrument for admission decisions confounded by unobserved patient severity.

[142] Market conditions $Z_t \sim \text{Bernoulli}(0.5)$ satisfy the backdoor criterion. Consumer sentiment U_t is the unobserved confounder, with $P(U_t{=}1 \mid Z_t{=}1) = 0.9$. An independent cost shock serves as the instrument. Promotions trigger marketing follow-ups M_t as the mediator. Two noisy proxies of U_t (CRM score and browsing behavior) satisfy the proximal conditions. The behavioral policy $\mu(\text{promote} \mid s, U_t, \text{IV}_t) = 0.55 + \rho \cdot 0.25 \cdot (2U_t - 1) + 0.15 \cdot (\text{IV}_t - 0.5)$, where ρ controls confounding strength. Each configuration uses 2,000 trajectories averaged over 20 seeds.

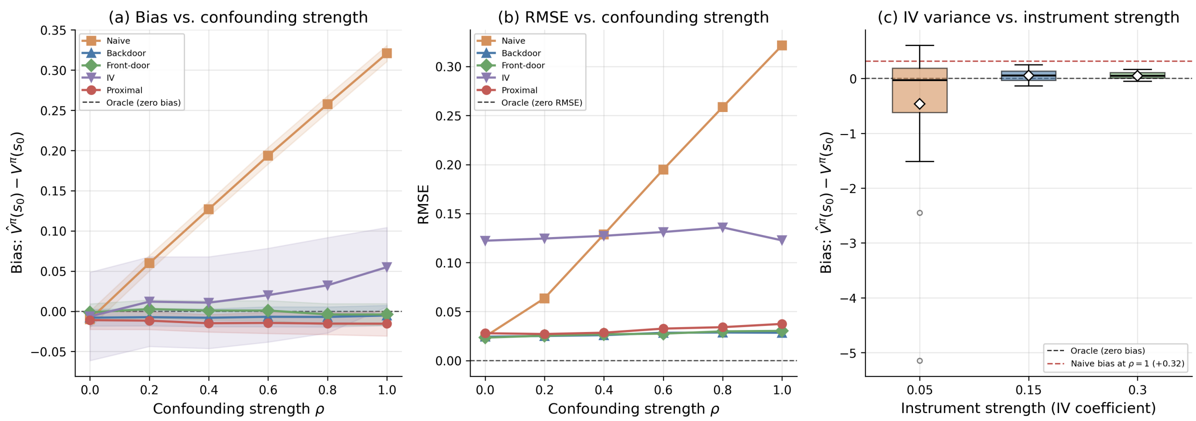

Figure 2: (a) Bias of five OPE estimators as a function of confounding strength ρ. (b) RMSE of five estimators as a function of ρ. (c) IV estimator bias distribution vs. instrument strength at $\rho = 1$; dashed red line is naive estimator bias.

8 Conclusion

Reinforcement learning algorithms are asymptotic approximations to classical dynamic programming operators. Value iteration becomes Q-learning when expectations are replaced by single samples; policy iteration becomes the natural policy gradient when the greedy improvement step is approximated by gradient ascent. The stochastic approximation framework guarantees that under appropriate step-size conditions, noisy iterates converge to the same fixed points as their deterministic counterparts. This survey has traced these connections through the theory of contractions and gradient domination, through structural estimation where RL serves as a computational tool for solving high-dimensional Bellman equations, through strategic games where independent learners converge to equilibrium, and through preference learning where the Bradley-Terry model bridges discrete choice econometrics and RLHF. The practical vulnerabilities of these methods are real: deep RL lacks convergence guarantees outside tabular settings, results are sensitive to hyperparameters and random seeds, and the absence of standardized simulators for economic environments remains a binding constraint. When guided by economic structure, reinforcement learning extends the reach of dynamic programming to problems that were previously intractable.

References

S. Adusumilli, M. Eckardt, and G. Tate. Estimation of dynamic discrete choice models with differentiable temporal-difference learning. *arXiv preprint arXiv:2209.15174*, 2022.

Alekh Agarwal, Sham Kakade, and Lin F. Yang. Model-based reinforcement learning with a generative model is minimax optimal. In *Conference on Learning Theory (COLT)*, 2020a.

Alekh Agarwal, Sham M. Kakade, Jason D. Lee, and Gaurav Mahajan. On the theory of policy gradient methods: Optimality, approximation, and distribution shift. *Journal of Machine Learning Research*, 22(98), 2021.

Rishabh Agarwal, Dale Schuurmans, and Mohammad Norouzi. An optimistic perspective on offline reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2020b.

Shun-ichi Amari. Natural gradient works efficiently in learning. *Neural Computation*, 10(2): 251–276, 1998.

András Antos, Csaba Szepesvári, and Rémi Munos. Learning near-optimal policies with Bellman-residual minimization based fitted policy iteration and a single sample path. *Machine Learning*, 71(1):89–129, 2008. doi: 10.1007/s10994-007-5038-2.

Peter Arcidiacono and Robert A. Miller. Conditional choice probability estimation of dynamic discrete choice models with unobserved heterogeneity. *Econometrica*, 79(6):1823–1867, 2011.

John Asker, Chaim Fershtman, Jihye Jeon, and Ariel Pakes. A computational framework for analyzing dynamic auctions: The market impact of information sharing. *The RAND Journal of Economics*, 51(3):805–839, 2020.

Tohid Atashbar and Shuping Shi. Solving macroeconomic models with deep reinforcement learning. *Journal of Economic Dynamics and Control*, 2023.

Mohammad Gheshlaghi Azar, Rémi Munos, and Hilbert J Kappen. Minimax pac bounds on the sample complexity of reinforcement learning with a generative model. *Machine Learning*, 91(1):7–32, 2013.

Leemon Baird. Residual algorithms: Reinforcement learning with function approximation. In *Proceedings of the Twelfth International Conference on Machine Learning*, pages 30–37. Morgan Kaufmann, 1995.

Andrew G. Barto, Richard S. Sutton, and Charles W. Anderson. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-13(5):834–846, 1983.

Andrew Bennett and Nathan Kallus. Proximal reinforcement learning: Efficient off-policy evaluation in partially observed Markov decision processes. *Operations Research*, 2021.

Dimitri P. Bertsekas. *Dynamic Programming and Optimal Control.* Athena Scientific, 1996.

Dimitri P. Bertsekas. Lessons from AlphaZero for optimal, model predictive, and adaptive control. *Athena Scientific Reports*, 2021.

Dimitri P. Bertsekas. *Abstract Dynamic Programming.* Athena Scientific, Belmont, MA, 3rd edition, 2022a.

Dimitri P. Bertsekas. Newton's method for reinforcement learning and model predictive control. *Results in Control and Optimization*, 7:100121, 2022b.

Dimitri P. Bertsekas and John N. Tsitsiklis. *Neuro-Dynamic Programming.* Athena Scientific, Belmont, MA, 1996.

Jalaj Bhandari, Daniel Russo, and Raghav Singal. A finite time analysis of temporal difference learning with linear function approximation. *Operations Research*, 69(3), 2021.

David Blackwell. Discounted dynamic programming. *Annals of Mathematical Statistics*, 36(1): 226–235, 1965.

Han Bleichrodt, Kirsten I. M. Rohde, and Peter P. Wakker. Koopmans' constant discounting for intertemporal choice: A simplification and a generalization. *Journal of Mathematical Psychology*, 52:341–347, 2008.

Tilman Börgers and Rajiv Sarin. Learning through reinforcement and replicator dynamics. *Journal of Economic Theory*, 77(1):1–14, 1997.

Vivek S. Borkar. Stochastic approximation with two time scales. *Systems & Control Letters*, 29(5):291–294, 1997.

Vivek S. Borkar and Sean P. Meyn. The ODE method for convergence of stochastic approximation and reinforcement learning. *SIAM Journal on Control and Optimization*, 38(2):447–469, 2000.

Michael Bowling and Manuela Veloso. Multiagent learning using a variable learning rate. *Artificial Intelligence*, 136(2):215–250, 2002.

Michael Bowling, Neil Burch, Michael Johanson, and Oskari Tammelin. Heads-up limit hold'em poker is solved. *Science*, 347(6218):145–149, 2015.

Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. The method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952.

Gianluca Brero, Alon Eden, Matthias Gerstgrasser, David C. Parkes, and Duncan Rheingans-Yoo. Reinforcement learning of sequential price mechanisms. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 13662–13670, 2021.

William A. Brock and Leonard J. Mirman. Optimal economic growth and uncertainty: The discounted case. *Journal of Economic Theory*, 4(3):479–513, 1972.

George W. Brown. Iterative solution of games by fictitious play, 1951.

Noam Brown and Tuomas Sandholm. Superhuman ai for heads-up no-limit poker: Libratus beats top professionals. *Science*, 359(6374):418–424, 2018.

Noam Brown and Tuomas Sandholm. Superhuman ai for multiplayer poker. *Science*, 365(6456): 885–890, 2019.

Noam Brown, Adam Lerer, Sam Gross, and Tuomas Sandholm. Deep counterfactual regret minimization. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97, pages 793–802. PMLR, 2019.

Jeremy I. Bulow. Durable-goods monopolists. *Journal of Political Economy*, 90(2):314–332, 1982.

Emilio Calvano, Giacomo Calzolari, Vincenzo Denicolò, and Sergio Pastorello. Artificial intelligence, algorithmic pricing, and collusion. *American Economic Review*, 110(10):3267–3297, 2020.

Shicong Cen, Chen Cheng, Yuxin Chen, Yuting Wei, and Yuejie Chi. Fast global convergence of natural policy gradient methods with entropy regularization. *Operations Research*, 70(4), 2022.

Khai Xiang Chiong, Alfred Galichon, and Matt Shum. Duality in dynamic discrete-choice models. *Quantitative Economics*, 7(1):83–115, 2016.

Paul F. Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Pierre Clavier, Erwan Le Pennec, and Matthieu Geist. Towards minimax optimality of model-based robust reinforcement learning. In *Conference on Uncertainty in Artificial Intelligence (UAI)*, 2024.

Ronald H. Coase. Durability and monopoly. *Journal of Law and Economics*, 15(1):143–149, 1972.

Matias Covarrubias. Dynamic oligopoly and monetary policy: A deep reinforcement learning approach. Job Market Paper, New York University, 2022.

Cristiano da Costa Cunha, Wei Liu, Tim French, and Ajmal Mian. Unifying causal reinforcement learning: Survey, taxonomy, algorithms and applications. *arXiv preprint arXiv:2512.18135*, 2025.

Constantinos Daskalakis, Paul W. Goldberg, and Christos H. Papadimitriou. The complexity of computing a nash equilibrium. *SIAM Journal on Computing*, 39(1):195–259, 2009.

Peter Dayan. The convergence of TD(λ) for general λ. *Machine Learning*, 8(3–4):341–362, 1992.

Eric V. Denardo. Contraction mappings in the theory underlying dynamic programming. *SIAM Review*, 9(2):165–177, 1967.

Zhihong Deng, Jing Jiang, Guodong Long, and Chengqi Zhang. Causal reinforcement learning: A survey. *Transactions on Machine Learning Research*, 2023.

Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7 (1):1–26, 1979.

Damien Ernst, Pierre Geurts, and Louis Wehenkel. Tree-based batch mode reinforcement learning. *Journal of Machine Learning Research*, 6:503–556, 2005.

Lasse Espeholt, Hubert Soyer, Rémi Munos, Karen Simonyan, Volodymyr Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, Shane Legg, and Koray Kavukcuoglu. IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In *International Conference on Machine Learning*, 2018.

Eyal Even-Dar and Yishay Mansour. Learning rates for q-learning. *Journal of Machine Learning Research*, 5:1–25, 2003.

John Fearnley. Exponential lower bounds for policy iteration. In *International Colloquium on Automata, Languages, and Programming (ICALP)*, pages 551–562, 2010.

Mattie Fellows, Matthew J. A. Smith, and Shimon Whiteson. Why target networks stabilise temporal difference methods. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 9886–9909. PMLR, 2023.

Jesús Fernández-Villaverde, Galo Nuño, and Jesse Perla. Taming the curse of dimensionality: Quantitative economics with deep learning. Working Paper 33117, National Bureau of Economic Research, 2024.

Chaim Fershtman and Ariel Pakes. Dynamic games with asymmetric information: A framework for empirical work. *The Quarterly Journal of Economics*, 127(4):1611–1661, 2012.

Mogens Fosgerau and Jesper R.-V. Sørensen. How McFadden met Rockafellar and learned to do more with less. *Journal of Mathematical Economics*, 100:102629, 2022.

Scott Fujimoto, David Meger, and Doina Precup. Off-policy deep reinforcement learning without exploration. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2052–2062. PMLR, 2019.

Matthieu Geist, Bruno Scherrer, and Olivier Pietquin. A theory of regularized Markov decision processes. In *International Conference on Machine Learning (ICML)*, 2019.

Christian Gourieroux, Alain Monfort, and Eric Renault. Indirect inference. *Journal of Applied Econometrics*, 8(S1):S85–S118, 1993.

Amy Greenwald and Keith Hall. Correlated q-learning. In *Proceedings of the 20th International Conference on Machine Learning*, pages 242–249, 2003.

Faruk Gul, Hugo Sonnenschein, and Robert Wilson. Foundations of dynamic monopoly and the Coase conjecture. *Journal of Economic Theory*, 39(1):155–190, 1986.

Tuomas Haarnoja, Haoran Tang, Pieter Abbeel, and Sergey Levine. Reinforcement learning with deep energy-based policies. In *Proceedings of the 34th International Conference on Machine Learning*, 2017.

Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *Proceedings of the 35th International Conference on Machine Learning*, pages 1861–1870, 2018.

Ben Hambly, Renyuan Xu, and Huining Yang. Recent advances in reinforcement learning in finance. *Mathematical Finance*, 33(3):437–503, 2023.

Johannes Heinrich and David Silver. Deep reinforcement learning from self-play in imperfect-information games. *arXiv preprint arXiv:1603.01121*, 2016.

Brett Hollenbeck. Horizontal mergers and innovation in concentrated industries. *Quantitative Marketing and Economics*, 2019.

V. Joseph Hotz and Robert A. Miller. Conditional choice probabilities and the estimation of dynamic models. *The Review of Economic Studies*, 60(3):497–529, 1993.

Ronald A. Howard. *Dynamic Programming and Markov Processes*. The Technology Press of M.I.T. and John Wiley and Sons, New York, NY, 1960.

Junling Hu and Michael P Wellman. Nash q-learning for general-sum stochastic games. *Journal of Machine Learning Research*, 4(Nov):1039–1069, 2003.

Yingyao Hu and Matthew Shum. Nonparametric identification of dynamic models with unobserved state variables. *Journal of Econometrics*, 171(1):32–44, 2012.

Yingyao Hu and Fangzhu Yang. Estimation of dynamic discrete choice models with unobserved state variables using reinforcement learning. *Working Paper, Johns Hopkins University*, 2025.

Mitsuru Igami. Artificial intelligence as structural estimation: Deep blue, bonanza, and alphago. *The Econometrics Journal*, 23(3):S1–S24, 2020.

Fedor Iskhakov, John Rust, and Bertel Schjerning. Machine learning and structural econometrics: Contrasts and synergies. *The Econometrics Journal*, 23(S1):S81–S124, 2020.

Tommi Jaakkola, Michael I. Jordan, and Satinder P. Singh. On the convergence of stochastic iterative dynamic programming algorithms. In *Advances in Neural Information Processing Systems 6*, pages 703–710. Morgan Kaufmann, 1994.

Ying Jin, Zhuoran Yang, and Zhaoran Wang. Is pessimism provably efficient for offline RL? In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 5084–5096. PMLR, 2021.

Sham M. Kakade. A natural policy gradient. *Advances in Neural Information Processing Systems*, 14, 2001.

Sham M. Kakade. *A Natural Policy Gradient*. PhD thesis, University College London, 2002.

Nathan Kallus and Angela Zhou. Confounding-robust policy evaluation in infinite-horizon reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

Michael Kearns, Yishay Mansour, and Andrew Y. Ng. A sparse sampling algorithm for near-optimal planning in large Markov decision processes. *Machine Learning*, 49(2–3):193–208, 2002.

David L. Kleinman. On an iterative technique for Riccati equation computations. *IEEE Transactions on Automatic Control*, 13(1):114–115, 1968.

Vijay R. Konda and John N. Tsitsiklis. Actor-critic algorithms. In *Advances in Neural Information Processing Systems 12*, pages 1008–1014. MIT Press, 2000.

Tjalling C. Koopmans. Stationary ordinal utility and impatience. *Econometrica*, 28:287–309, 1960.

Tomasz Korbak, Ethan Perez, and Christopher L. Buckley. RL with KL penalties is better viewed as Bayesian inference. *arXiv preprint arXiv:2205.11275*, 2022.

Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit Q-Learning. In *International Conference on Learning Representations*, 2022.

Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative Q-Learning for offline reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

Harold J. Kushner and Dean S. Clark. *Stochastic Approximation Methods for Constrained and Unconstrained Systems*. Springer-Verlag, New York, 1978.

Tze Leung Lai and Herbert Robbins. Asymptotically efficient adaptive allocation rules. *Advances in Applied Mathematics*, 6(1):4–22, 1985.

Sascha Lange, Thomas Gabel, and Martin Riedmiller. Batch reinforcement learning. *Reinforcement Learning: State-of-the-Art*, pages 45–73, 2012.

Sergey Levine. Reinforcement learning and control as probabilistic inference: Tutorial and review. *arXiv preprint arXiv:1805.00909*, 2018.

Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*, 2020.

Arthur Lewbel. The identification zoo: Meanings of identification in econometrics. *Journal of Economic Literature*, 57(4):835–903, 2019.

Gen Li, Yuting Wei, Yuejie Chi, and Yuxin Chen. Softmax policy gradient methods can take exponential time to converge. *Mathematical Programming*, 196:579–632, 2022.

Gen Li, Laixi Shi, Yuxin Chen, Yuting Wei, and Yuejie Chi. Is q-learning minimax optimal? a tight sample complexity analysis. *Operations Research*, 72(1), 2024a.

Gen Li, Yuting Wei, Yuejie Chi, Yuantao Gu, and Yuxin Chen. Breaking the sample size barrier in model-based reinforcement learning with a generative model. *Operations Research*, 72(1), 2024b.

Luofeng Liao, Zuyue Fu, Zhuoran Yang, Yixin Wang, Dingli Ma, Mladen Kolar, and Zhaoran Wang. Instrumental variable value iteration for causal offline reinforcement learning. *Journal of Machine Learning Research*, 25(303):1–56, 2024.

Han-Dong Lim and Donghwan Lee. Regularized q-learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Long-Ji Lin. *Self-Improving Reactive Agents Based on Reinforcement Learning, Planning and Teaching*. PhD thesis, Carnegie Mellon University, 1992. Also published in *Machine Learning*, 8(3–4):293–321, 1992.

Michael L Littman. Markov games as a framework for multi-agent reinforcement learning. In *Machine Learning Proceedings 1994*, pages 157–163. Morgan Kaufmann, 1994.

Michael L. Littman. Friend-or-foe q-learning in general-sum games. In *Proceedings of the 18th International Conference on Machine Learning*, pages 322–328, 2001.

Nikolay Lomys and Luca Magnolfi. Estimation of games under no regret: Structural econometrics for ai. Working Paper NET Institute Working Paper No. 24-05, Social Science Research Network (SSRN), 2024. Available at SSRN: https://ssrn.com/abstract=4717195 or http://dx.doi.org/10.2139/ssrn.4717195.

Lilia Maliar, Serguei Maliar, and Pablo Winant. Deep learning for solving dynamic economic models. *Journal of Monetary Economics*, 122:76–101, 2021.

Alan S. Manne. Linear programming and sequential decisions. *Management Science*, 6(3): 259–267, 1960.

Albert Marcet and Thomas J. Sargent. Convergence of least squares learning mechanisms in self-referential linear stochastic models. *Journal of Economic Theory*, 48(2):337–368, 1989.

Amir massoud Farahmand, Rémi Munos, and Csaba Szepesvári. Error propagation for approximate policy and value iteration. In *Advances in Neural Information Processing Systems 22 (NeurIPS)*, pages 568–576, 2010.

Daniel McFadden. Conditional logit analysis of qualitative choice behavior. *Frontiers in Econometrics*, pages 105–142, 1974.

Daniel McFadden. Modeling the choice of residential location. In Anders Karlqvist, Lars Lundqvist, Folke Snickars, and Jörgen W. Weibull, editors, *Spatial Interaction Theory and Planning Models*, pages 75–96. North-Holland, 1978.

Jincheng Mei, Chenjun Xiao, Csaba Szepesvari, and Dale Schuurmans. On the global convergence rates of softmax policy gradient methods. In *International Conference on Machine Learning*, 2020.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.

Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning*, pages 1928–1937. PMLR, 2016.

Thomas M. Moerland, Joost Broekens, Aske Plaat, and Catholijn M. Jonker. Model-based reinforcement learning: A survey. *Foundations and Trends in Machine Learning*, 2023.

Benjamin Moll. The trouble with rational expectations in heterogeneous agent models: A challenge for macroeconomics. *The Economic Journal*, 2025. doi: 10.1093/ej/ueaf104.

Rémi Munos and Csaba Szepesvári. Finite-time bounds for fitted value iteration. *Journal of Machine Learning Research*, 9:815–857, 2008.

Rémi Munos and Csaba Szepesvári. Finite-time bounds for fitted value iteration. *Journal of Machine Learning Research*, 9(27):815–857, 2008.

Rémi Munos, Tom Stepleton, Anna Harutyunyan, and Marc G. Bellemare. Safe and efficient off-policy reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 29, 2016.

Michael Oberst and David Sontag. Counterfactual off-policy evaluation with Gumbel-Max structural causal models. In *Proceedings of the 36th International Conference on Machine Learning*, pages 4923–4932, 2019.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744, 2022.

Ariel Pakes and Paul McGuire. Computing Markov-perfect Nash equilibria: Numerical implications of a dynamic differentiated product model. *RAND Journal of Economics*, 25(4): 555–589, 1994.

Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, Cambridge, 2nd edition, 2009.

Silviu Pitis. Rethinking the discount factor in reinforcement learning: A decision theoretic approach. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, 2019.

Moshe A. Pollatschek and Benjamin Avi-Itzhak. Algorithms for stochastic games with geometrical interpretation. *Management Science*, 15(7):399–415, 1969.

Martin L. Puterman and Shelby L. Brumelle. On the convergence of policy iteration in stationary dynamic programming. *Mathematics of Operations Research*, 4(1):60–69, 1979.

Marek Pycia and Peter Troyan. A theory of simplicity in games and mechanism design. *Econometrica*, 91(4):1495–1526, 2023.

Liqun Qi and Jie Sun. A nonsmooth version of Newton's method. *Mathematical Programming*, 58(1–3):353–367, 1993.

Guannan Qu and Adam Wierman. Finite-time analysis of asynchronous stochastic approximation and q-learning. *Journal of Machine Learning Research*, 21(1):1–28, 2020.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Paria Rashidinejad, Banghua Zhu, Cong Ma, Jiantao Jiao, and Stuart Russell. Bridging offline reinforcement learning and imitation learning: A tale of pessimism. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

Sai Srivatsa Ravindranath, Zhe Feng, Di Wang, Manzil Zaheer, Aranyak Mehta, and David C. Parkes. Deep reinforcement learning for sequential combinatorial auctions. Submitted to ICLR 2025, 2024. Available at `https://openreview.net/forum?id=SVd9Ffcdp8`.

Pranjal Rawat. Designing auctions when algorithms learn to bid. *Working Paper*, 2026.

Herbert Robbins. Some aspects of the sequential design of experiments. *Bulletin of the American Mathematical Society*, 58(5):527–535, 1952.

Stéphane Ross and J. Andrew Bagnell. Agnostic system identification for model-based reinforcement learning. In *Proceedings of the 29th International Conference on Machine Learning*, 2012.

G. A. Rummery and M. Niranjan. On-line q-learning using connectionist systems. Technical report, Cambridge University, 1994.

John Rust. Optimal replacement of gmc bus engines: An empirical model of harold zurcher. *Econometrica*, 55(5):999–1033, 1987.

John Rust. Structural estimation of Markov decision processes. In Robert F. Engle and Daniel McFadden, editors, *Handbook of Econometrics*, volume 4, chapter 51, pages 3081–3143. Elsevier, 1994.

John Rust. Numerical dynamic programming in economics. In Hans M. Amman, David A. Kendrick, and John Rust, editors, *Handbook of Computational Economics*, volume 1, pages 619–729. Elsevier, 1996.

John Rust and Pranjal Rawat. Structural econometrics and inverse reinforcement learning: Inferring preferences and beliefs from human behavior. Working Paper, Georgetown University, 2026.

Manuel S. Santos and John Rust. Convergence properties of policy iteration. *SIAM Journal on Control and Optimization*, 42(6):2094–2115, 2004.

Thomas J. Sargent. *Bounded Rationality in Macroeconomics*. Oxford University Press, 1993.

John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. *Proceedings of the 32nd International Conference on Machine Learning*, pages 1889–1897, 2015.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Lior Shani, Yonathan Efroni, and Shie Mannor. Adaptive trust region policy optimization: Global convergence and faster rates for regularized MDPs. In *AAAI Conference on Artificial Intelligence*, 2020.

Lloyd S. Shapley. Stochastic games. *Proceedings of the National Academy of Sciences*, 39(10): 1095–1100, 1953.

Lloyd S. Shapley. Some topics in two-person games. In M. Dresher, L. S. Shapley, and A. W. Tucker, editors, *Advances in Game Theory*, pages 1–28. Princeton University Press, 1964.

Yoav Shoham, Rob Powers, and Trond Grenager. If multi-agent learning is the answer, what is the question? *Artificial Intelligence*, 171(7):365–377, 2007.

Aaron Sidford, Mengdi Wang, Xian Wu, Lin F. Yang, and Yinyu Ye. Near-optimal time and sample complexities for solving Markov decision processes with a generative model. In *Advances in Neural Information Processing Systems*, 2018.

David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.

David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of go without human knowledge. *Nature*, 550(7676):354–359, 2017.

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.

Satinder Singh, Tommi Jaakkola, Michael L Littman, and Csaba Szepesvári. Convergence results for single-step on-policy reinforcement-learning algorithms. *Machine learning*, 38: 287–308, 2000.

Satinder P. Singh and Richard C. Yee. An upper bound on the loss from approximate optimal-value functions. *Machine Learning*, 16(3):227–233, 1994.

Nancy L. Stokey. Rational expectations and durable goods pricing. *Bell Journal of Economics*, 12(1):112–128, 1981.

Richard S. Sutton. Learning to predict by the methods of temporal differences. *Machine Learning*, 3(1):9–44, 1988. doi: 10.1023/A:1022633531479.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, 2nd edition, 2018.

Richard S. Sutton, David A. McAllester, Satinder P. Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. *Advances in Neural Information Processing Systems*, 12, 1999.

Richard S. Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 2000.

Richard S. Sutton, Hamid Reza Maei, Doina Precup, Shalabh Bhatnagar, David Silver, Csaba Szepesvári, and Eric Wiewiora. Fast gradient-descent methods for temporal-difference learning with linear function approximation. In *Proceedings of the 26th International Conference on Machine Learning*, pages 993–1000. ACM, 2009.

Oskari Tammelin. Solving large imperfect information games using cfr+. *arXiv preprint arXiv:1407.5042*, 2014.

Gerald Tesauro. Td-gammon, a self-teaching backgammon program, achieves master-level play. *Neural Computation*, 6(2):215–219, 1994.

Haoxing Tian, Ioannis Ch. Paschalidis, and Alex Olshevsky. Convergence of actor-critic methods with multi-layer neural networks. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.

Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society, Series B*, 58(1):267–288, 1996.

Emanuel Todorov. Linearly-solvable Markov decision problems. In *Advances in Neural Information Processing Systems*, volume 19, 2006.

Nenad Tomasev, Ulrich Paquet, Demis Hassabis, and Vladimir Kramnik. Assessing game balance with AlphaZero: Exploring alternative rule sets in chess. *arXiv preprint arXiv:2009.04374*, 2020.

John N. Tsitsiklis. Asynchronous stochastic approximation and q-learning. *Machine Learning*, 16(3):185–202, 1994. doi: 10.1007/bf00993306.

John N. Tsitsiklis. On the convergence of optimistic policy iteration. *Journal of Machine Learning Research*, 3:59–72, 2002.

John N. Tsitsiklis and Benjamin Van Roy. Analysis of temporal-difference learning with function approximation. In *Advances in Neural Information Processing Systems 9 (NIPS)*, 1997.

Martin J. Wainwright. Stochastic approximation with cone-contractive operators: Sharp ℓ_∞-bounds for Q-learning. *Annals of Statistics*, 47(6):3168–3197, 2019.

Yue Wang, Tomasz Żak, and Csaba Szepesvári. Near-optimal sample complexity for iterated CVaR reinforcement learning with a generative model. *arXiv preprint arXiv:2503.08934*, 2025.

Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.

Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.

Ronald J. Williams and Jing Peng. Function optimization using connectionist reinforcement learning algorithms. *Connection Science*, 3(3):241–268, 1991.

Yue Wu, Weitong Zhang, Pan Xu, and Quanquan Gu. A finite-time analysis of two time-scale actor-critic methods. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Lin Xiao. On the convergence rates of policy gradient methods. *Journal of Machine Learning Research*, 23, 2022.

Yinyu Ye. The simplex and policy-iteration methods are strongly polynomial for the Markov decision problem with a fixed discount rate. *Mathematics of Operations Research*, 36(4): 593–603, 2011.

Andrea Zanette. Exponential lower bounds for batch reinforcement learning: Batch RL can be exponentially harder than online RL. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*. PMLR, 2021.

Junzhe Zhang and Elias Bareinboim. Near-optimal reinforcement learning in dynamic treatment regimes. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

Junzhe Zhang and Elias Bareinboim. Designing optimal dynamic treatment regimes: A causal reinforcement learning approach. In *Proceedings of the 37th International Conference on Machine Learning*, pages 11012–11022, 2020.

Shangtong Zhang, Hengshuai Yao, and Shimon Whiteson. Breaking the deadly triad with a target network. In *International Conference on Machine Learning (ICML)*, 2021.

Weipeng Zhang. Distributed randomized multiagent policy iteration in reinforcement learning. *Results in Control and Optimization*, 12:100255, 2023.

Shiyu Zhao. *Mathematical Foundations of Reinforcement Learning*. Springer, 2025. doi: 10.1007/978-981-97-3944-8.

Stephan Zheng, Alexander Trott, Sunil Srinivasa, David C. Parkes, and Richard Socher. The AI economist: Taxation policy design via two-level deep multiagent reinforcement learning. *Science Advances*, 8(18):eabk2607, 2022.

Brian D. Ziebart, Andrew Maas, J. Andrew Bagnell, and Anind K. Dey. Maximum entropy inverse reinforcement learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2008.

Martin Zinkevich, Michael Johanson, Michael Bowling, and Carmelo Piccione. Regret minimization in games with incomplete information. In *Advances in Neural Information Processing Systems*, volume 20, 2008.

A Glossary of Acronyms and Terms

Acronyms

A2C/A3C	Advantage Actor-Critic / Asynchronous Advantage Actor-Critic. Policy gradient algorithms that use an advantage function baseline to reduce variance (Section 2).
BCQ	Batch-Constrained Q-learning. An offline RL algorithm that restricts the learned policy to actions supported by the behavioral data (Section 6.2).
CCP	Conditional Choice Probabilities. Choice probabilities conditional on state, used in structural estimation as an alternative to full-solution methods (Section 4).
CFR	Counterfactual Regret Minimization. An iterative algorithm for computing Nash equilibria in extensive-form games by minimizing regret at each information set (Section 5).
CQL	Conservative Q-Learning. An offline RL algorithm that adds a penalty for overestimating Q-values on out-of-distribution actions (Section 6.2).
DDC	Dynamic Discrete Choice. A class of structural econometric models in which agents make sequential discrete decisions under uncertainty (Section 4).
DDPG	Deep Deterministic Policy Gradient. An actor-critic algorithm for continuous action spaces that combines a deterministic policy gradient with a learned Q-function (Section 2).
DP	Dynamic Programming. A collection of algorithms that compute optimal policies by solving the Bellman equation, given a known model of the environment (Section 2).

DPO	Direct Preference Optimization. A method that optimizes a language model directly on preference data without training a separate reward model (Section 6.4).
DQN	Deep Q-Network. Q-learning with a neural network function approximator, target network, and experience replay (Section 2).
FQI/FVI	Fitted Q-Iteration / Fitted Value Iteration. Approximate dynamic programming algorithms that use supervised learning to fit value functions from batch data (Section 3.2.4).
GLIE	Greedy in the Limit with Infinite Exploration. A condition on exploration schedules ensuring convergence to the optimal policy (Section 2).
GMM	Generalized Method of Moments. An econometric estimation method that matches sample moments to population moment conditions (Section 4).
IQL	Implicit Q-Learning. An offline RL algorithm that avoids querying out-of-distribution actions by using expectile regression on the value function (Section 6.2).
IRL	Inverse Reinforcement Learning. The problem of inferring a reward function from observed behavior, assuming the agent acts approximately optimally (Section 2).
IV	Instrumental Variables. An econometric technique for identifying causal effects in the presence of endogeneity by exploiting exogenous variation.
KL	Kullback-Leibler divergence. A measure of how one probability distribution diverges from a reference distribution, used in trust-region and regularization methods (Section 2).
LLM	Large Language Model. A neural network trained on large text corpora, the primary object of alignment via RLHF and DPO (Section 6.4).
LQR	Linear-Quadratic Regulator. An optimal control method that computes the policy for linear dynamics with a quadratic cost function via the Riccati equation.
MARL	Multi-Agent Reinforcement Learning. RL in settings with multiple interacting agents, where each agent's optimal policy depends on the others' behavior (Section 5).
MC	Monte Carlo. Methods that estimate value functions from complete episode returns rather than bootstrapped estimates (Section 2).
MDP	Markov Decision Process. A formal model of sequential decision-making defined by states, actions, transition probabilities, rewards, and a discount factor (Section 2).
MLE	Maximum Likelihood Estimation. A statistical method that estimates parameters by maximizing the likelihood of the observed data (Section 4).
MPC	Model Predictive Control. A control method that solves a finite-horizon optimization problem at each timestep using an explicit model of the dynamics, then applies only the first action (Section 1.1).
NE	Nash Equilibrium. A strategy profile in which no player can improve their payoff by unilaterally deviating (Section 5).
NFXP	Nested Fixed Point. Rust's algorithm for structural estimation of DDC models that nests value function iteration inside a maximum likelihood loop (Section 4).
NPG	Natural Policy Gradient. A policy gradient method that preconditions the gradient by the inverse Fisher information matrix (Section 2).

PEVI — Pessimistic Value Iteration. An offline RL algorithm that subtracts an uncertainty penalty from value estimates to avoid overestimation on unseen state-action pairs (Section 6).

PI — Policy Iteration. A dynamic programming algorithm that alternates between policy evaluation and policy improvement until convergence (Section 2).

PID — Proportional-Integral-Derivative controller. A feedback controller that computes a control signal from the weighted sum of the current error, its integral, and its derivative (Section 1.1).

POMDP — Partially Observable MDP. An MDP in which the agent cannot directly observe the state and must maintain beliefs from noisy observations.

PPO — Proximal Policy Optimization. A policy gradient algorithm that constrains updates to a trust region via a clipped surrogate objective (Section 2).

RL — Reinforcement Learning. A framework for sequential decision-making in which an agent learns a policy by interacting with an environment and observing rewards (Section 2).

RLHF — Reinforcement Learning from Human Feedback. A framework for aligning model behavior with human preferences by training a reward model from pairwise comparisons and optimizing a policy against it (Section 6.4).

SAC — Soft Actor-Critic. An off-policy actor-critic algorithm that maximizes a combined objective of expected return and policy entropy (Section 2).

SARSA — State-Action-Reward-State-Action. An on-policy TD control algorithm that updates Q-values using the action actually taken in the next state (Section 2).

SFT — Supervised Fine-Tuning. The initial stage of LLM alignment in which the model is trained on curated demonstrations before preference optimization (Section 6.4).

SPE — Subgame Perfect Equilibrium. A refinement of Nash equilibrium requiring that strategies constitute a Nash equilibrium in every subgame (Section 5).

TD — Temporal Difference. A class of prediction and control algorithms that update value estimates using bootstrapped targets rather than complete returns (Section 3.2).

TRPO — Trust Region Policy Optimization. A policy gradient algorithm that constrains each update to lie within a KL-divergence trust region of the previous policy (Section 2).

UCB — Upper Confidence Bound. An algorithm that selects the arm with the highest sum of empirical mean and an exploration bonus (Section 2).

VI — Value Iteration. A dynamic programming algorithm that iteratively applies the Bellman optimality operator until the value function converges (Section 2).